CS 1261: Fairness, Privacy, and Cryptography
Lecture Notes

Course Taught by Professor Cynthia Dwork
Notes Compiled by Cristiana Murgoci

Spring 2026

Contents

Chapter 1

Three Flavors of Indistinguishability

1.1 Statistical Indistinguishability

The course opens by comparing two coins:

- Coin 1: $p = 1/2$
- Coin 2: $p = 1/2 + b$

1.1.1 Distinguishing Biased Coins

Let $X \sim \text{Binomial}(n, p)$. Then:

$$\mathbb{E}[X] = pn, \qquad \text{Var}(X) = p(1-p)n.$$

By Chernoff bounds,

$$\Pr[|X - \mathbb{E}[X]| \geq t\sigma] \leq e^{-\Theta(t^2)}.$$

Theorem 1.1 (Sample Complexity of Bias Detection)**.** *To distinguish a coin of bias b from a fair coin with constant confidence,*

$$n = \Theta\left(\frac{1}{b^2}\right)$$

samples are necessary and sufficient.

Insight: Statistical indistinguishability is governed by variance scaling as $\sqrt{n}$.

1.2 Differential Privacy and Adjacent Datasets

Two datasets x, y are *adjacent* if they differ in exactly one individual's record. Concretely: x might be everyone in the room, and y might be everyone in the room plus one additional person. Only that one record differs; everything else is identical.

Why adjacency? The goal of differential privacy is to protect *individuals*: to ensure that no single person's decision to contribute their data can meaningfully change what an observer learns. Defining privacy with respect to adjacent datasets captures this precisely.

Why randomized mechanisms? Differential privacy necessarily involves randomized algorithms. For a given input, a randomized mechanism does not produce the same output every time — it produces a *probability distribution* over outputs. Privacy is then a statement about these distributions: the distributions on adjacent inputs must be nearly identical.

Definition 1.1 (ε-Differential Privacy)**.** *A randomized mechanism M satisfies ε-*differential privacy *if for all adjacent datasets x, y and all measurable output sets S:*

$$\Pr[M(x) \in S] \;\leq\; e^{\varepsilon} \cdot \Pr[M(y) \in S].$$

Interpretation. Every possible transcript of an interaction with M is essentially equally likely whether or not any given individual is present in the dataset. An observer who sees the output of M cannot reliably infer whether a specific person's data was included.

Statistical vs. computational protection. DP is a *statistical* guarantee: it is indistinguishability in the face of unlimited computational power. Unlike cryptographic notions, DP does not rely on hardness assumptions. The protection comes from how many observations the adversary gets — running the mechanism more times leaks more — not from the adversary's computational limitations. Even an adversary with infinite computing power who knows every other person's data cannot determine whether a target individual participated.

Real-world motivation.

- **US Census.** The Census Bureau must release aggregate statistics about the population while protecting respondents' individual records.
- **Medical research.** Studies on shared datasets (e.g., the Framingham Heart Study) require guarantees that individual patient data remains private even as the data is used for research by many groups over many years.
- **Industry.** Google applied DP to mobility data analysis in a Scandinavian country so that engineers could analyze aggregate trajectories without being exposed to individual movement histories — making the analysis both privacy-preserving and more comfortable for the analysts themselves.

1.3 Computational Indistinguishability

In cryptography, indistinguishability is restricted to polynomial-time adversaries. The key definitions below are from Barak's lecture notes (§2.4).

Definition 1.1: Concrete Indistinguishability (Barak Def. 2.9)

Random variables X, Y over $\{0,1\}^n$ are (T, ε)-indistinguishable, written $X \approx_{T,\varepsilon} Y$, if for every algorithm (circuit) D of size at most T:

$$\big|\Pr[D(X) = 1] - \Pr[D(Y) = 1]\big| \;\leq\; \varepsilon.$$

D is called a *distinguisher*. T bounds its running time; ε bounds its *advantage*.

The indistinguishability game. An adversary Eve tries to tell X from Y. A challenger flips a hidden bit $b \in \{0,1\}$, samples from X if b=0 or Y if b=1, and gives the sample to Eve. Eve outputs a guess b'. The scheme is *secure* if

$$\Pr[\text{Eve wins}] = \Pr\big[b' = b\big] \;\le\; \tfrac{1}{2} + \tfrac{\varepsilon}{2},$$

equivalently, Eve's *advantage* $|\Pr[b'{=}1 \mid b{=}0] - \Pr[b'{=}1 \mid b{=}1]| \le \varepsilon$.

Definition 1.2: Asymptotic (Ensemble) Indistinguishability (Barak Def. 2.10)

Ensembles $\{X_n\}$ and $\{Y_n\}$ are *computationally indistinguishable*, written $\{X_n\} \approx \{Y_n\}$, if for every polynomial $p(\cdot)$ and all sufficiently large n:

$$X_n \;\approx_{p(n),\,1/p(n)}\; Y_n.$$

Equivalently, no PPT adversary can distinguish X_n from Y_n with non-negligible advantage.

Properties. Computational indistinguishability ($\approx$) satisfies:

- *Symmetry*: $X \approx Y \Rightarrow Y \approx X$.
- *Transitivity*: $X \approx Y$ and $Y \approx Z \Rightarrow X \approx Z$ (proved by a hybrid argument).
- *Closure under efficient computation*: if $X \approx Y$ and f is poly-time, then $f(X) \approx f(Y)$.

Unlike DP, computational indistinguishability:

- quantifies over efficient algorithms only;
- relies on hardness assumptions (not information-theoretic).

1.3.1 Pseudorandom Generators: A Concrete Example

The canonical application of computational indistinguishability is the *pseudorandom generator (PRG)*.

Setup. A PRG $G : \{0,1\}^n \to \{0,1\}^{2n}$ takes an n-bit *seed* and deterministically produces a $2n$-bit *output string*. (The stretch factor of 2 is for concreteness; any polynomial stretch works.) The machinery of G is completely deterministic — the only randomness is in the choice of the seed.

How many pseudorandom strings are there? There are 2^n possible seeds, so there are at most 2^n possible outputs — a tiny fraction of all 2^{2n} strings of length $2n$. A truly random string of length $2n$ almost certainly falls *outside* the image of G, because $|\mathrm{Im}(G)|/2^{2n} \le 2^n/2^{2n} = 2^{-n}$.

The indistinguishability requirement. An exponential-time distinguisher could enumerate all 2^n outputs of G, check membership, and distinguish pseudorandom from truly random strings easily. The requirement is therefore restricted to *polynomial-time* distinguishers:

Definition 1.3: Pseudorandom Generator (PRG)

A deterministic function $G : \{0,1\}^n \to \{0,1\}^{2n}$ is a *PRG* if the ensemble $\{G(U_n)\}$ is computationally indistinguishable from $\{U_{2n}\}$:

$$\{G(U_n)\}_{n\ge1} \;\approx\; \{U_{2n}\}_{n\ge1},$$

where U_k denotes the uniform distribution on $\{0,1\}^k$. No polynomial-time algorithm can distinguish a pseudorandom string from a truly random string with non-negligible advantage.

PRGs are the building block for stream ciphers and are studied in depth in Chapter 14.

1.4 The Third Flavor: Outcome Indistinguishability (Preview)

The chapter title promises three flavors. The third arises in the context of *prediction algorithms* and *algorithmic fairness*.

The problem of evaluating prediction algorithms. Prediction algorithms assign probabilities to events: the probability that it will rain tomorrow, that a patient's tumor will metastasize, that a person will repay a loan. Evaluating such algorithms is fundamentally difficult because these are *non-repeatable events* — we cannot rewind the future, re-run it many times, and measure the frequency of outcomes. There is no universally agreed mathematical definition of the probability of a non-repeatable event.

Calibration: necessary but not sufficient. One natural criterion is *calibration* (Dawid, 1982): a forecaster is calibrated if, among all instances on which it predicted probability p, the fraction of true outcomes is approximately p. For example, among all days on which the forecaster said "70% chance of rain," it should have actually rained on approximately 70% of those days.

Calibration is clearly desirable. Alarmingly, however, it is achievable by a complete charlatan who knows nothing about the phenomenon being predicted — one can construct a perfectly calibrated forecaster that is entirely uninformative. This shows that calibration alone is not a meaningful guarantee of accuracy or fairness.

The outcome indistinguishability framework. Inspired by Feynman's account of how physics validates a new law — build an experiment to test whether the law's *computed consequences* match *reality*, and if they don't, the law is wrong regardless of its elegance — we want a richer notion of what it means for a model's predictions to be valid.

Outcome Indistinguishability (informal)

A model $\tilde{p}$ is *outcome indistinguishable* with respect to a class of distinguishers $\mathcal{D}$ if no test $D \in \mathcal{D}$ can tell the difference between:

- outcomes drawn from the *real world* (true distribution), and
- outcomes drawn from the *model's distribution* $\tilde{p}$.

The class of distinguishers $\mathcal{D}$ encodes what kinds of tests we care about. By varying $\mathcal{D}$, one recovers:

- simple calibration (distinguishers that only see the predicted probability and the outcome);
- group-conditional calibration (distinguishers that also see demographic attributes);
- complex fairness criteria (distinguishers with access to richer features).

This gives a *hierarchy* of fairness notions ordered by distinguisher power, and connects algorithmic fairness directly to the indistinguishability framework used throughout the course. The full theory is developed in Chapter 4.

Summary: three flavors.

Flavor	**Limitation on distinguisher**	**Application**
Statistical	Number of samples	Differential Privacy
Computational	Polynomial time	Cryptography, PRGs
Outcome	Specified test class	Algorithmic Fairness

Chapter 2

Algorithmic Fairness

2.1 Representation and Bias

Algorithms never operate on individuals directly. Instead, each individual is mapped through a *representation function* $\rho : \mathcal{X} \to \mathcal{Z}$ that extracts a finite set of features. The algorithm sees only $\rho(x)$, not the full person x. This compression is a fundamental source of bias.

Choice of features matters. Consider a cancer study. Two research groups each study whether a particular treatment causes metastasis. Group A examines positions $\{r_1, \ldots, r_k\}$ in a tumor's DNA and finds a 30% metastasis rate among patients who look like you in those positions. Group B examines positions $\{b_1, \ldots, b_m\}$ and finds a 5% rate. What can we conclude about your prognosis? Nothing — because the two groups studied *different projections* of your tumor. Only a study examining both sets of features simultaneously would be informative. Whenever the representation matters this much, it is a vector for the introduction of bias.

2.1.1 Sources of Bias

Non-representative training data. A classifier trained to minimize error on a distribution $\mathcal{D}$ only generalizes to *that distribution*. If a minority group constitutes a small fraction of $\mathcal{D}$, near-zero error overall can be achieved by performing arbitrarily badly on the minority while doing well on the majority. The algorithm has no incentive to be accurate on groups that barely affect aggregate loss.

Biased labels. Labels themselves may encode historical discrimination. If the same behavior by a child of color is systematically coded as "defiant" while an identical behavior by a white child is coded as "spirited," the training data absorbs this bias. The algorithm then learns to replicate it. Similarly, *rearrest* is used as a proxy for recidivism in criminal justice tools, but rearrest rates reflect policing intensity and systemic inequality, not solely individual behavior. People who commit crimes may not be rearrested; people who are arrested may not have committed a crime.

Differentially expressive features. The same feature value can mean different things for different populations. The Allegheny County Child Protective Services screening tool (Virginia Eubanks, *Automating Inequality*, 2014) accessed Medicaid records to flag prior drug treatment. A poor family on Medicaid has a visible treatment history; a wealthy family that paid privately at a clinic has none. The feature "no drug treatment history" means *different things* depending on socioeconomic status. Other examples: income of $100,000/year has different implications for men and women

given structural wage inequality; number of children is positively correlated with career success for men and negatively for women.

Principled criteria for feature selection. The question of *which* features to include in a predictor is under-studied but critical. Several principled objections arise:

- **Subjective assessment.** Features that are subjectively coded (e.g., "kempt vs. unkempt" in early risk instruments) introduce the assessor's bias directly into the model, while appearing to be objective data.
- **Autonomy.** Features outside an individual's control — immutable characteristics like race or biological sex — make predictions that cannot be acted upon. They also raise serious concerns about accountability for outcomes.
- **Private sphere.** Marital status, family composition, and similar features may belong to a domain where prediction is simply inappropriate, regardless of predictive accuracy.
- **Correlation vs. causation.** A feature may be predictive today only because it correlates with a causal factor. If social norms shift (e.g., if men's smoking rates converge with women's), the feature loses its predictive value — and any policies built on it become wrong. Example: women's greater longevity correlates with biological sex but is partly explained by proximal factors (risk-taking behavior, occupational stress, smoking rates) that are socially contingent.
- **The "New York Times" test.** A practical heuristic: if you would be embarrassed to explain in a news article that you used feature X to make prediction Y, you should reconsider the feature. This does not replace formal analysis, but it is a useful sanity check on whether your modeling choices are defensible to a non-specialist audience.

Measurement and hardware bias. Bias can be embedded in the hardware itself. Camera lenses are historically optimized for white skin tones; microphones and broadcast frequencies were calibrated for male vocal ranges, leading to the perception that women's voices are "shrill" — a perception produced by the technology, not by the voices. These are not labeling errors; they are biases in the *measurement apparatus*.

Label inflation and denominator problems. When labels reflect reporting rates rather than ground truth, populations that are surveilled more will appear to have higher rates of the labeled behavior. CPS receives more calls about families in dense urban neighborhoods (where neighbors can hear a crying baby) than about identical situations in rural areas. "Number of prior calls" is not "prior abuse" — it is a function of surveillance density, which correlates with poverty and race.

2.2 Group Fairness

Group fairness properties are *statistical requirements* imposed on a classifier's behavior across demographic groups.

2.2.1 The Confusion Matrix

For a binary classifier with true label $Y \in \{0, 1\}$ and predicted label $\hat{Y} \in \{0, 1\}$, all outcomes fall into four cells:

	Predicted positive	**Predicted negative**
Truly positive	True Positive (TP)	False Negative (FN)
Truly negative	False Positive (FP)	True Negative (TN)

Definition 2.1: Error Rates and Positive Predictive Value

For a group G:

$$\text{False Positive Rate (FPR)}_G = \frac{FP_G}{FP_G + TN_G} = \Pr\Big[\hat{Y} = 1 \mid Y = 0, G\Big]$$

$$\text{False Negative Rate (FNR)}_G = \frac{FN_G}{TP_G + FN_G} = \Pr\Big[\hat{Y} = 0 \mid Y = 1, G\Big]$$

$$\text{Positive Predictive Value (PPV)}_G = \frac{TP_G}{TP_G + FP_G} = \Pr\Big[Y = 1 \mid \hat{Y} = 1, G\Big]$$

The *base rate* of group G is $p_G = \Pr[Y = 1 \mid G]$, the fraction of truly positive individuals in G. It is a property of the population, not the classifier.

2.2.2 Chouldechova's Impossibility Theorem

Theorem 2.1: Impossibility of Simultaneous Fairness (Chouldechova, 2017)

For any imperfect classifier and two groups S, T with *unequal base rates* $p_S \neq p_T$, it is impossible to simultaneously achieve:

1. Equal false positive rates: $\text{FPR}_S = \text{FPR}_T$,
2. Equal false negative rates: $\text{FNR}_S = \text{FNR}_T$,
3. Equal positive predictive value: $\text{PPV}_S = \text{PPV}_T$.

Proof sketch. There is a mathematical identity linking these four quantities for any group G:

$$\text{PPV}_G = \frac{p_G(1 - \text{FNR}_G)}{p_G(1 - \text{FNR}_G) + (1 - p_G)\text{FPR}_G}.$$

This identity holds independently within each group. Requiring equal FPR and equal FNR across S and T pins two of the three varying quantities; the identity then forces $\text{PPV}_S = \text{PPV}_T$ if and only if $p_S = p_T$. If base rates differ, at least one of the three conditions must be violated.

Real-world instantiation: COMPAS. In 2016, ProPublica (Julia Angwin) reported that Northpointe's COMPAS recidivism risk tool exhibited higher false positive rates for Black defendants and higher false negative rates for white defendants. Northpointe responded that their tool was *calibrated* — equal positive predictive value across racial groups. Both claims were correct. Chouldechova's theorem explains why: differential base rates in rearrest (themselves a product of differential policing) make it mathematically impossible to equalize all three criteria simultaneously.

2.2.3 Statistical Parity and Its Limits

Definition 2.2: Statistical Parity

A classifier satisfies *statistical parity* with respect to groups G and $\bar{G}$ if the acceptance (positive prediction) rates are equal:

$$\Pr\left[\hat{Y} = 1 \mid G\right] = \Pr\left[\hat{Y} = 1 \mid \bar{G}\right].$$

Statistical parity is *meaningful in the breach*: a large violation is a warning sign worth investigating. But it is not a solution concept. A discriminatory actor can satisfy statistical parity while circumventing its intent: if a steakhouse owner wishes to exclude group G and is required to show equal advertising rates across G and $\bar{G}$, they can target the ads at the *vegetarians* within G — people who will not come regardless. The statistical parity requirement is met; the discriminatory intent is achieved.

2.2.4 Scoring Functions: Analogous Impossibility

For scoring functions $s : \mathcal{X} \to [0, 1]$ (rather than binary classifiers):

Definition 2.3: Fairness Criteria for Scores

- **Calibration:** within each bin of predicted score v, the fraction of positive outcomes is $\approx v$, independently within each group.
- **Balance for the positive class:** $\mathbb{E}[s \mid Y = 1, G_1] = \mathbb{E}[s \mid Y = 1, G_2]$ (positive individuals receive the same average score across groups).
- **Balance for the negative class:** $\mathbb{E}[s \mid Y = 0, G_1] = \mathbb{E}[s \mid Y = 0, G_2]$.

Theorem 2.2: Impossibility for Scores (Kleinberg–Mullainathan–Raghavan, 2016)

For two groups with unequal base rates, no imperfect predictor can simultaneously satisfy calibration, balance for the positive class, and balance for the negative class.

Proof sketch. Calibration implies that the total score assigned to group G_t equals the number of positive individuals in G_t:

$$\sum_{i \in G_t} s_i = \mu_t \qquad (t = 1, 2),$$

where μ_t is the number of positives in group t. The balance conditions fix X (average score for negatives) and Y (average score for positives) to be group-independent. Expanding the sum:

$$\mu_t = Y \cdot \mu_t + X \cdot (n_t - \mu_t),$$

which simplifies to $\mu_t = n_t X + \mu_t(Y - X)$ for groups of equal size $n_t = n$. This gives a pair of lines (one per group) in the (X, Y) plane; they coincide only when base rates $\mu_1/n = \mu_2/n$ are equal. Otherwise, the lines intersect only at $X = 0, Y = 1$ (perfect prediction).

Takeaway. Group fairness desiderata are not merely in political tension — they are *algebraically incompatible* for imperfect classifiers when base rates differ.

2.3 Individual Fairness

Definition 2.1 (Individual Fairness)**.** *For task metric d:*

$$\|\mu_u - \mu_v\| \leq d(u, v).$$

This is a Lipschitz constraint over distributions.

2.3.1 Fairness Linear Program

The individually fair classification problem can be cast as a linear program:

$$\min_{\{\mu_u\}_{u \in \mathcal{U}}} \mathbb{E}_{u,o}[L(u, o)] \quad \text{subject to} \quad \|\mu_u - \mu_v\| \leq d(u, v) \quad \forall\, u, v \in \mathcal{U}.$$

Fairness is a hard constraint; utility is the soft objective. Linear programs are solvable in polynomial time.

Limitations. Three practical obstacles limit this approach:

1. **Scale.** The universe $\mathcal{U}$ may contain billions of individuals. A polynomial running time in $|\mathcal{U}|$ is infeasible in practice (e.g., $|\mathcal{U}|^2$ constraints is already a trillion for a billion-person universe). One needs something closer to linear time.

2. **Generalization.** The LP is built on the individuals observed during training. It provides no fairness guarantee for individuals outside the training set.

3. **Metric access.** The LP requires the task metric d as input. Where does d come from? This is the metric-learning problem, addressed in Chapter **??**.

PAC learning (probably approximately correct learning) offers a partial remedy to the first two problems by working on a small sample rather than the full universe — producing a classifier that is likely to be approximately fair on the population.

2.4 Task-Competitive Composition

Motivation. Consider an online advertising setting where multiple advertisers compete for each user's attention in a real-time auction. When you visit a news website, an instantaneous auction determines which ad you see. Higher bidders displace lower bidders. Can fairness be guaranteed across this system?

A concrete example. A *grocery delivery service* bids \$1 for the attention of expectant parents (high lifetime value; subscription retention is strong). A *tech company* bids \$0.50 for qualified undergraduate candidates. Both advertisers design their classifiers to be individually fair for their own task — the grocery service advertises equally to similarly-situated expectant households; the tech company advertises equally to similarly-qualified candidates.

The grocery service outbids the tech company whenever both want the same user. The effective result: the tech company can only reach users the grocery service passed on. Expectant parents — disproportionately on parental leave or career transitions, disproportionately female — are systematically excluded from tech job ads, not by any intentional discrimination, but by the financial dynamics of the auction. Neither advertiser violated its own fairness condition.

Historical instantiation. A 2010 Wall Street Journal investigation reported that a major ad network categorized users into demographic bins with names like "White Picket Fence" and "Double

Income, No Kids." Credit card companies using this network were accused of *steering* minority users toward credit products with less favorable terms — a serious concern given federal anti-discrimination law in credit. The steering emerged from the composition of individually-designed targeting strategies, not from any single actor's misconduct.

Formal setup. A randomized classifier is a map $C : \mathcal{U} \to \Delta(\mathcal{O})$ assigning each individual a distribution over outcomes. For binary outcomes $\mathcal{O} = \{0, 1\}$, define $p_u := \Pr[C(u) = 1]$. Individual (metric) fairness requires $\|\mu_u - \mu_v\| \leq d(u, v)$; for binary outcomes this becomes $|p_u - p_v| \leq d(u, v)$.

Naive task-competitive composition. Consider tasks T (grocery) and T' (tech), with task-specific metrics d and d' and individually fair classifiers C and C' respectively. Ties go to T.

- User u receives the T' ad only if T did not bid on them.
- $\Pr[u \text{ receives } T'] = (1 - p_u)\, p'_u$.

The difference in T'-exposure between two individuals u, v:

$$(1 - p_u)p'_u - (1 - p_v)p'_v.$$

Key observation. Even if p is Lipschitz with respect to d and p' is Lipschitz with respect to d', the product $(1 - p)p'$ need not be Lipschitz under either metric. The multiplicative interaction introduced by outbidding breaks both fairness conditions.

Proof of violation. Choose u, v with $0 < p_u < p_v$ (task T favors v) and $p'_u \geq p'_v > 0$ (task T' favors u), with $d'(u, v)$ maximized. The gap in T'-exposure is:

$$(1 - p_u)p'_u - (1 - p_v)p'_v \;=\; \underbrace{d'(u, v)}_{\text{allowed gap}} \;+\; \underbrace{(p_v - p_u)p'_v}_{\text{violation term}>0}\,.$$

Setting $\alpha = p_v/p_u$ and choosing probabilities so that $\alpha \geq p'_u/p'_v$ (achievable because α can be pushed to exceed this ratio without violating C''s fairness), the violation term is strictly positive. The system as a whole violates individual fairness for task T', even though each classifier is individually fair in isolation.

Theorem 2.3: Naive Composition Fails

Let d and d' be non-trivial task metrics (i.e., some pair of individuals has $d(u, v) \in (0, 1)$). Then there exist individually fair classifiers C for T and C' for T' such that their naive task-competitive composition violates individual fairness for T'.

Implication: coordination is necessary. Fairness of the whole system does not follow automatically from fairness of its parts. An advertising network that wants to guarantee multi-task individual fairness must *actively coordinate* across advertisers — it is insufficient for each advertiser to independently satisfy its own fairness criterion. This is not merely a theoretical concern: when the structure of a market (auctions, outbidding, priority rules) causes tasks to interact, the platform bears responsibility for the emergent aggregate behavior.

2.5 The Denominator Problem in Auditing

Auditing a classifier for fairness requires a *benchmark*: a reference quantity against which to compare observed rates. The choice of benchmark determines the conclusion.

Population-based benchmark. Compare the stop rate for group S against the stop rate for group T, using total population as the denominator:

$$\frac{\text{stops in } S}{|S|} \quad \text{vs.} \quad \frac{\text{stops in } T}{|T|}.$$

In hypothetical data (Neil and Winship, *Annals of Criminology*): S has 15,000 criminals out of 100,000; T has 10,000 out of 100,000. If 10% of S are stopped vs. 5% of T, the population benchmark concludes a 2:1 disparity — strong evidence of discrimination against S.

Usage-based benchmark. Now suppose every person in a public space is stopped independently with probability 1/4, uniformly, with no consideration of race. Approximately 25% of S and 25% of T in the space are stopped. This is by construction non-discriminatory. Yet if S uses the space at twice the rate of T (for socioeconomic or geographic reasons), the *population-based* ratio of stops will be 2:1, exactly matching the earlier "evidence of discrimination" — even though the stopping rule is blind to race.

The denominator problem

The denominator reflects an *assumption* about what the police are doing and who is exposed to police contact. Different assumptions — total population, at-risk population, population using a specific space, criminally active population — yield different ratios from identical data and can support opposite conclusions.

Implications. When evaluating claims based on benchmark tests:

- Is the denominator justified by the mechanism being studied?
- Do stop rates vary by time of day, location, or type of crime in ways that interact with demography?
- Are the police targeting specific behaviors or areas, and does that targeting confound the comparison?

Different defensible answers produce different conclusions from the same data. This does not mean fairness auditing is impossible, but it means conclusions require explicit assumptions that should be stated and scrutinized.

Structural limits: a summary. Taken together, the results of this chapter suggest that group fairness is not a well-posed solution concept:

- Natural desiderata (FPR, FNR, PPV) are mutually exclusive when base rates differ.
- Statistical parity is gameable by a sophisticated bad actor.
- Auditing via benchmarks is highly sensitive to unstated denominator assumptions.

- None of the above is resolved by introducing a human into the decision loop — a human decision-maker is still an algorithm, just one with flesh rather than silicon.

The appropriate response to a group fairness *failure* is investigation, not automatic remediation. The mathematical impossibility results explain the failure; they do not prescribe a solution.

2.6 Paper Readings

Chouldechova (2017) — *Fair Prediction with Disparate Impact.* When base rates differ across demographic groups, no classifier can simultaneously satisfy calibration within each group, equal false positive rates, and equal false negative rates. The argument is purely algebraic: once calibration fixes the conditional expectations, the error rates are fully determined by the base rates, leaving no freedom to equalize them. This result is the mathematical backbone of the COMPAS controversy and demonstrates that apparent fairness trade-offs can be structural impossibilities rather than design failures.

Dwork, Hardt, Pitassi, Reingold, Zemel (2012) — *Fairness Through Awareness.* Introduces individual fairness as a Lipschitz condition: a randomized classifier is fair if similar individuals (under a task-specific metric d) receive outcome distributions within $D(M(u), M(v)) \leq d(u, v)$ of each other. The fairness-constrained classification problem can be expressed as a linear program, and the central open question — where the metric comes from — motivates Chapter 3. This paper established the individual-vs.-group distinction that now structures most of the algorithmic fairness literature.

Chapter 3

Learning the Metric: Human Arbiter

3.1 Individual Fairness: Treat Similar People Similarly

The most intuitive articulation of fairness is also one of the hardest to formalize: *similar individuals should be treated similarly.* This is the principle of *individual fairness*, introduced by Dwork, Hardt, Pitassi, Reingold, and Zemel (2012) as a formal counterpart to informal notions of non-discrimination.

The central challenge: "similar" is not a property of individuals in isolation — it is task-specific and context-dependent. Two candidates might be similar for the purpose of hiring engineers but dissimilar for the purpose of assessing surgical risk. There is no universal similarity function; the metric must be chosen by someone who understands the task.

Definition 3.1: Individual Fairness (Lipschitz Condition)

Let $(\mathcal{X}, d)$ be a metric space on individuals and $(\Delta(\mathcal{Y}), D)$ a metric space on distributions over outcomes. A randomized classifier $M : \mathcal{X} \to \Delta(\mathcal{Y})$ satisfies *individual fairness* if it is (D, d)-Lipschitz:

$$D\big(M(u),\, M(v)\big) \;\leq\; d(u, v) \qquad \forall\, u, v \in \mathcal{X}.$$

That is: the distance between the outcome distributions assigned to two individuals is bounded by their task-specific similarity distance.

Why a metric? The Lipschitz condition requires d to be a true metric (symmetric, satisfies triangle inequality, $d(u, u) = 0$). The triangle inequality is not just a technical convenience: it prevents the situation where u and w are each deemed "similar" to v but treated very differently.

The metric D on outcomes. A natural choice for D is total variation distance between the outcome distributions. If outcomes are binary (e.g., loan approved/denied), total variation between two Bernoulli distributions simplifies to $|p_u - p_v|$ — the absolute difference in approval probabilities.

3.2 The Metric Problem

Individual fairness does not say *which* metric d to use. This is intentional: the choice of d is a *normative decision*, not a mathematical one. The question "are these two people similar for this task?" is a policy question, not a data question.

Why the metric cannot be learned from the data. A natural thought: learn d from historical outcomes. If the historical data comes from a biased process (discriminatory hiring, unequal access to credit), then distances learned from those outcomes will encode the bias. Using biased historical data to define "similarity" and then requiring the algorithm to respect that similarity perpetuates the original discrimination under the guise of mathematical fairness.

The metric must be an *external normative input* that reflects what we believe people ought to be judged on — not what they have historically been judged on.

Who specifies the metric? Several approaches have been proposed:

1. **Domain experts.** A hiring committee, a medical board, or a panel of legal experts specifies which features are task-relevant and how differences in those features should be weighted.

2. **Regulatory requirements.** Anti-discrimination law may mandate that certain features (race, sex, religion) be ignored, implicitly defining a metric that is insensitive to those coordinates.

3. **Human arbiter / oracle.** A designated human (or panel) answers queries about specific pairs or triplets of individuals. This externalizes the normative question: the math defers to human judgment.

3.3 The Human Arbiter Model

The *human arbiter* approach treats the task-specific metric d as accessible only through a human oracle that can answer comparison queries. We do not assume the oracle reveals exact distances; it only answers ordinal questions.

Definition 3.2: Triplet Oracle

A *triplet oracle* for metric (X, d) takes as input three individuals $a, b, c \in X$ and returns the identity of the closer one:

$$\mathcal{O}(a, b, c) = \begin{cases} b & \text{if } d(a, b) \leq d(a, c), \\ c & \text{otherwise.} \end{cases}$$

The oracle answers: "Is b or c closer to a?"

Why triplets? Exact distances are hard for humans to provide: asking "how similar are these two candidates on a scale of 0 to 1?" yields inconsistent, poorly calibrated answers. Comparative judgments — "which of these two candidates is more similar to a third?" — are more reliable and cognitively natural. Triplet queries have been validated empirically as a basis for metric learning from human feedback.

3.4 Learning an Ordering via Triplet Queries

From comparisons to orderings. Fix a representative individual $r \in \mathcal{X}$. The triplet oracle can answer queries of the form "which of u, v is closer to r?" This is exactly the comparison oracle needed to run any comparison-based sorting algorithm.

Theorem 3.1: MergeSort Query Complexity

Given n individuals and access to a triplet oracle anchored at r, MergeSort recovers a total ordering of all individuals by $d(r, \cdot)$ using $O(n \log n)$ oracle queries.

Proof. MergeSort makes $\Theta(n \log n)$ comparisons. Each comparison is one triplet query. The sort returns individuals in non-decreasing order of $d(r, \cdot)$, which is a valid total ordering because $d(r, \cdot)$ is a real-valued function.

From an ordering to distances. A total ordering gives us the ranks $\sigma(u)$ for each u, but not the actual distances $d(r, u)$. To estimate $d(r, u)$, we use the gaps between adjacent elements in the sorted order. This requires stronger oracle access (e.g., asking for approximate ratios of distances, not just ordinal comparisons), and recovery is only approximate.

3.5 Approximate Metric Recovery: α-Submetrics

Exact metric recovery from ordinal queries is information-theoretically impossible in general: many metrics are consistent with the same set of triplet answers. We therefore aim for *approximate* recovery.

Definition 3.3: α-Submetric

A function $d' : \mathcal{X} \times \mathcal{X} \to \mathbb{R}_{\geq 0}$ is an *α-approximation* to metric d if:

$$d(u, v) \;\leq\; d'(u, v) \;\leq\; \alpha \cdot d(u, v) \qquad \forall\, u, v \in \mathcal{X}.$$

An *α-submetric* satisfies $d'(u, v) \leq d(u, v) + \alpha$ (additive approximation).

Representative-based distance. Fix a representative r. Define:

$$d_r(u, v) \;=\; \big|d(r, u) - d(r, v)\big|.$$

By the reverse triangle inequality, $d_r(u, v) \leq d(u, v)$ for all u, v (it is a 1-Lipschitz projection of d). It is a pseudometric: it satisfies all metric axioms except it may assign zero distance to distinct points (if $d(r, u) = d(r, v)$ but $u \neq v$).

Using multiple representatives. Choose a set of representatives $R = \{r_1, \ldots, r_k\}$. Define:

$$\hat{d}(u, v) \;=\; \max_{r \in R} d_r(u, v) \;=\; \max_{r \in R} \big|d(r, u) - d(r, v)\big|.$$

This is always a lower bound: $\hat{d}(u, v) \leq d(u, v)$. If R is chosen such that for every pair (u, v) there exists $r \in R$ with $d(r, u)$ and $d(r, v)$ differing by close to $d(u, v)$, then $\hat{d}$ approximates d well.

For doubling-dimension metric spaces (where every ball of radius 2ρ can be covered by a bounded number of balls of radius ρ), a random sample of size $O(\log n)$ is sufficient for a good approximation with high probability.

3.6 The Mahalanobis Distance: A Canonical Parametric Form

When individuals are represented as vectors in $\mathbb{R}^p$, the most expressive linear metric is the *Mahalanobis distance*:

Definition 3.4: Mahalanobis Distance

For a positive semi-definite matrix $A \in \mathbb{R}^{p \times p}$:

$$d_A(u, v) = \sqrt{(u - v)^\top A (u - v)}.$$

Special cases: $A = I$ gives Euclidean distance; $A = \Sigma^{-1}$ (inverse covariance) gives the classical Mahalanobis distance that whitens the feature space. The matrix A encodes which features matter and how much: a large diagonal entry for feature j means differences in j are penalized heavily.

Metric learning as semidefinite programming. Given a set of "must-link" pairs $\mathcal{S}$ (similar individuals) and "cannot-link" pairs $\mathcal{D}$ (dissimilar individuals), one can find the best-fit A by solving:

$$\begin{aligned} \min_{A \succeq 0} \quad & \sum_{(u,v) \in \mathcal{S}} d_A(u, v)^2 \\ \text{subject to} \quad & \sum_{(u,v) \in \mathcal{D}} d_A(u, v)^2 \geq 1. \end{aligned}$$

This is a semidefinite program (SDP): the constraint $A \succeq 0$ (positive semi-definiteness) is convex, so the problem can be solved in polynomial time. The constraint set $\mathcal{S}$ and $\mathcal{D}$ can be populated from human arbiter responses to pairwise queries.

Historical context. The Mahalanobis distance was introduced by P.C. Mahalanobis in 1936, originally to measure anthropometric distances between population groups in British India — arguing that intra-caste variation exceeds inter-caste variation, and that geographic proximity is a more principled basis for social classification than caste hierarchy. The original application was explicitly political; the tool it introduced is genuinely powerful and now ubiquitous in statistics and machine learning. This history is a reminder that mathematical tools are not neutral: they are invented to answer specific questions, those questions carry normative assumptions, and the tools can be repurposed in both better and worse directions.

3.7 Limitations and Critiques

Who is the arbiter? The human arbiter model defers the normative question to a human — but does not resolve it. If the designated arbiter is not representative (e.g., drawn from a homogeneous population), the metric will reflect their perspective, not the affected community's. Diversity of the arbiter pool matters as much as the mathematical formalism.

Whose similarity? The metric is task-specific: similar for hiring engineers is not the same as similar for bail decisions. In practice, the same features (education, neighborhood) are used across many tasks, creating de facto cross-task conflation of similarity.

Tension with group fairness. Individual fairness and group fairness (e.g., equalized odds, demographic parity) are not equivalent, and can conflict. A metric that treats individuals as similar when they have the same qualifications may still produce racially disparate outcomes if qualifications are unequally distributed across groups as a result of historical inequality. Individual fairness is silent on this; it only asks that the algorithm respect the metric, not that the metric itself be equitable.

The bootstrapping problem. Validating a fairness metric requires knowing what the fair outcomes are — but knowing fair outcomes is equivalent to having the metric. There is no ground truth to check against: the arbiter's responses are constitutive, not descriptive, of the fairness standard.

3.8 Paper Readings

Dwork, Hardt, Pitassi, Reingold, Zemel (2012) — *Fairness Through Awareness.* The foundational paper for this chapter: proposes the human arbiter model in which a designated oracle answers comparison queries about individual similarity, externalizing the normative question of who counts as similar for a given task. The paper proves that the resulting metric can be used to formulate fair classification as a linear program, and explicitly frames the open problem — how to elicit or learn the metric in practice — that the rest of this chapter addresses.

Mukherjee, Yao, Gaboardi, Learned-Miller, Agarwal (2020) — *Two Simple Ways to Learn Individual Fairness Metrics from Data.* Shows that a Mahalanobis distance encoding task-specific similarity can be learned from human-provided pairwise labels (similar/dissimilar) or triplet comparisons without requiring full expert specification. Both approaches reduce to convex optimization: the pairwise-constraint version is a semidefinite program; the triplet version uses gradient descent. The resulting metric is interpretable as a feature reweighting, but the quality of the metric depends entirely on the raters' judgments — the method systematizes elicitation without validating whether the elicited standard is itself equitable.

Chapter 4

Outcome Indistinguishability & the Multi-X Framework

4.1 Motivation: the meaning of a single probability

Richard Feynman famously asked: what does it mean to say there is a 12% chance of rain tomorrow? For a repeatable experiment we can check the prediction by running many trials; but tomorrow is a *unique* event. The same question applies to recidivism prediction, loan approval, or medical diagnosis: the ground truth $\pi_i^\star$ for individual i is never directly observable.

The Outcome Indistinguishability framework sidesteps this epistemological problem by asking a *behavioral* question: can a downstream observer—using whatever statistical tests they care about—distinguish behavior under the model from behavior under the true distribution? If not, the model is, in every practical sense, as good as the truth.

Outcome indistinguishability: A model is good relative to a collection of distinguishers $\mathcal{C}$ if no distinguisher $c \in \mathcal{C}$ can tell whether outcomes were drawn from real life or from the model.

This mirrors the scientific method: propose a model, derive observable consequences, and test whether those consequences match reality. Passing the tests does not prove the model is universally correct; it only says the model is good for the uses captured by the tests in $\mathcal{C}$.

Connection to computational indistinguishability. The OI framework was inspired in part by the polynomial-time indistinguishability definitions from cryptography (Chapter 14). Both use the same template: fix a class of efficient tests; require that two distributions (real-world outcomes vs. model outcomes, or truly random bits vs. pseudorandom bits) cannot be told apart by any test in that class. OI is the fairness-flavored instantiation of the same core idea.

4.2 Formal setup

- Universe of individuals/features: $\mathcal{X}$.
- Individuals are drawn from a population distribution:

$$I \sim \mathcal{D}_{\mathcal{X}}.$$

- Ground-truth per-individual success probability:

$$p_i^\star \in [0, 1].$$

The real-life outcome is:

$$Y_i^\star \sim \text{Bernoulli}(p_i^\star).$$

- Model / predictor:

$$\tilde{p}_i \in [0,1].$$

The model-generated outcome is:

$$\tilde{Y}_i \sim \text{Bernoulli}(\tilde{p}_i).$$

- Distinguishers:

$$c : (i, \tilde{p}_i, y) \mapsto [0,1].$$

The distinguisher receives information about an individual, possibly the model's prediction, and an outcome, and outputs a bounded real value.

The goal is that c should behave nearly the same whether the outcome is drawn from real life or from the model.

4.3 Distinguisher hierarchy

Different real-world observers have different amounts of information. We organize them into a four-level hierarchy of increasing power:

1. **No-access** distinguisher: sees only the raw input i and the binary outcome y; receives no prediction $\tilde{p}_i$.
2. **Sample-access** distinguisher: sees $(i, \tilde{p}_i, y)$ but only for finitely many sampled individuals. Cannot query the model on arbitrary inputs.
3. **Oracle-access** distinguisher: can query the model on any $i \in \mathcal{X}$, receiving $\tilde{p}_i$, then observe the outcome y.
4. **Full-access** distinguisher: knows the model $\tilde{p}$ in full; can compute $\tilde{p}_i$ for every i without querying.

Demanding OI against a stronger class $\mathcal{C}$ is a strictly stronger requirement. In the algorithm we will work with *real-valued* distinguishers: $c : \mathcal{X} \times [0,1] \times \{0,1\} \to [0,1]$, mapping (individual, prediction, outcome) to a score in $[0,1]$. The distinguishing gap becomes

$$\Delta_c = \mathbb{E}_i\big[\mathbb{E}[c(i, \tilde{p}_i, \tilde{Y}_i)] - \mathbb{E}[c(i, \tilde{p}_i, Y_i^\star)]\big] = \mathbb{E}_i\big[(\pi_i^\star - \tilde{p}_i)\, \delta_c(i)\big],$$

where $\delta_c(i) := c(i, \tilde{p}_i, 1) - c(i, \tilde{p}_i, 0)$ is the per-individual contrast (how much c changes when the outcome flips).

4.4 Real world versus model world

Fix a distinguisher $c \in \mathcal{C}$.

Define the expected behavior of c in the real world:

$$A_c = \mathbb{E}_{I \sim \mathcal{D}_\mathcal{X}} \left[\mathbb{E}\left[c(I, \tilde{p}_I, Y_I^\star) \mid I\right]\right].$$

Define the expected behavior of c in the model world:

$$B_c = \mathbb{E}_{I\sim\mathcal{D}_\mathcal{X}}\left[\mathbb{E}\left[c(I,\tilde{p}_I,\tilde{Y}_I)\mid I\right]\right].$$

The distinguishing gap is:

$$\Delta_c = A_c - B_c.$$

The model is ε-outcome-indistinguishable with respect to $\mathcal{C}$ if

$$|\Delta_c| \le \varepsilon \qquad \forall c \in \mathcal{C}.$$

4.5 Decomposition of the distinguishing gap

Proposition 4.1 (Gap decomposition)**.** *For any distinguisher $c : \mathcal{X} \times [0,1] \times \{0,1\} \to [0,1]$ and any model $\tilde{p}$,*

$$\Delta_c \;=\; \mathbb{E}_I\Big[(\pi^\star_I - \tilde{p}_I)\,\delta_c(I)\Big],$$

where $\delta_c(i) = c(i,\tilde{p}_i,1) - c(i,\tilde{p}_i,0)$.

Proof. Write $\mathbb{E}[c(i,\tilde{p}_i,\tilde{Y}_i)] = \tilde{p}_i\,c(i,\tilde{p}_i,1) + (1-\tilde{p}_i)\,c(i,\tilde{p}_i,0)$ and similarly for the ground-truth term (replacing $\tilde{p}_i$ with $\pi^\star_i$ for the Bernoulli draw while keeping $\tilde{p}_i$ as the prediction fed to c). Subtracting and simplifying:

$$\mathbb{E}[c(i,\tilde{p}_i,\tilde{Y}_i)] - \mathbb{E}[c(i,\tilde{p}_i,Y^\star_i)] = \tilde{p}_i\,\delta_c(i) - \pi^\star_i\,\delta_c(i) = -(\pi^\star_i - \tilde{p}_i)\,\delta_c(i).$$

Taking expectations over I and absolute values gives the result. □

Interpretation. The gap is an inner product between the error vector $(\pi^\star_i - \tilde{p}_i)$ and the sensitivity vector $\delta_c(i)$. A distinguisher can detect the model's flaws exactly when its sensitivity is *aligned* with those flaws.

4.6 Algorithm: iterative $\tilde{p}$-nudging

Start with the trivial predictor:

$$\tilde{p}^{(0)}(i) = \frac{1}{2} \qquad \forall i.$$

Repeat:

1. If $|\Delta_c| \le \varepsilon$ for every $c \in \mathcal{C}$, stop.
2. Otherwise, choose a distinguisher c with $|\Delta_c| > \varepsilon$.
3. Update:

$$\tilde{p}^{(t+1)}(i) = \Pi_{[0,1]}\left(\tilde{p}^{(t)}(i) + \frac{\Delta_c}{2}\delta_c(i)\right).$$

Here $\Pi_{[0,1]}$ is projection back into the interval $[0,1]$.

Intuition for the update

If $\Delta_c > 0$, then on average the model is underpredicting in directions where $\delta_c(i) > 0$. So we nudge $\tilde{p}_i$ upward for such individuals.
If $\delta_c(i) < 0$, the same update nudges $\tilde{p}_i$ downward.
Thus the update is a gradient-like correction against the distinguisher that currently detects

the largest systematic error.

4.7 Circuit interpretation

The algorithm can be viewed as gradually building a predictor circuit.

- Initially, the circuit outputs $1/2$ for every individual.
- Each failed distinguisher contributes one additional correction layer.
- After T updates, the predictor is the result of composing these T nudges.

The computationally hard part is often finding a distinguisher $c \in \mathcal{C}$ with large $|\Delta_c|$. This is related to agnostic learning. But once such a distinguisher is found, the update itself is simple.

4.8 Convergence: potential function argument

Define the *mean-squared error potential*

$$\Phi(\tilde{p}) \;=\; \mathbb{E}_I\big[(\pi^\star_I - \tilde{p}(I))^2\big] \;\in [0,1].$$

Lemma 4.1 (Descent per update)**.** *Suppose at step t we pick a violating c with $\Delta_c \geq \varepsilon$ and apply the update*

$$\tilde{p}^{(t+1)}(i) \;\leftarrow\; \Pi_{[0,1]}\Big(\tilde{p}^{(t)}(i) + \tfrac{\Delta_c}{2}\,\delta_c(i)\Big).$$

Then

$$\Phi(\tilde{p}^{(t)}) - \Phi(\tilde{p}^{(t+1)}) \;\geq\; \tfrac{3}{4}\,\Delta_c^2.$$

Proof sketch. Let $\eta = \Delta_c/2$ and $e_i = \pi^\star_i - \tilde{p}^{(t)}_i$. Ignoring projection (which only helps), the new squared error at individual i is

$$(e_i - \eta\,\delta_c(i))^2 = e_i^2 - 2\eta\, e_i\,\delta_c(i) + \eta^2\,\delta_c(i)^2.$$

Taking expectations and using $\mathbb{E}[e_i\,\delta_c(i)] = \Delta_c$ and $\mathbb{E}[\delta_c(i)^2] \leq 1$ (since $c \in [0,1]$):

$$\Phi(\tilde{p}^{(t+1)}) \;\leq\; \Phi(\tilde{p}^{(t)}) - 2\eta\Delta_c + \eta^2 = \Phi(\tilde{p}^{(t)}) - \Delta_c^2 + \tfrac{\Delta_c^2}{4} = \Phi(\tilde{p}^{(t)}) - \tfrac{3}{4}\,\Delta_c^2. \qquad \square$$

Corollary 4.1 (Iteration bound)**.** *Starting from $\Phi(\tilde{p}^{(0)}) \leq 1$, after at most $\lceil 4/\varepsilon^2 \rceil$ updates every distinguisher in $\mathcal{C}$ has gap $\leq \varepsilon$. The algorithm terminates in $O(1/\varepsilon^2)$ rounds.*

Proof. Each violating step drops Φ by at least $\frac{3}{4}\varepsilon^2$. Since $\Phi \geq 0$ and $\Phi \leq 1$, there can be at most $\frac{1}{(3/4)\varepsilon^2} = \frac{4}{3\varepsilon^2}$ such steps before no violating c remains. $\square$

Connection to gradient descent

The nudge update $\tilde{p} \leftarrow \Pi_{[0,1]}(\tilde{p} + \eta\,\delta_c)$ is exactly a *projected gradient step* on the potential Φ, where δ_c serves as the gradient direction. The algorithm is therefore an instance of projected gradient descent, and the $O(1/\varepsilon^2)$ bound matches the standard convergence rate for gradient descent with step-size proportional to the gradient norm.

4.9 Multi-X: accuracy, calibration, and validity

Definition 4.1 (Multi-accuracy)**.** *A predictor $\tilde{p}$ is* $(\mathcal{S}, \varepsilon)$-multi-accurate *if for every set $S \in \mathcal{S}$:*

$$\Big| \mathbb{E}_{I \in S}\big[\pi^\star_I - \tilde{p}(I)\big] \Big| \;\le\; \varepsilon.$$

Equivalently, choosing $c(i, \tilde{p}_i, y) = \mathbf{1}[i \in S] \cdot y$, multi-accuracy says the model has no systematic over- or under-prediction on any group in $\mathcal{S}$.

Multi-accuracy is a special case of OI: it corresponds to taking $\mathcal{C}$ to be the class of indicator-function distinguishers $c(i, \tilde{p}_i, y) = \mathbf{1}[i \in S] \cdot y$. The iterative algorithm above therefore produces a multi-accurate predictor in $O(1/\varepsilon^2)$ rounds whenever $\mathcal{S}$ is finite (or efficiently enumerable).

Connection to group fairness. Multi-accuracy is stronger than requiring average calibration per group: it bounds the *signed* error, preventing the model from simultaneously over-predicting for one demographic subgroup while under-predicting for another. By choosing $\mathcal{S}$ to contain all protected groups and their pairwise (and higher-order) intersections, we approach *intersectional* fairness.

Multi-calibration. Strengthening multi-accuracy to require calibration *within each score bucket* of each group yields *multi-calibration* (Hébert-Johnson, Kim, Reingold, Rothblum, 2018):

$$\forall S \in \mathcal{S},\ \forall v \in [0,1]: \quad \mathbb{E}\big[\pi^\star_I \mid I \in S,\ \tilde{p}_I = v\big] \;\approx\; v.$$

The same nudge-and-project algorithm achieves this; it is a strictly stronger guarantee than multi-accuracy.

4.9.1 Limitations

1. **Small groups.** When a group $S \in \mathcal{S}$ is small, the empirical estimate of $\mathbb{E}_{I \in S}[\pi^\star_I - \tilde{p}_I]$ has high variance. Meaningful multi-accuracy guarantees require sample sizes that grow with $|\mathcal{S}|$ and shrink with group size.
2. **Intersectionality at scale.** Requiring accuracy on *all* intersections $S_1 \cap S_2 \cap \cdots$ causes $|\mathcal{S}|$ to grow exponentially in the number of protected attributes, making the $O(1/\varepsilon^2)$ bound vacuous without additional structure (e.g., computationally efficient distinguisher classes).
3. **Gameable accuracy definitions.** A model can achieve multi-accuracy while still making badly miscalibrated predictions within fine-grained subgroups, motivating the stronger multi-calibration guarantee.

4.10 Connection to fairness

Multi-accuracy protects against systematic group-level prediction errors. If a predictor systematically overpredicts loan repayment for one group, multi-accuracy pushes those predictions down. If it systematically underpredicts for another group, multi-accuracy pushes those predictions up.

By choosing $\mathcal{S}$ to include protected groups and intersections of protected groups, one obtains a form of intersectional fairness.

However, small groups are difficult:

- If S is tiny, the training sample may contain too few examples from S.
- Then the algorithm may not be able to detect or correct systematic error on S.

Thus Multi-X helps with intersectionality only for groups explicitly included in $\mathcal{S}$ and sufficiently represented in the data.

4.11 Paper Readings

Hébert-Johnson, Kim, Reingold, Rothblum (2018) — ***Multicalibration: Calibration for the Computationally Identifiable Masses.*** Introduces multicalibration: the requirement that a predictor be calibrated not just globally but on every subgroup that a polynomial-time algorithm can identify. The main result is an efficient boosting-style algorithm that iterates through subgroups, applies a small nudge wherever calibration fails, and halts after $O(1/\alpha^2)$ rounds — the same structure as the OI algorithm in this chapter. Multicalibration implies multi-accuracy and is implied by outcome indistinguishability against oracle-access distinguishers, making it a key waypoint in the OI hierarchy.

Dwork, Kim, Reingold, Rothblum, Yona (2021) — ***Outcome Indistinguishability.*** Formally defines the OI framework: a predictor is outcome indistinguishable with respect to a class $\mathcal{C}$ of distinguishers if no $c \in \mathcal{C}$ can tell real-world outcomes from model-generated ones. The paper proves the equivalences between OI, multi-accuracy, and multicalibration at each level of the distinguisher hierarchy, and shows that the nudge-and-project algorithm achieves ε-OI in $O(1/\varepsilon^2)$ rounds. It is the primary reference for the material in this chapter.

Trevisan, Tulsiani, Vadhan (2009) — ***Regularity, Boosting, and Efficiently Simulating Every High-Entropy Distribution.*** Proves that any distribution with high Shannon entropy can be efficiently simulated by a distribution that is computationally indistinguishable from it, by iteratively correcting the simulator against the best-fitting distinguisher from a fixed class. The boosting-style argument and the potential function that bounds the number of correction steps are structurally identical to those in the OI convergence proof, providing the complexity-theoretic underpinning for the algorithm analyzed in this chapter.

Chapter 5

Differential Privacy and Reconstruction

5.1 Fundamental Law of Information Recovery

Theorem 5.1 (Dinur–Nissim)**.** *If too many queries are answered too accurately, database reconstruction is possible.*

Specifically: error $o(n)$ implies blatant non-privacy; even $o(\sqrt{n})$ error fails under many random queries.

5.2 Database Reconstruction Model

Let the database be $d \in \{0,1\}^n$. A query corresponds to a subset $S \subseteq [n]$: True answer: $\sum_{i \in S} d_i$, mechanism response: $R(S) = \sum_{i \in S} d_i + \text{noise}$. We assume a uniform accuracy guarantee: $\forall S \subseteq [n],\ \left|R(S) - \sum_{i \in S} d_i\right| \leq E$.

5.3 Blatant Non-Privacy

Definition (Blatant Non-Privacy)

An algorithm is *blatantly non-private* if an adversary can reconstruct all but $o(n)$ entries.

5.4 Reconstruction from Accurate Answers

Reconstruction Theorem

If all subset queries are answered with error at most E, then the Hamming distance between any consistent database c and the real database d satisfies $H(c,d) \leq 4E$.

Proof Sketch. Both d and any consistent c satisfy $|R(S) - \sum_S d_i| \leq E$ and $|R(S) - \sum_S c_i| \leq E$. By triangle inequality, $|\sum_S d_i - \sum_S c_i| \leq 2E$ for all S. Setting $S_1 = \{i : d_i = 1\}$ and $S_0 = \{i : d_i = 0\}$:

$$H(c,d) = \left|\sum_{i \in S_0} d_i - \sum_{i \in S_0} c_i\right| + \left|\sum_{i \in S_1} d_i - \sum_{i \in S_1} c_i\right| \leq 2E + 2E = 4E.$$

If $E = o(n)$ then $H(c,d) = o(n)$: almost the entire database is reconstructable.

5.5 Reconstruction from Linearly Many Random Queries

Take $q \in \{-1, 1\}^n$ drawn uniformly at random; true answer $q \cdot d$; noise $|\text{noise}| \leq \frac{\sqrt{\alpha n}}{3}$.

Signal from Distant Databases

If $H(c, d) \geq \alpha n$, then $q \cdot (d - c)$ is a sum of αn independent ± 1 variables. By anti-concentration, $\Pr_q[|q \cdot (d - c)| \geq \sqrt{\alpha n}] \geq \kappa$ for some constant $\kappa > 0$.

After βn independent queries, by union bound:

$$\Pr[\exists \text{ distant } c \text{ that survives}] \leq 2^{\alpha n} \cdot p^{\beta n}, \quad p = 1 - \kappa.$$

Choosing $\beta > \frac{\alpha+\gamma}{\log_2(1/p)}$ drives this to $2^{-\gamma n}$, eliminating all distant databases with high probability.

Theorem (Random Query Reconstruction)

If $O(n)$ random queries are answered with noise $o(\sqrt{n})$, then database reconstruction is possible with high probability.

5.6 Concentration vs. Anti-Concentration

Let $S = \sum_{i=1}^{k} X_i$, $X_i \in \{-1, 1\}$ i.i.d.

Concentration (Chernoff): $\Pr[|S| \geq t] \leq 2e^{-t^2/(2k)}$. Most mass lies near 0.

Anti-Concentration: $\Pr\left[|S| \geq \sqrt{k}\right] \geq \kappa > 0$ (from CLT: $S/\sqrt{k} \approx \mathcal{N}(0, 1)$).

Key Insight for Reconstruction

If noise $< \frac{\sqrt{\alpha n}}{3}$, then with constant probability each random query produces a signal larger than the noise. After $\Theta(n)$ queries, all distant databases are eliminated.

5.7 Connection to Differential Privacy

Noise must scale at least $\Omega(\sqrt{n})$ to prevent reconstruction. Differential privacy $\Pr[M(x) \in S] \leq e^{\varepsilon} \Pr[M(y) \in S]$ prevents catastrophic reconstruction even under adaptive querying.

5.8 Paper Readings

Dinur and Nissim (2003) — ***Revealing Information While Preserving Privacy.*** Proves that any mechanism answering all subset-sum queries on an n-bit database with error $o(\sqrt{n})$ allows efficient reconstruction of almost the entire database. The attack issues $O(n)$ random queries, forming an overdetermined linear system whose solution approximates the private bits; the $\sqrt{n}$ error threshold is tight. This result is the formal justification for differential privacy: it establishes that useful statistical answers and full individual-level privacy cannot coexist, so any practically private mechanism must inject noise of order at least $\sqrt{n}$.

Chapter 6

Laplace and Exponential Mechanisms

6.1 Global Sensitivity

Definition 6.1 (Global Sensitivity)**.** *For a function $f : \mathcal{X}^n \to \mathbb{R}$:*

$$\Delta f = \max_{adj\ x,y} |f(x) - f(y)|.$$

Sensitivity measures how much one individual's data can change the output.

6.2 Laplace Mechanism

Theorem 6.1: Laplace Mechanism

The mechanism

$$M(x) = f(x) + \mathrm{Lap}\left(\frac{\Delta f}{\varepsilon}\right)$$

satisfies ε-differential privacy.

Proof idea. For adjacent x, y, the ratio of output densities is bounded:

$$\frac{\mathrm{pdf}_{M(x)}(t)}{\mathrm{pdf}_{M(y)}(t)} \leq e^{\varepsilon}.$$

Interpretation

Noise scale is:

$$\text{sensitivity}/\varepsilon.$$

Stronger privacy ($\varepsilon \downarrow$) means more noise.

6.3 Accuracy for a Single Query

Let $Y \sim \mathrm{Lap}\left(\frac{\Delta f}{\varepsilon}\right)$. We want $\Pr[|Y| \leq \alpha] \geq 1 - \beta$. Using the Laplace tail bound:

$$\Pr[|Y| \geq t \cdot b] = e^{-t}, \quad b = \frac{\Delta f}{\varepsilon}.$$

Set $t = \ln(1/\beta)$, giving:

$$\alpha = \frac{\Delta f}{\varepsilon} \ln(1/\beta).$$

Theorem 6.2: Laplace Accuracy (Single Query)

$$\alpha = O\left(\frac{\Delta f}{\varepsilon} \log \frac{1}{\beta}\right).$$

6.4 High-Dimensional Laplace Mechanism

For vector-valued queries $f : \mathcal{X}^n \to \mathbb{R}^k$, define the ℓ_1 sensitivity:

$$\Delta_1 f = \max_{\text{adj } x,y} \|f(x) - f(y)\|_1.$$

For instance, if we ask simultaneously how many people are over six feet tall and how many prefer vanilla to strawberry, one person can change *both* counts by 1, giving ℓ_1 sensitivity 2. In general, for k arbitrary counting queries, one person can affect all k counts, so $\Delta_1 f = k$ in the worst case.

The mechanism draws k independent noise terms and adds one to each coordinate:

$$M(x) = f(x) + (Y_1, \ldots, Y_k), \qquad Y_i \overset{\text{i.i.d.}}{\sim} \text{Lap}\left(\frac{\Delta_1 f}{\varepsilon}\right).$$

The privacy proof is identical to the one-dimensional case, applied coordinate by coordinate.

Privacy depends on ℓ_1 sensitivity, not on the dimension k.

6.5 Accuracy for k Queries

We want $\Pr[\max_i |Y_i| \le \alpha] \ge 1 - \beta$. By a union bound:

$$\Pr[\exists\, i : |Y_i| \ge \alpha] \le k \cdot \Pr[|Y| \ge \alpha].$$

Setting $\Pr[|Y| \ge \alpha] \le \beta/k$ and solving via the Laplace tail:

$$\exp\left(-\frac{\alpha\varepsilon}{\Delta_1 f}\right) \le \frac{\beta}{k} \quad \Longrightarrow \quad \alpha = \frac{\Delta_1 f}{\varepsilon} \ln\left(\frac{k}{\beta}\right).$$

Theorem 6.3: Laplace Accuracy (k Queries)

$$\alpha = O\left(\frac{\Delta_1 f}{\varepsilon} \log \frac{k}{\beta}\right).$$

Key insight

Accuracy degrades only logarithmically in the number of queries.

6.6 Histogram Queries: Sensitivity Is Always 1

A *histogram query* partitions the population into disjoint cells and counts how many individuals fall into each cell. Examples include:

- Country of birth (each person is born in exactly one country).
- City of residence (to a first approximation, each person lives in one city).
- Any attribute that takes a unique value per individual.

Theorem 6.4: Histogram Sensitivity

No matter how many cells a histogram has, its ℓ_1 sensitivity is **1**. Adding or removing one individual changes exactly one cell count by ± 1 and leaves all other cells unchanged, because the partition ensures each individual belongs to exactly one cell.

For a histogram with k cells:

$$\Delta_1 f_{\text{hist}} = 1 \quad \text{vs.} \quad \Delta_1 f_{\text{arbitrary}} = k.$$

The Laplace mechanism adds noise $\text{Lap}(1/\varepsilon)$ to each cell, independent of k.

This is one of the foremost practical advantages of DP for administrative statistics. Most government statistics — population counts by region, age, income bracket, and so on — are histograms or *contingency tables* (multi-dimensional histograms). All have sensitivity 1.

Choosing the right algorithm. For a given ε, there are often many ε-DP algorithms for the same task with different accuracy. Exploiting histogram structure yields dramatically better accuracy at no privacy cost. The comparison between algorithms is always on *utility*, never on privacy.

6.7 Privacy Budget Interpretation

Differential privacy can be viewed as having a total budget ε.

- Answer one query: use full ε.
- Answer many queries: split the budget.

Naively splitting gives ε/k per query, forcing noise $\propto k/\varepsilon$.

Naive composition leads to noise growing linearly in the number of queries. Advanced composition (Chapter 8) and the Gaussian mechanism (Chapter 9) reduce this to $O(\sqrt{k})$.

6.8 Differential Privacy vs. Cryptographic Privacy

The cryptographic setting. A sender transmits an encrypted message. The legitimate receiver learns the plaintext; the eavesdropper learns nothing useful. These two parties are *different*. Modern cryptography — under standard hardness assumptions — achieves this.

The DP setting. The data analyst is *both* the legitimate user *and* the potential adversary. We cannot simultaneously let the analyst learn something useful and prevent them from learning anything.

The fundamental law of information release

If a data analysis algorithm teaches the analyst anything useful about the population, then *some* information about the individuals in the dataset is necessarily leaked. This is unavoidable. The goal of DP is not to prevent leakage entirely, but to ensure it is *small*, *calibrated*, and *formally bounded*.

An additional subtlety: even a well-intentioned analyst who publishes results enables a later party to combine them with other analyses from the same dataset. Privacy loss accumulates over time across all users, which is why composition theorems matter.

6.9 The Exponential Mechanism

The Laplace mechanism works for real-valued outputs. But many natural outputs are *discrete* or *categorical* — the most common search query string, the best expert among a panel, the optimal auction price. Adding noise to these does not make sense.

Definition 6.1: Exponential Mechanism

Let Ψ be a finite set of possible outputs, and let $u : \mathcal{X}^n \times \Psi \to \mathbb{R}$ be a *utility function* with sensitivity

$$\Delta u = \max_{\psi \in \Psi} \max_{\text{adj } x,y} |u(x,\psi) - u(y,\psi)|.$$

The *exponential mechanism* $\mathcal{M}_E$ outputs $\psi \in \Psi$ with probability proportional to

$$\exp\left(\frac{\varepsilon\, u(x,\psi)}{2\,\Delta u}\right).$$

Theorem 6.5: Exponential Mechanism Is ε-DP

$\mathcal{M}_E$ satisfies ε-differential privacy.

Proof sketch. Fix adjacent x, y and any $\psi \in \Psi$. Let $Z_x = \sum_{\psi'} e^{\varepsilon u(x,\psi')/2\Delta u}$.

$$\frac{\Pr[\mathcal{M}_E(x) = \psi]}{\Pr[\mathcal{M}_E(y) = \psi]} = \frac{e^{\varepsilon u(x,\psi)/2\Delta u}}{Z_x} \cdot \frac{Z_y}{e^{\varepsilon u(y,\psi)/2\Delta u}} = e^{\varepsilon(u(x,\psi)-u(y,\psi))/2\Delta u} \cdot \frac{Z_y}{Z_x}.$$

The first factor is at most $e^{\varepsilon/2}$ by definition of Δu. For the second, each term in Z_y is at most $e^{\varepsilon/2}$ times the corresponding term in Z_x, so $Z_y/Z_x \le e^{\varepsilon/2}$. Multiplying: ratio $\le e^{\varepsilon}$. □

Why the factor of 2? Adding one individual can *increase* the utility of some outcomes and *decrease* the utility of others simultaneously. The $2\Delta u$ denominator accounts for this: the numerator contributes at most $e^{\varepsilon/2}$, and the ratio of normalizing constants contributes another $e^{\varepsilon/2}$.

Theorem 6.6: Exponential Mechanism Accuracy

Let $u^* = \max_\psi u(x,\psi)$ and $\Psi^*(x) = \{\psi : u(x,\psi) = u^*\}$. For any $V \le u^*$:

$$\Pr\big[\mathcal{M}_E(x) \text{ outputs } \psi \text{ with } u(x,\psi) \le V\big] \;\le\; \frac{|\Psi|}{|\Psi^*|} \cdot e^{\varepsilon(V-u^*)/(2\Delta u)}.$$

Proof sketch. Bound the numerator by the total unnormalized weight of all bad outputs, and the denominator by the weight of the optimal outputs alone:

$$\frac{|\Psi| \cdot e^{\varepsilon V/2\Delta u}}{|\Psi^*| \cdot e^{\varepsilon u^*/2\Delta u}} = \frac{|\Psi|}{|\Psi^*|} \cdot e^{\varepsilon(V-u^*)/2\Delta u}.$$

□

Outputs far below the optimum are exponentially unlikely. This bound is a template: plug in $|\Psi|$, $|\Psi^*|$, and the gap $u^* - V$ for each application.

6.10 Limitations of Independent Noise

Laplace answers queries independently, with noise uncorrelated across queries. It cannot exploit structure shared between queries.

Better mechanisms (e.g., SmallDB, Chapter 7) introduce *correlated noise* to answer many queries more efficiently.

Chapter 7

SmallDB: Nets, Exponential Mechanism, Accuracy

7.1 Motivation: Answering Exponentially Many Queries

With n data points and Laplace noise, we can accurately answer roughly n adaptive queries. Can we do better? Remarkably, yes: with the exponential mechanism we can answer *exponentially many* counting queries by releasing a synthetic database.

Setup. We have a real database X of n rows and a large collection $\mathcal{Q}$ of counting queries. Instead of answering each query directly, we release a *synthetic database* $\tilde{X}$ that approximates the fractional answers to all queries in $\mathcal{Q}$. The *fractional query* is $f_q(X) = \frac{1}{n}\sum_{i=1}^{n} q(x_i)$. We want $\tilde{X}$ such that for all $q \in \mathcal{Q}$:

$$|f_q(\tilde{X}) - f_q(X)| \leq \alpha.$$

7.2 The SmallDB Algorithm

Outcome space. The exponential mechanism chooses $\tilde{X}$ from the set of all *small databases* of size

$$m = \left\lceil \frac{\log |\mathcal{Q}|}{\alpha^2} \right\rceil,$$

drawn from the same universe as X. Crucially, m depends only on $|\mathcal{Q}|$ and α, *not on* n.

Utility function. For a candidate small database ψ:

$$u(X, \psi) = -\max_{q \in \mathcal{Q}} |f_q(X) - f_q(\psi)|.$$

All utilities are ≤ 0; a utility close to 0 means ψ agrees well with X on all queries.

Sensitivity. Under the swap adjacency (replace one row of X), each fractional query changes by at most $1/n$, so $\Delta u = 1/n$.

Privacy and accuracy. The exponential mechanism with parameter ε is ε-DP, and with high probability outputs $\tilde{X}$ within $\alpha + O\left(\frac{\log |\mathcal{Q}|}{n\varepsilon}\right)$ of X on all queries.

7.3 The Histogram Representation

Fix the universe $\mathcal{X}$ of all possible data rows with types $\mathcal{X}_1, \mathcal{X}_2, \ldots$

Definition 7.1: Histogram Representation

The *histogram* of a database X of size n is the vector $(h_1, h_2, \ldots)$ where h_j is the number of rows in X of type $\mathcal{X}_j$. Every database corresponds to a unique histogram (up to row order), and vice versa.

A small database of size m corresponds to a histogram with entries summing to m. Even if the universe has 2^d types, a small database of size $m \ll 2^d$ has at most m nonzero histogram entries.

7.4 The Net Theorem and the Probabilistic Method

Definition 7.2: α-Net (w.r.t. $\mathcal{Q}$)

A set R of databases is an *α-net for $\mathcal{Q}$* if for every database X there exists $\psi \in R$ with

$$\max_{q \in \mathcal{Q}} |f_q(X) - f_q(\psi)| \;\le\; \alpha.$$

Theorem 7.1: Small Databases Form a Net

Let $\mathcal{Q}$ be any finite collection of counting queries. The set R of all databases of size $m = \lceil \log|\mathcal{Q}|/\alpha^2 \rceil$ is an α-net for $\mathcal{Q}$: every large database is within α of some element of R.

Proof via the probabilistic method. Fix any database X of size n. Construct a random small database Z of size m by drawing $Z_1, \ldots, Z_m$ independently, each equal to type $\mathcal{X}_j$ with probability h_j/n.

Step 1. For any $q \in \mathcal{Q}$: $\mathbb{E}[q(Z_i)] = \sum_j \frac{h_j}{n} \cdot q(\mathcal{X}_j) = f_q(X)$. So $f_q(Z) = \frac{1}{m}\sum_{i=1}^{m} q(Z_i)$ is an average of m i.i.d. $\{0,1\}$-valued variables with mean $f_q(X)$.

Step 2 (Chernoff bound). $\Pr[|f_q(Z) - f_q(X)| \ge \alpha] \le 2e^{-2m\alpha^2}$.

Step 3 (Union bound). $\Pr[\max_{q \in \mathcal{Q}} |f_q(Z) - f_q(X)| \ge \alpha] \;\le\; |\mathcal{Q}| \cdot 2e^{-2m\alpha^2} \;<\; 1$ for $m = \lceil \log|\mathcal{Q}|/\alpha^2 \rceil$.

Step 4 (Probabilistic method). Since the probability of Z being bad is strictly less than 1, a good Z must exist. That Z lies in R, so R is an α-net. □

The Probabilistic Method

To prove that a good object *exists*: construct a random object, show the probability of it being bad is strictly less than 1, and conclude a good one must exist. The argument does not find the object explicitly, only guarantees its existence. This technique, popularized by Erdős, appears throughout combinatorics and theoretical CS.

Why this is surprising. The size m depends only on $|\mathcal{Q}|$ and α, completely independent of n. No matter how large the original database, a tiny synthetic database of size m can approximate all queries.

7.5 Accuracy of SmallDB

From the exponential mechanism accuracy theorem:

$$d(X, \widehat{Z}) \;\lesssim\; \alpha + \frac{1}{\varepsilon n}\log|\mathcal{R}| + \frac{1}{\varepsilon n}\log(1/\beta),$$

where α is the net approximation error, the second term is the privacy penalty, and the third is a confidence correction.

7.6 Size of the Net

Each database in $\mathcal{R}$ has m rows drawn from $\mathcal{X}$, so $|\mathcal{R}| \approx |\mathcal{X}|^m$, giving:

$$\log |\mathcal{R}| \approx m \log |\mathcal{X}| = O\left(\frac{\log |\mathcal{Q}|}{\alpha^2} \log |\mathcal{X}|\right).$$

7.7 Optimizing the Error

Plugging into the accuracy bound and balancing α against the privacy term:

$$\alpha \approx \frac{\log |\mathcal{Q}| \log |\mathcal{X}|}{\varepsilon n \alpha^2} \quad \Longrightarrow \quad \alpha^3 \approx \frac{\log |\mathcal{Q}| \log |\mathcal{X}|}{\varepsilon n} \quad \Longrightarrow \quad \alpha \approx \left(\frac{\log |\mathcal{Q}| \log |\mathcal{X}|}{\varepsilon n}\right)^{1/3}.$$

The absolute error satisfies $n\alpha \approx n^{2/3}$.

Theorem 7.2: SmallDB Accuracy

$$\alpha = O\left(\left(\frac{\log |\mathcal{Q}| \log |\mathcal{X}|}{\varepsilon n}\right)^{1/3}\right).$$

7.8 Why This Does Not Break Privacy Lower Bounds

Reconstruction theorems say: too many queries answered too accurately $\Rightarrow$ reconstruction of the database. SmallDB avoids this because the error $\approx n^{2/3}$ is large enough to prevent reconstruction — the noise is not too small.

7.9 Feature Dimension Warning

The accuracy bound depends on $\log |\mathcal{X}|$. If individuals have many features, $|\mathcal{X}|$ grows exponentially and the error grows accordingly.

Important takeaway

DP works best when:

- n is large (many individuals), and
- $\log |\mathcal{X}|$ is small (relatively few features per individual).

7.10 Comparison with Laplace

- **Laplace:** answers each query independently with noise $\sim k/\varepsilon$; no structure across queries is exploited.
- **SmallDB:** releases one synthetic dataset; answers are correlated; supports query classes of exponential size.

SmallDB uses **correlated noise** implicit in the synthetic database to answer exponentially many queries at error $O(n^{-2/3})$, far better than Laplace's $O(k/\varepsilon n)$ for large k.

Chapter 8

Advanced Composition and Privacy Loss

8.1 Motivation: Going Beyond Simple Composition

Simple composition says that k applications of an ε-DP mechanism cost $k\varepsilon$ in privacy. For large k this bound is too pessimistic. Advanced composition shows the true cost scales like $\sqrt{k}\,\varepsilon$ — an exponential improvement for high-query regimes.

The key insight is that privacy loss is a *random variable*, not a fixed constant. It can be *negative*, and the positive and negative losses partially cancel. The cumulative loss behaves like a biased random walk.

8.2 The Adaptive Composition Game

Advanced composition is proved via a general game that captures cumulative privacy loss across *different* databases and mechanisms, not just repeated queries to one database.

The game.

1. The challenger fixes a secret bit $b \in \{0, 1\}$ at the start.

2. In each round $i = 1, \ldots, k$:

   - The adversary chooses a pair of adjacent databases x_i^0, x_i^1 and an ε-DP mechanism M_i (adaptively, based on all prior outputs).
   - The challenger returns $y_i \sim M_i(x_i^b)$.

3. After k rounds, the adversary guesses b.

The adversary may use a different pair of databases and a different mechanism each round. This models your cumulative privacy loss across all databases you participate in throughout your life, not just one interaction.

8.3 Privacy Loss Random Variables

Definition 8.1: Privacy Loss Random Variable

For mechanism M run on database x, compared against adjacent database y, the *privacy loss random variable* for output c is:

$$\mathcal{L}^{(c)}_{M(x)\|M(y)} = \ln \frac{\Pr[M(x) = c]}{\Pr[M(y) = c]}.$$

We sample this random variable by running $M(x)$ and observing the output c.

For an ε-DP mechanism: $|\mathcal{L}| \leq \varepsilon$ almost surely (this is exactly the bounded log-ratio condition).

8.3.1 Negative Privacy Loss and the Random Walk Intuition

When is privacy loss negative? The loss is negative when $\Pr[M(x) = c] < \Pr[M(y) = c]$: the observed output is *more likely* under the adjacent database y than under the true database x. Instead of giving a hint toward x, the output points away from it, giving the adversary a false read.

The random walk picture. Imagine your database is x. Each mechanism output shifts the adversary's belief slightly toward x (positive loss) or away from it (negative loss). Since x is the true database, outputs are *somewhat more likely* under x, so the walk has a slight positive drift. But many steps go in the negative direction, partially canceling the drift. The cumulative loss after k steps behaves like $O(\sqrt{k} \cdot \varepsilon)$ rather than $k \cdot \varepsilon$, just as a random walk with drift μ has standard deviation $O(\sqrt{k})$ after k steps.

Expected loss is $O(\varepsilon^2)$, not $O(\varepsilon)$. A technical lemma (proved using KL divergence properties) shows:

$$\mathbb{E}[\mathcal{L}^{(Y)}_{M(x)\|M(y)}] \;\leq\; \varepsilon(e^{\varepsilon} - 1) \;\leq\; \varepsilon^2.$$

(The improved bound ε^2 without the factor of 2 was established in later work.) Since $\varepsilon < 1$, the expected loss per step is $\varepsilon^2 \ll \varepsilon$ — the positive and negative losses cancel to a much smaller residual than the worst case.

8.4 Adversary Views and Bad Views

Two parallel worlds. Define the adversary's *view* as everything it observes: its own randomness R and all outputs $y_1, \ldots, y_k$. There are two parallel versions:

- V^0: the view when the secret bit is 0 (challenger always runs $M_i(x_i^0)$).
- V^1: the view when the secret bit is 1 (challenger always runs $M_i(x_i^1)$).

Privacy fails if these two views are distinguishable.

By the chain rule of probability and the DP bound at each step:

$$\ln \frac{\Pr[V^0 = v]}{\Pr[V^1 = v]} = \sum_{i=1}^{k} \mathcal{L}_i,$$

where $\mathcal{L}_i$ is the privacy loss at step i, conditioned on the shared history.

Bad views. A view v is *bad* if $\Pr[V^0 = v] > e^{\varepsilon^*} \Pr[V^1 = v]$ for some target ε^*. Privacy fails only when the adversary lands in a bad view. We decompose:

$$\Pr[V^0 \in S] \;\le\; \Pr[V^0 \text{ is bad}] + e^{\varepsilon^*} \Pr[V^1 \in S]$$

for any set S. If the probability of a bad view is at most δ, the mechanism is (ε^*, δ)-DP.

8.5 Applying Azuma–Hoeffding

The privacy losses $\mathcal{L}_1, \ldots, \mathcal{L}_k$ form a *martingale difference sequence*: each is bounded $|\mathcal{L}_i| \le \varepsilon$, and $\mathbb{E}[\mathcal{L}_i \mid \mathcal{L}_1, \ldots, \mathcal{L}_{i-1}] \le \varepsilon^2$. The Azuma–Hoeffding inequality applies to exactly this setting:

$$\Pr\left[\sum_{i=1}^{k} \mathcal{L}_i > k\varepsilon^2 + t\varepsilon\sqrt{k}\right] \;\le\; e^{-t^2/2}.$$

Setting $t = \sqrt{2\ln(1/\delta)}$ gives the right-hand side equal to δ, and:

Theorem 8.1: Advanced Composition for Pure DP

For every $\delta > 0$, the adaptive composition of k mechanisms each satisfying ε-DP satisfies (ε^*, δ)-DP, where:

$$\varepsilon^* = \sqrt{2k\ln(1/\delta)} \cdot \varepsilon + k\varepsilon(e^{\varepsilon} - 1).$$

For small ε, the dominant term is $O\Big(\sqrt{k\ln(1/\delta)} \cdot \varepsilon\Big)$, compared to the naive $k\varepsilon$ of simple composition.

Optimal privacy budget. Equating the two terms and solving for ε as a function of k: choosing $\varepsilon \approx 1/\sqrt{k}$ minimizes the total privacy loss for a fixed number of queries.

Chapter 9

Approximate Differential Privacy and the Gaussian Mechanism

9.1 Motivation: The Limits of Pure DP

All the mechanisms we have seen so far satisfy *pure* ε-differential privacy: for all adjacent x, y and all events S, $\Pr[M(x) \in S] \leq e^{\varepsilon} \Pr[M(y) \in S]$. This is a worst-case, information-theoretic guarantee.

The ubiquitous Gaussian distribution — beloved in statistics for its tail concentration and its connection to least-squares estimation — cannot satisfy this guarantee. Accommodating Gaussian noise requires a relaxed definition.

9.2 Why Gaussian Noise Does Not Give Pure DP

The natural question is: why use Laplace noise rather than Gaussian noise? Gaussian noise is familiar, analytically convenient, and has lighter tails. The answer is that Gaussian noise *cannot* satisfy pure ε-DP.

The log-ratio argument. Pure ε-DP requires the log-ratio $\ln \frac{p_X(c)}{p_Y(c)}$ to be bounded by ε for *all* outputs c and all adjacent X, Y. For the Laplace distribution, this ratio is bounded everywhere because the PDF decays exponentially and the ratio of two shifted Laplace PDFs converges to a fixed constant. For the Gaussian $\mathcal{N}(f(X), \sigma^2)$ vs. $\mathcal{N}(f(Y), \sigma^2)$:

$$\ln \frac{p_{f(X)}(c)}{p_{f(Y)}(c)} = \frac{(c - f(Y))^2 - (c - f(X))^2}{2\sigma^2} = \frac{(f(X) - f(Y))(2c - f(X) - f(Y))}{2\sigma^2}.$$

This is *linear in* c and grows without bound as $c \to \pm\infty$. No finite ε can bound the log-ratio over all outputs.

Key fact

Gaussian noise gives (ε, δ)-DP (approximate DP) for appropriate parameters, but *never* pure ε-DP for any finite ε.

9.3 ℓ_2 Sensitivity

Definition 9.1: ℓ_2 Sensitivity

The ℓ_2 *sensitivity* of a function $f : \mathcal{X}^n \to \mathbb{R}^k$ is:

$$\Delta_2 f = \max_{\text{adjacent } x,y} \|f(x) - f(y)\|_2.$$

Comparison to ℓ_1 sensitivity. For k counting queries (each output coordinate is a fraction in $[0, 1]$):

- ℓ_1 sensitivity: $\Delta_1 f = k$ (each query can change by $1/n$ per person, and one person can affect all k queries, giving total ℓ_1 change k/n; scaling to sensitivity-1 queries gives $\Delta_1 = k$).
- ℓ_2 sensitivity: $\Delta_2 f = \sqrt{k}$ (Cauchy–Schwarz: $\|\mathbf{1}\|_2 = \sqrt{k}$).

Since $\sqrt{k} \ll k$ for large k, the Gaussian mechanism can add far less noise than Laplace for high-dimensional queries — at the cost of the pure-DP guarantee.

9.4 Approximate (ε, δ)-Differential Privacy

We relax the pure DP guarantee by allowing a small additive failure probability δ.

Definition 9.2: (ε, δ)-Differential Privacy (Approximate DP)

A randomized mechanism M satisfies (ε, δ)-*differential privacy* if for all pairs of adjacent databases x, y and all measurable events S:

$$\Pr[M(x) \in S] \;\leq\; e^{\varepsilon} \Pr[M(y) \in S] \;+\; \delta.$$

The special case $\delta = 0$ recovers pure ε-DP.

Interpretation. With probability at most δ, the mechanism may "fail" — the ratio of output probabilities on adjacent databases may exceed e^{ε}. On the remaining $1 - \delta$ probability mass, the standard e^{ε} bound holds.

Quantifier order: pure vs. approximate. There is a subtle but important difference:

- **Pure DP.** Fix x; simultaneously for *all* adjacent y and all S, the ratio condition holds with probability 1.
- **Approximate DP.** Fix x and y; a priori the probability of a "bad" event is at most δ. But an adaptive adversary who chooses y *after* seeing the output might be able to find an adjacent pair that violates the ratio for that particular output.

Pure DP is strictly stronger. In practice, approximate DP is widely used because it enables the Gaussian mechanism and better composition bounds.

9.5 How Small Must δ Be?

The insurance argument: why $\delta = 1/n$ is not safe

Consider the following "mechanism" on a database of size n: select one uniformly random individual and publish their complete record. This satisfies $(0,\, 1/n)$-DP, yet it explicitly exposes one person's data with probability $1/n$.
From any single person's perspective, their chance of exposure is small. But consider an insurer covering the database operator against privacy-loss claims: this mechanism generates a privacy breach *every single time it is run*, and the insurer must pay damages on every run.

Rule of thumb. δ should be *cryptographically small*: decaying faster than the inverse of any polynomial in n. Values like $\delta = n^{-2}$ or smaller are considered acceptable in practice.

9.6 ℓ_2 Sensitivity

The Gaussian mechanism adds noise calibrated to the ℓ_2 *sensitivity*, which can be substantially smaller than the ℓ_1 sensitivity used by Laplace.

Definition 9.3: ℓ_2 Sensitivity

For $f : \mathcal{X}^n \to \mathbb{R}^k$, the ℓ_2 *sensitivity* is

$$\Delta_2 f \;=\; \max_{\text{adj } x,\, y} \|f(x) - f(y)\|_2.$$

Comparison for counting queries. For k arbitrary counting queries, adding or removing one individual can change each count by at most 1, so $f(x) - f(y)$ has at most k nonzero entries each in $\{-1, 0, 1\}$.

- ℓ_1 sensitivity: $\Delta_1 f = k$ (worst case: one individual affects all k counts).
- ℓ_2 sensitivity: $\Delta_2 f = \sqrt{k}$ (since $\|\cdot\|_2 \le \sqrt{k}\,\|\cdot\|_\infty$ and each coordinate is at most 1).

For k arbitrary counting queries:

$$\Delta_1 f = k, \qquad \Delta_2 f = \sqrt{k}.$$

The ℓ_2 sensitivity is smaller by a factor of $\sqrt{k}$.

9.7 The Gaussian Mechanism

Definition 9.4: Gaussian Mechanism

Let $f : \mathcal{X}^n \to \mathbb{R}^k$. For privacy parameters $\varepsilon, \delta > 0$, the *Gaussian mechanism* releases

$$M(x) \;=\; f(x) + (Y_1, \ldots, Y_k), \qquad Y_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,\, \sigma^2),$$

where

$$\sigma \;=\; \frac{\Delta_2 f \cdot \sqrt{2\ln(1.25/\delta)}}{\varepsilon}.$$

Theorem 9.1: Gaussian Mechanism Privacy

With σ as defined above, the Gaussian mechanism satisfies (ε, δ)-differential privacy.

Why ℓ_2 sensitivity? The mechanism adds independent Gaussian noise of the same variance to each coordinate. The privacy loss depends on $\|f(x) - f(y)\|_2$: the ℓ_2 distance between the outputs on adjacent databases. Calibrating σ to $\Delta_2 f$ ensures coverage of the worst-case pair.

9.8 Accuracy of the Gaussian Mechanism

For a single coordinate $Y_i \sim \mathcal{N}(0, \sigma^2)$, standard Gaussian tail bounds give

$$\Pr[|Y_i| \geq t\sigma] \;\leq\; 2e^{-t^2/2}.$$

Applying a union bound over all k coordinates:

Theorem 9.2: Gaussian Accuracy (k Queries)

With probability at least $1 - \beta$, the Gaussian mechanism satisfies

$$\max_{1 \leq i \leq k} |M(x)_i - f(x)_i| \;=\; O\left(\frac{\Delta_2 f}{\varepsilon} \sqrt{\ln \frac{k}{\beta} \cdot \ln \frac{1}{\delta}}\right).$$

9.9 Laplace vs. Gaussian: A Comparison

For k arbitrary counting queries (so $\Delta_1 f = k$, $\Delta_2 f = \sqrt{k}$):

	Laplace	**Gaussian**
Privacy guarantee	Pure ε-DP	(ε, δ)-DP
Noise scale	$\Delta_1 f / \varepsilon$	$\Delta_2 f \cdot \sqrt{\ln(1/\delta)}/\varepsilon$
Savings for k queries	—	factor $\sqrt{k}$
Queries must be fixed?	No (online)	No (online)

Gaussian noise scales like $\sqrt{k}/\varepsilon$ instead of k/ε, matching the improvement from advanced composition — but at the cost of approximate rather than pure DP.

9.10 Connection to Advanced Composition

Advanced composition and the Gaussian mechanism both exploit the same phenomenon: with high probability, privacy loss accumulates like $O(\sqrt{k}\,\varepsilon)$, not $k\varepsilon$.

Unified picture: square-root savings

- **Advanced composition (pure DP):** Run k mechanisms each ε-DP. Cumulative loss is $O\left(\sqrt{k \ln(1/\delta)}\,\varepsilon\right)$ with probability $1 - \delta$.
- **Gaussian mechanism (approximate DP):** Answer k queries with ℓ_2-calibrated noise. Error per query scales like $\sqrt{k}/\varepsilon$.

Both results pay a small failure probability δ in exchange for a $\sqrt{k}$ improvement over the naive

$k\varepsilon$ bound. In both cases δ should be cryptographically small.

Chapter 10

Sparse Vector and Private Multiplicative Weights

10.1 Motivation: Answering Many Queries Cheaply

So far every mechanism we have studied pays a privacy cost proportional to the number of queries answered. Sequential composition says k queries each at privacy level ε costs $k\varepsilon$ total. Can we do better when most queries have values that fall below a threshold?

The key observation: *reporting that an answer is below a threshold is cheap.* If a verifier only needs to know *which* queries are above a threshold T, we can inspect many queries privately while paying full privacy cost only for the ones we actually reveal.

10.2 The Sparse Vector Technique

Definition 10.1: Above-Threshold (Sparse Vector)

Given a sequence of sensitivity-1 queries $q_1, q_2, \ldots$ on dataset x, a threshold T, a count limit c, and privacy parameter ε:

1. Sample a noisy threshold $\hat{T} \leftarrow T + \mathrm{Lap}\left(\frac{2}{\varepsilon}\right)$.
2. For each query q_i:
   - Sample $\nu_i \leftarrow \mathrm{Lap}\left(\frac{4c}{\varepsilon}\right)$.
   - If $q_i(x) + \nu_i \geq \hat{T}$: output $\top$, decrement c.
   - Else: output $\bot$.
   - Halt when $c = 0$.

Theorem 10.1: SVT Privacy

Above-Threshold satisfies ε-differential privacy.

Proof sketch. The crucial insight is that $\bot$ outputs reveal essentially nothing about x: saying "this query is below the noisy threshold" is a statement about noise, not data. The formal argument proceeds by noting that:

- The noisy threshold $\hat{T}$ is sampled once and depends on x only through the fixed T; adjacent datasets shift the threshold by at most 1 (sensitivity of T).

- Each $\top$ output involves comparing $q_i(x) + \nu_i$ to $\hat{T}$. Changing one person shifts $q_i(x)$ by at most 1, absorbed by noise of scale $4c/\varepsilon$.
- $\perp$ outputs are never re-sampled, so they leak nothing about x beyond the already-noisy comparison.

A coupling argument over adjacent x, y bounds the log-likelihood ratio by ε.

Key insight

We may privately inspect arbitrarily many candidate queries but pay full privacy cost only for the (at most c) queries that cross the threshold. Rejections are "free" in the privacy budget. This mirrors the logic of Report Noisy Max: the mechanism compresses its output, and the privacy analysis exploits that compression.

Accuracy. If a query q_i satisfies $q_i(x) \geq T + \alpha$ (well above threshold), it is reported $\top$ with high probability; if $q_i(x) \leq T - \alpha$ (well below), it is reported $\perp$ with high probability. The error probability for a single query is $e^{-\Omega(\alpha\varepsilon/c)}$, so for $c = O(1)$ and m queries, the overall error is small when $\alpha = O\big(\frac{c}{\varepsilon} \log m\big)$.

Application: private query release. If an analyst submits queries one at a time and only cares which ones are significant (above threshold), SVT answers all of them at the same privacy cost as answering c individually. This is useful when the "interesting" queries are sparse among a large pool of candidates.

10.3 Private Multiplicative Weights (PMW)

Sparse vector tells us *which* queries are significant; Private Multiplicative Weights uses those answers to build a *synthetic database* that answers future queries accurately.

Setting. We have a true dataset x over a universe $\mathcal{U}$ of records, and we want to maintain a *candidate distribution* $\mathbf{z}$ over $\mathcal{U}$ such that for any future linear query q, $q(\mathbf{z}) \approx q(x)$.

Algorithm. Initialize $\mathbf{z}^{(0)}$ to the uniform distribution over $\mathcal{U}$. At each round t:

1. The analyst submits query q_t.
2. If $|q_t(\mathbf{z}^{(t)}) - q_t(x)| \leq \alpha$ (small error): output the noisy answer and do not update ($\mathbf{z}^{(t+1)} = \mathbf{z}^{(t)}$).
3. Otherwise (*significant mistake*): update via multiplicative weights:

$$\mathbf{z}_u^{(t+1)} \;\propto\; \mathbf{z}_u^{(t)} \cdot \exp\Big(-\eta\, q_t(u) \cdot \operatorname{sgn}\big(q_t(\mathbf{z}^{(t)}) - q_t(x)\big)\Big),$$

   where $\eta > 0$ is a learning rate. This multiplicatively *downweights* records u that contribute to the mistake.

Potential function analysis. Define the *potential*

$$\Phi_t \;=\; \mathrm{KL}\Big(x \,\|\, \mathbf{z}^{(t)}\Big) \;=\; \sum_u x_u \ln \frac{x_u}{\mathbf{z}_u^{(t)}} \;\geq\; 0.$$

Two facts drive the analysis:

- **Upper bound:** $\Phi_0 = \mathrm{KL}(x\|\mathbf{z}^{(0)}) \le \ln|\mathcal{U}|$ (uniform initialization maximizes entropy).
- **Descent:** each significant update decreases Φ_t by at least $\Omega(\eta\alpha)$.

Theorem 10.2: PMW Convergence

The number of significant updates (rounds in which the algorithm updates $\mathbf{z}$) is at most

$$T^* \;=\; O\!\left(\frac{\ln|\mathcal{U}|}{\eta^2}\right).$$

Proof. Each significant mistake means $|q_t(\mathbf{z}^{(t)}) - q_t(x)| > \alpha$, which causes Φ_t to drop by at least $\eta\alpha - O(\eta^2)$ (by a Taylor expansion of ln after the multiplicative update, setting $\eta = \alpha/2$). Since $\Phi_t \ge 0$ and $\Phi_0 \le \ln|\mathcal{U}|$, there can be at most $\ln|\mathcal{U}|/(\eta\alpha - O(\eta^2)) = O(\ln|\mathcal{U}|/\eta^2)$ such drops.

Privacy. Each significant round uses one noisy answer (via the Laplace mechanism at scale $O(1/(\varepsilon n))$) to decide whether to update. By the convergence theorem, only T^* such answers are ever used, so the total privacy cost is $T^* \cdot \varepsilon_0$ for a per-query budget ε_0. Choosing $\varepsilon_0 = \varepsilon/T^*$ gives ε-DP overall.

Accuracy. After the synthetic database stabilizes (no more significant updates), it answers any query in the query class to within error α. Setting $\alpha = O(\sqrt{\ln|\mathcal{U}|}/\sqrt{n})$ and the noise scale accordingly gives:

Theorem 10.3: PMW Accuracy

For any class $\mathcal{Q}$ of linear queries, PMW with appropriate parameters outputs a synthetic database $\mathbf{z}$ such that

$$\max_{q\in\mathcal{Q}} \big|q(\mathbf{z}) - q(x)\big| \;=\; O\!\left(\frac{(\ln|\mathcal{U}|)^{1/2}}{n^{1/2}}\right)$$

with high probability, under ε-differential privacy.

SVT + PMW together

Sparse Vector and PMW are complementary. SVT identifies *which* queries the current hypothesis answers badly (using privacy budget only for those). PMW uses those bad queries to update the hypothesis. Together they implement a private boosting loop: repeatedly find a hard query, update the synthetic data, repeat — until no hard query remains. This is the essence of the SmallDB algorithm (Chapter 7).

10.4 Paper Readings

Dwork, Naor, Reingold, Rothblum, Vadhan (2009) — *On the Complexity of Differentially Private Data Release.* Introduces the Sparse Vector Technique as a tool for answering many queries while charging privacy cost only for those whose answers cross a significance threshold. The main result characterizes when private data release with low error is computationally feasible, and shows that restricting to above-threshold queries dramatically reduces the effective privacy cost relative to answering everything. The SVT mechanism analyzed in this chapter is a direct instantiation of these ideas.

Hardt and Rothblum (2010) — *A Multiplicative Weights Mechanism for Privacy-Preserving Data Analysis.* Shows that the private multiplicative weights algorithm achieves near-optimal accuracy for releasing all linear queries over a data universe under differential privacy. The key insight is a potential function argument: the KL divergence from the true data distribution to the current synthetic hypothesis decreases by a fixed amount per significant update, bounding the total number of corrective rounds by $O(\log |\mathcal{U}|/\eta^2)$. This gives the first efficient differentially private synthetic data generator with provable accuracy guarantees for an exponentially large query class.

Chapter 11

The US Census, Reconstruction, and Why DP Was Needed

11.1 Origin Story

When Cynthia Dwork and colleagues began thinking about privacy for statistical analysis of large datasets around 2001, the model they had in mind was the US Census. The Census is a great cause: it counts the population, determines political representation, and allocates hundreds of billions of dollars in federal funds—with no commercial interest involved. Crucially, unlike most data holders, the Census Bureau is *legally mandated* to protect confidentiality, providing a genuine customer for rigorous mathematical privacy guarantees.

The connection between differential privacy and the Census developed over several years. Adam Smith (later a co-inventor of DP) was among the early collaborators who recognized that the Bureau's existing confidentiality methods—while well-intentioned—could not withstand the kind of reconstruction attacks that were becoming computationally feasible. In 2017, the Bureau carried out internal reconstruction tests, confirmed that their prior methods were insufficient, and decided to modernize using differential privacy. The 2020 decennial census was the result.

11.2 Why the Census Matters

The decennial census is a constitutionally mandated enumeration of all people—not just citizens—living in the United States. Its outputs drive:

- **Congressional apportionment.** The 435 seats in the House of Representatives are allocated among states by the Huntington–Hill algorithm based on state population totals. When one state loses population and another gains, a seat transfers.
- **Electoral college.** Each state's electoral votes equal its House seats plus two senators—directly derived from Census counts.
- **Redistricting.** Within each state, congressional and legislative district boundaries are drawn to equalize population ($\leq 5\%$ variation is the judicial standard). These numbers also feature in Voting Rights Act lawsuits about minority representation.
- **Federal funding.** Hundreds of billions of dollars in formula-driven programs (education, infrastructure, housing) flow to states and localities based on population counts.
- **Policy research.** Researchers and policymakers use fine-grained demographic data to measure disparities, plan school resources, and target public services.

Marginalized populations—undocumented immigrants, people experiencing homelessness, communities with historical distrust of government—are systematically harder to enumerate. States that invest more in outreach get more accurate (and advantageous) counts.

11.3 What the Census Collects

The 2020 decennial census form collected, for each person in each household:

- **Name** and whether the person normally lives elsewhere (college students, incarcerated individuals, and rehab residents are counted at their current location).
- **Relationship to person 1** (spouse/partner, biological/adopted child, parent-in-law, roommate, foster child, etc.).
- **Sex** (two options in 2020).
- **Age and date of birth.**
- **Ethnicity**: Hispanic/Latino origin (Mexican, Puerto Rican, Cuban, etc.) or not. In Census terminology "ethnicity" always means Hispanic or non-Hispanic.
- **Race**: checkboxes for White (with origin detail), Black or African American, American Indian or Alaska Native (with tribal affiliation), Chinese, Filipino, Asian Indian, Vietnamese, Korean, Japanese, other Asian, Native Hawaiian, Samoan, Chamorro, other Pacific Islander, or some other race. Multiple boxes may be checked, yielding 63 possible combinations.

What counts as a "race" on the Census form is explicitly political and has changed dramatically over time—Jews and Italians have at various points been proposed as separate races. Race is always self-identification; people change their declared race between censuses, and the Bureau does not challenge such changes.

11.4 Legal Confidentiality Requirements

The Census is governed by Title 13 of the US Code, which imposes strict confidentiality obligations:

Title 13 protections (key provisions)

- Every Census Bureau employee takes a **lifetime oath** of confidentiality—binding even after leaving the Bureau.
- **Penalties**: disclosing any personal information carries up to five years in prison, a $250,000 fine, or both.
- Census data **may not be shared with any other part of government** for law enforcement or any non-statistical purpose. (This was enacted after the WWII internment of Japanese Americans, in part enabled by Census data.)
- The Secretary of Commerce **may publish statistics** (tabulations) but not anything that "discloses the information reported by or on behalf of any particular respondent."
- "In no case shall information furnished be used to the **detriment of any respondent**."

Because responding to the Census is legally required, confidentiality protections are essential for encouraging accurate responses, especially from people who fear the government. The Bureau does not do law enforcement; even people who are breaking the law should feel safe in reporting their true situation.

11.5 Why Privacy Harms Are Real

1. **Section 8 housing eviction.** Federal housing leases specify maximum household occupancy. If Census data reveals that a household has more occupants than their lease permits, the family could be evicted and become homeless—a direct, concrete harm from privacy compromise.

2. **Legal prosecution.** A family reports a 13-year-old girl where a prior census recorded a 3-year-old boy. In states where supporting gender re-identification is illegal, this could result in prosecution.

3. **Doxing and harassment.** Even if your neighbors know your household is a same-sex couple, a compiled national list of gay households is qualitatively different from distributed local knowledge. Information known in a distributed fashion has different safety implications than information in a centralized, searchable list that could be weaponized.

The Bureau's own rules: responses cannot be used to the detriment of any respondent. Each example above is a case where published statistics could, via reconstruction, be used to harm the very people who trusted the Bureau with their data.

11.6 Census Geography

The Bureau's fundamental geographic unit is the *census block*—the smallest unit, ranging from zero to roughly 2,000 inhabitants (the density of a city block). The Bureau uses the analogy of **pixels**: it is acceptable to blur the pixels a little, as in an Impressionist painting—when you step back, you still see the correct picture.

Blocks aggregate upward in a hierarchy:

$$\text{Block} \to \text{Block Group} \to \text{Census Tract} \to \text{County} \to \text{State} \to \text{Division} \to \text{Region}$$

Additional overlapping geographic entities—zip code tabulation areas, school districts, voting districts—are also constructed from block-level data. American Indian, Alaska Native, and Native Hawaiian areas have their own separate spine because these communities cross state boundaries and have very small populations that require special handling.

11.7 The Old Method: Record Swapping

Before differential privacy, the 2010, 2000, and 1990 censuses used a method called *record swapping*:

1. Identify "at-risk" households—those that stand out in their neighborhood (racial minorities in a non-diverse block, unusually large households, same-sex couples in an area without others like them, etc.).

2. Find a "match" household elsewhere in the country that agrees on total population and voting-age population.

3. Swap all the detailed records between the two households.

The exact algorithm was **secret**, may not have been fully algorithmic, and was not applied uniformly across demographic groups. The swapped dataset—the "100% data file"—was treated as the Bureau's official reference, and all tabulations were computed from it.

The problem. The swapping rate was low (typically a small fraction of households), which means the 100% data file was very close to the original Census Edited File (CEF). And as we will see next, no amount of swapping could protect against reconstruction from the sheer volume of statistics released.

11.8 Scale of the 2010 Census Release

From the 2010 census, the Bureau published:

- **150 billion** statistics in total.
- **86 billion** linearly independent statistics at the block level.
- **241 million** linearly independent statistics at the tract level.

This is a massive collection of accurate, fine-grained aggregate counts with essentially **no noise added for privacy**.

11.9 The Reconstruction Attack

The Bureau's internal team (later published in 2025 after security clearance) showed that published statistics could reconstruct the 100% data file.

BEARS fields. The attack focused on five fields (the acronym **BEARS**): **B**lock, **E**thnicity, **A**ge, **R**ace, **S**ex. Each individual can be characterized by their combination of BEARS values—their "person type."

The reconstruction procedure.

1. From the published tabulations, extract the count of each BEARS person-type in each block. This yields a large system of linear/integer constraints that any consistent microdata set must satisfy.
2. Feed the constraints to a commercial integer program solver to reconstruct a candidate dataset $\hat{X}$ consistent with the published statistics.
3. The reconstructed dataset $\hat{X}$ is called the *Reconstructed 100% Data File* (RHDF).

Solution variability. The solver may find multiple solutions. The attackers exploited this by solving a second integer program: given the published solution, find the dataset Z maximizing distance from it—but the true dataset is also a valid solution. If this maximum distance is small, the true dataset cannot be far from the reconstruction. *The attacker can certify how accurate the attack was using only public information.*

In practice, most blocks had only one solution consistent with the published statistics, meaning the published data uniquely determined individual-level BEARS information for most people.

Re-identification. Once BEARS fields are reconstructed, the attacker links the reconstructed dataset to commercial databases (voter rolls, credit records, etc.) that share block, age, race, and sex. Reconstruction followed by re-identification is then trivial.

Connection to the Fundamental Law

This attack is exactly the Fundamental Law of Information Recovery (Chapter 5) in action. The Bureau released overly accurate answers to far too many statistics—86 billion at the block level alone. By the Dinur–Nissim reconstruction theorem, this was blatantly non-private, regardless of the swapping applied beforehand. The Bureau knew about the 2003 reconstruction theorem before the 2010 census; the techniques to exploit it had simply become faster and cheaper.

The lawyers' reaction. When the Bureau's statisticians and computer scientists presented these findings, the lawyers asked: "Can we retract the 2010 Census publications?" The data were already on the public web and could not be clawed back. The lawyers concluded: "We can't do this again." The Bureau's argument to itself: the distinction between publishing microdata (protected by subsampling) and publishing tabular statistics (no explicit protection) is *not justified*. From enough accurate tabulations, one can reconstruct microdata without any sampling secrecy.

11.10 Why Swapping Was Not Enough

The Bureau's own argument: since the RHDF closely reproduced the 100% data file (post-swapping), and the 100% data file was already very close to the original CEF (swapping was low-rate), the reconstruction was essentially recovering the true census microdata. The Bureau decided this violated its own confidentiality mandate.

No amount of swapping can protect against reconstruction when the number of published statistics exceeds the effective entropy of the dataset.

11.11 Concentrated Differential Privacy

The scale of the Census—billions of statistics, millions of geographic units, tight accuracy requirements for redistricting—motivated a variant of differential privacy called *concentrated differential privacy*.

Motivation. Recall from advanced composition (Chapter 8): to achieve (ε, δ)-DP over k mechanisms, each individual mechanism must use noise of order $\varepsilon/(\sqrt{k \ln(1/\delta)})$. The $\sqrt{\ln(1/\delta)}$ factor is annoying: when δ must be cryptographically small, this factor can be as large as 10, forcing $10\times$ more noise.

The key insight: the $\varepsilon\delta$ guarantee is itself probabilistic. Why require each building block to *never* exceed $\varepsilon/(\sqrt{k}\sqrt{\ln 1/\delta})$? If in expectation each step contributes $\varepsilon/\sqrt{k}$, and the excess cancels out, we get the same asymptotic behavior without the $\sqrt{\ln 1/\delta}$ penalty.

Definition. Informally, *concentrated differential privacy* (CDP) requires that the privacy loss random variable $\mathcal{L}$ behaves like a Gaussian:

$$\forall k: \quad \Pr[\mathcal{L} > k\varepsilon] \;\leq\; e^{-\Theta(k^2)}.$$

This is exactly the tail behavior of a Gaussian distribution—hence "concentrated."

Why Gaussian noise achieves CDP

Adding noise from $\mathcal{N}(0, \sigma^2)$ makes the privacy loss random variable itself sub-Gaussian with mean proportional to σ^{-2}. This is why the Gaussian mechanism naturally fits the CDP framework.

Zero-concentrated DP (zCDP). Dwork and Rothblum introduced an initial version of CDP. Bun and Steinke later introduced *zero-concentrated differential privacy* (zCDP), using a slightly different characterization of sub-Gaussian random variables that achieves exact closure under post-processing. **zCDP is the variant used in the 2020 Census.**

Practical benefit. Under zCDP, composing k mechanisms eliminates the $\sqrt{\ln(1/\delta)}$ overhead entirely. For the Census, where δ must be extremely small, this represents a factor of up to $10\times$ reduction in the noise that must be added—a massive accuracy gain for redistricting.

11.12 The Lawsuits: Why 16 States Sued

Following the Census Bureau's adoption of differential privacy for the 2020 decennial census, sixteen states—predominantly Republican-led—brought suit, arguing that DP produced unreliable data. The stated technical argument:

> "Because differential privacy creates false information by design, it prevents the states from accessing municipal-level information crucial to performing this essential government function."

A few observations on this argument:

State-level counts are preserved. Differential privacy in the 2020 Census (the TopDown algorithm) preserves exact state-level population totals, which are the constitutional basis for House apportionment and electoral college votes. DP noise is applied only at finer geographic levels. Any accuracy concerns about redistricting must be weighed against the accuracy gains from having a mathematically rigorous, auditable guarantee, versus the old swapping method whose error was uncharacterized and secret.

11.13 Paper Readings

Abowd et al. (2025) — *The 2010 Census Confidentiality Protection Evaluation.* Using only publicly released 2010 Census tabulations, the authors reconstructed the BEARS fields (block, ethnicity, age, race, sex) for essentially all individuals in the 100% data file, then linked the reconstruction to commercial databases to re-identify a large fraction of the population. The paper was classified for years due to its sensitivity and released only after the 2020 Census was finalized; it provided the definitive technical justification for the Bureau's adoption of differential privacy.

Abowd et al. (2022) — *The 2020 Census Disclosure Avoidance System.* Full specification of the TopDown algorithm used in the 2020 Census: a hierarchical allocation of a global zCDP privacy budget that processes the geographic hierarchy from nation down to census block, using integer programming at each level to ensure consistency of counts across geographic units. The algorithm and its code are publicly available on GitHub, and it was tested against 2010 data during an extended public comment period — allowing affected communities, including Native American tribes whose geographic units cross state lines, to advocate for changes in budget allocation before the 2020 enumeration.

Chapter 12

DP Protects Against Overfitting

12.1 Friedman's Paradox

Consider a standard regression setting. We have a design matrix $X \in \mathbb{R}^{n \times d}$ where each row is an observation (a person) and each column is a feature. We observe an outcome vector $Y \in \mathbb{R}^n$ and believe it satisfies

$$Y = X\beta + \varepsilon$$

for some true weight vector β and small noise ε. A natural data-analysis workflow is:

1. Fit β by minimizing $\|\varepsilon\|$.
2. Inspect the coefficients: drop features whose fitted weight is near zero ("clearly irrelevant").
3. Refit the equation on the remaining features and report a clean model.

This seems entirely reasonable. Friedman's paradox shows it is catastrophically wrong.

The setup. Let every entry of X be an independent draw from $\mathcal{N}(0,1)$, and let Y also be a vector of independent $\mathcal{N}(0,1)$ draws. There is *no real relationship* between X and Y whatsoever.

The problem. Even so, the procedure above finds "significant" predictive features. The reason is anti-concentration: by ordinary sampling variation, some features will, purely by accident, be slightly further from their expected value than others. The procedure *selects* those accidentally-extreme features and then declares victory — but the relationship is a statistical fluke, not a real signal.

Formally: we are implicitly taking the *maximum* over many random variables. The maximum of d i.i.d. mean-zero variables is not mean-zero; it is $\Theta(\sqrt{\log d})$. By keeping only the features with large fitted weights, we are conditioning on a high-probability event of the noise, then treating the conditioned noise as signal.

Key insight

The methodological error is *adaptive feature selection*: the decision of which features to keep is made by looking at the data, and then the same data are used to evaluate the selected features. This is a form of double-dipping.

12.2 Exploratory vs. Adaptive Data Analysis

In statistics, the general practice of "looking around in the data for a signal before deciding what to ask" is called **exploratory data analysis (EDA)**. It is ubiquitous and has no universally accepted principled remedy.

In computer science the same phenomenon goes by a different name: **adaptive data analysis**. The analyst adapts the next question to the answers received so far. This dependence between queries and the data set is the root cause of overfitting.

Why naivety and malice are indistinguishable. Friedman's paradox shows that a completely well-intentioned analyst using a standard procedure can produce results indistinguishable from deliberate data manipulation. We therefore cannot rely on the good intentions of analysts when designing statistical methodology: we must build defenses that are robust even against the worst case.

Overfitting, formally.

Definition 12.1: Overfitting

A predictor, classifier, or hypothesis *overfits* if it identifies a pattern that is specific to the data set used to find it and does not generalize to the distribution from which that data set was drawn.

Concretely: if a trained classifier achieves 95% accuracy on the training set but only 55% accuracy on fresh draws from the same distribution, it has overfit. The training effort was wasted; the model learned noise, not signal.

12.3 The Accuracy Adversary

We formalize the problem by defining an adversary whose sole goal is to cause overfitting.

Setup. Fix a universe $\mathcal{X}$, a distribution $\mathcal{D}$ on $\mathcal{X}$, a data set $X = (x_1, \dots, x_n)$ drawn i.i.d. from $\mathcal{D}$, and a class $\mathcal{Q}$ of *counting queries*: functions $q : \mathcal{X} \to \{0, 1\}$.

For a query q define:

$$\hat{\mu}(q, X) = \frac{1}{n} \sum_{i=1}^{n} q(x_i) \qquad \text{(empirical mean on } X\text{)} \qquad \mu(q) = \mathop{\mathbb{E}}_{z \sim \mathcal{D}}[q(z)] \qquad \text{(true distributional mean)}.$$

Definition 12.2: Accuracy Adversary

The *accuracy adversary* is a computationally unbounded algorithm $\mathcal{A}$ that interacts adaptively with a mechanism M operating on X. Its goal is to find a query $q \in \mathcal{Q}$ such that

$$\left|\hat{\mu}(q, X) - \mu(q)\right| > \tau,$$

i.e., to find a query for which the data set X is *not* τ*-representative* of the population. The threshold $\tau > 0$ is a parameter chosen by the analyst (e.g., $\tau = 0.03$ corresponds to a polling accuracy of $\pm 3\%$).

The adversary is modeled as purely malicious for the same reason cryptographic security definitions assume the worst-case attacker: it makes the guarantee unconditional. An honest-but-naive analyst is captured as a special case.

12.4 Non-Adaptive Baseline: Chernoff Bound

If the queries are fixed *before* the data set is collected (non-adaptive), standard concentration bounds already give strong guarantees.

Theorem 12.1: Non-Adaptive Accuracy (Chernoff Bound)

For any fixed query q and data set X of size n drawn i.i.d. from $\mathcal{D}$:

$$\Pr\left[|\hat{\mu}(q, X) - \mu(q)| > \tau\right] \;\leq\; 2e^{-2n\tau^2}.$$

By a union bound over m fixed queries, to ensure that all m queries are simultaneously τ-accurate with probability $\geq 1 - \beta$, it suffices to have

$$n = O\left(\frac{\log(m/\beta)}{\tau^2}\right).$$

This grows only *logarithmically* in the number of queries.

The key word is "fixed in advance." This guarantee fails completely if queries are chosen adaptively based on answers to previous queries.

12.5 The Killer Query

To see why adaptivity destroys non-adaptive accuracy guarantees, suppose the adversary has somehow learned the data set X exactly. Define:

$$q^*(z) = \begin{cases} 1 & z \in X \\ 0 & z \notin X. \end{cases}$$

Then:

$$\hat{\mu}(q^*, X) = 1 \qquad \text{(every element of } X \text{ maps to 1)}$$

$$\mu(q^*) = \Pr_{z \sim \mathcal{D}}[z \in X] = \frac{n}{|\mathcal{X}|} \approx 0 \quad \text{(data set is a tiny fraction of the universe).}$$

The gap is ≈ 1, which exceeds any reasonable τ. The killer query is trivial to construct once the data set is known.

How might an adversary learn X? Via a *coding query*: define $q_{\text{code}}(z) = 2^{-z}$ (treating elements as integers), then $\sum_{z \in X} 2^{-z}$ is a binary encoding of X. Alternatively, the adversary could launch a database reconstruction attack (as studied in Chapter 11). Either way: **if the adversary learns X, the game is over**.

This motivates our goal: answer queries in a way that prevents the adversary from learning too much about X, even adaptively.

12.6 Formal Model of Adaptive Data Analysis

The adaptive interaction. The adversary $\mathcal{A}$ proceeds in rounds:

1. Choose query Q_1 (with no access to X; this round is trivially non-adaptive).
2. Receive answer $A_1 = M(X, Q_1)$ from mechanism M.
3. Choose Q_2 based on A_1. Receive $A_2 = M(X, Q_2)$.

4. ...

5. Choose Q_i based on $(A_1, \ldots, A_{i-1})$. Receive A_i.

The adversary wins if any Q_i is bad for X (i.e., $|\hat{\mu}(Q_i, X) - \mu(Q_i)| > \tau$).

Why the first query is safe. At round 1, $\mathcal{A}$ has not seen any answer from $M(X, \cdot)$. Therefore Q_1 is completely independent of X. By the Chernoff bound, $\Pr[X \text{ is bad for } Q_1] \leq 2e^{-2n\tau^2}$, regardless of the adversary's strategy.

Simple solution: data splitting. Reserve $1/\tau^2$ fresh samples for each query. Answer Q_i using its own reserved subsample (then discard those samples). This handles $n\tau^2$ queries but wastes data linearly. We want exponentially better.

12.7 Max-Information

The key to proving that differential privacy prevents overfitting is a new information-theoretic quantity introduced by Dwork et al.

Definition 12.3: Max-Information

Let $(\mathbf{D}, \mathbf{\Phi})$ be jointly distributed random variables, where $\mathbf{D}$ is a random data set and $\mathbf{\Phi}$ is a random query (or more generally, any random variable correlated with $\mathbf{D}$ via adaptive interaction). The *max-information* $I_\infty(\mathbf{D}; \mathbf{\Phi})$ is the smallest $k \geq 0$ such that for all d in the support of $\mathbf{D}$ and all φ in the support of $\mathbf{\Phi}$:

$$\Pr[\mathbf{D} = d \mid \mathbf{\Phi} = \varphi] \;\leq\; 2^k \cdot \Pr[\mathbf{D} = d].$$

Intuition.

- $k = 0$: $\mathbf{D}$ and $\mathbf{\Phi}$ are completely independent. Observing the query tells you nothing about the data set.
- $k > 0$: observing $\mathbf{\Phi} = \varphi$ can increase the probability of any particular data set by at most a factor of 2^k. As k grows, the query leaks more about the data set (and vice versa).

Connection to mutual information. Max-information is a worst-case variant of mutual information $I(\mathbf{D}; \mathbf{\Phi})$ from information theory. Standard mutual information measures the *average* reduction in uncertainty; max-information measures the *worst-case* increase in log-probability. The symmetry $I_\infty(\mathbf{D}; \mathbf{\Phi}) = I_\infty(\mathbf{\Phi}; \mathbf{D})$ follows from Bayes' theorem: the condition $\Pr[\mathbf{D} = d \mid \mathbf{\Phi} = \varphi] \leq 2^k \Pr[\mathbf{D} = d]$ is equivalent to $\Pr[\mathbf{\Phi} = \varphi \mid \mathbf{D} = d] \leq 2^k \Pr[\mathbf{\Phi} = \varphi]$ (up to the same constant k), by a Bayes' theorem rearrangement.

12.8 Differential Privacy Bounds Max-Information

Theorem 12.2: DP Bounds Max-Information

Let M be an ε-differentially private mechanism operating on data sets of size n. Let $\mathbf{\Phi}$ be the random query produced by any adaptive adversary interacting with M, and let $\mathbf{D}$ be the random data set. Then:

$$I_\infty(\mathbf{D}; \mathbf{\Phi}) \;\leq\; O(\varepsilon n).$$

Proof. We show $\Pr[\mathbf{D} = d \mid \mathbf{\Phi} = \varphi] \le e^{\varepsilon n} \cdot \Pr[\mathbf{D} = d]$ for all d, φ, which gives $k = O(\varepsilon n)$ after taking $\log_2$.

Step 1 (Averaging argument). By definition of conditional probability:

$$\Pr[\mathbf{\Phi} = \varphi] = \sum_{d'} \Pr\big[\mathbf{\Phi} = \varphi \mid \mathbf{D} = d'\big] \cdot \Pr\big[\mathbf{D} = d'\big].$$

This is an average of $\Pr[\mathbf{\Phi} = \varphi \mid \mathbf{D} = d']$ over d'. Therefore there exists some d^* achieving at most the average:

$$\Pr[\mathbf{\Phi} = \varphi] \;\ge\; \Pr[\mathbf{\Phi} = \varphi \mid \mathbf{D} = d^*]. \tag{1}$$

Step 2 (Group privacy). Any two data sets d and d' of size n differ on at most n entries. By group privacy of ε-DP applied n times:

$$\Pr[\mathbf{\Phi} = \varphi \mid \mathbf{D} = d] \;\le\; e^{\varepsilon n} \cdot \Pr[\mathbf{\Phi} = \varphi \mid \mathbf{D} = d^*]. \tag{2}$$

Combining (1) and (2).

$$\Pr[\mathbf{\Phi} = \varphi \mid \mathbf{D} = d] \;\le\; e^{\varepsilon n} \cdot \Pr[\mathbf{\Phi} = \varphi].$$

By Bayes' theorem (symmetry of max-information), this is equivalent to

$$\Pr[\mathbf{D} = d \mid \mathbf{\Phi} = \varphi] \;\le\; e^{\varepsilon n} \cdot \Pr[\mathbf{D} = d]. \qquad \square$$

The proof uses exactly two properties of differential privacy: *post-processing* (the adversary's query choice is a post-processing of the DP outputs) and *group privacy* (all data sets are within n substitutions of each other).

12.9 DP Protects Against Overfitting

We now combine the max-information bound with the standard Chernoff sampling assumption to prove the main result.

Assumption. For a fixed query q, the probability over a randomly drawn data set X that X is *bad* for q (i.e., not τ-representative) is at most 2^{-cn} for some constant $c > 0$ (this follows from the Chernoff bound with $c = 2\tau^2/\ln 2$). Let B_q denote the set of data sets that are bad for q.

Theorem 12.3: DP Protects Against Overfitting

Let M be ε-differentially private with $\varepsilon \le c/2$. Let $\mathbf{\Phi}$ be the random adaptive query produced by any adversary interacting with M. Then:

$$\Pr_{\mathbf{D},\mathbf{\Phi}}\big[\mathbf{D} \in B_{\mathbf{\Phi}}\big] \;\le\; 2^{\varepsilon n - cn} \;\le\; 2^{-cn/2},$$

which is exponentially small in n.

Proof.

$$\begin{aligned}
\Pr[\mathbf{D} \in B_{\mathbf{\Phi}}] &= \sum_{\varphi} \Pr[\mathbf{\Phi} = \varphi] \cdot \Pr[\mathbf{D} \in B_\varphi \mid \mathbf{\Phi} = \varphi] && \text{(law of total probability)} \\
&\le \sum_{\varphi} \Pr[\mathbf{\Phi} = \varphi] \cdot \sum_{d \in B_\varphi} \Pr[\mathbf{D} = d \mid \mathbf{\Phi} = \varphi] \\
&\le \sum_{\varphi} \Pr[\mathbf{\Phi} = \varphi] \cdot \sum_{d \in B_\varphi} e^{\varepsilon n} \Pr[\mathbf{D} = d] && \text{(by Theorem 12.2)} \\
&= e^{\varepsilon n} \sum_{\varphi} \Pr[\mathbf{\Phi} = \varphi] \cdot \Pr[\mathbf{D} \in B_\varphi] && \text{(regroup)} \\
&\le e^{\varepsilon n} \cdot 2^{-cn} && \text{(Chernoff assumption: } \Pr[\mathbf{D} \in B_\varphi] \le 2^{-cn} \text{ for every fixed } \varphi) \\
&= 2^{\varepsilon n - cn}.
\end{aligned}$$

Setting $\varepsilon \le c/2$ gives $2^{-cn/2}$, which is exponentially small. □

The key intellectual leap. Overfitting is reframed as a *win for the accuracy adversary*. Once we define the adversary formally, standard tools from differential privacy — post-processing, group privacy, and composition — give a complete, quantitative, and mechanistic explanation of *why* DP prevents overfitting.

12.10 The Reusable Holdout

Standard holdout (and its failure mode). The classical defense against overfitting is to reserve a *holdout set* H that is never used during training. The analyst works freely on the training set T, derives a hypothesis, and evaluates it once on H. If H gives a good result, the hypothesis is accepted.

The problem: if the hypothesis fails on H, the analyst wants to try again. But now the choice of what to try next has been influenced by H's answer. The holdout set is *tainted* and cannot be safely reused. In practice, analysts use holdout sets many times, invalidating the statistical guarantees.

The fix: wrap H in a DP mechanism.

Reusable Holdout

Train freely on T (no privacy restrictions). But access the holdout set H *only through a differentially private mechanism*. Because DP prevents the adversary from learning too much about H, the holdout set can be queried repeatedly while maintaining valid statistical guarantees.

Conceptually: the accuracy adversary has the training set T hardwired into its strategy. Its only access to H is through the private mechanism. Theorem 12.3 then guarantees that no adaptive sequence of queries can overfit H.

12.11 The Algorithm: Numeric Sparse Vector

The reusable holdout has a concrete algorithmic instantiation. Given:

- A training set T and holdout set H,

- A privacy budget B (total number of DP queries to H),
- An overfitting budget $k_{\max}$ (number of detected overfits allowed),
- Parameters τ (accuracy threshold) and σ (noise level),

the algorithm handles each incoming statistical query $q : \mathcal{X} \to [-1, 1]$ as follows:

1. Draw a random sample $S \subseteq T$ (from the training set).
2. Compare the holdout answer $\hat{\mu}(q, H) + \text{noise}$ against the training estimate $\hat{\mu}(q, S)$.
3. If the two estimates are *close* (within a threshold $\tilde{\tau}$): output the training estimate and continue. No privacy budget is spent.
4. If the two estimates are *far apart* (the training answer is overfit): output the holdout estimate, decrement the overfitting budget, and continue.
5. If the overfitting budget is exhausted: halt.

Theorem 12.4: Numeric Sparse Vector Achieves Exponential Improvement

The reusable holdout algorithm can answer exponentially many queries in total, as long as at most $k_{\max}$ of them trigger the "overfit detected" branch. Only those $k_{\max}$ holdout accesses spend privacy budget; the rest are answered from the training set for free.

The connection to Sparse Vector. This algorithm is precisely **Numeric Sparse Vector** (a variant of the Sparse Vector Technique from Chapter 11), applied with the holdout mechanism as the private database and the analyst's queries as the input stream. Sparse Vector was introduced as an abstract privacy tool; here it reappears as the heart of a practically motivated reusable holdout mechanism.

12.12 Paper Readings

Dwork et al. (2015) — *Preserving Statistical Validity in Adaptive Data Analysis.* Shows that running adaptive statistical queries under differential privacy bounds the max-information I_∞ between the dataset and the sequence of query answers, which in turn implies that all reported p-values and confidence intervals remain honest. Classical statistical practice assumes queries are chosen independently of the data; this paper formalizes what goes wrong when they are not, and proves that DP is a sufficient condition for validity even under arbitrary adaptivity. The result reframes differential privacy as a tool for statistical integrity rather than purely for individual privacy, connecting the two halves of this course.

Chapter 13

Foundations of Modern Cryptography

13.1 What is Cryptography?

- **Cryptography:** building secure systems
- **Cryptanalysis:** breaking them
- **Cryptology:** both together

Big-picture takeaway

Modern cryptography studies how to achieve secrecy, authenticity, proof, randomness, privacy, and computation even in adversarial environments. *Cryptography makes the impossible possible.*

13.2 What Problems Does Cryptography Solve?

- **Secure communication:** secrecy (confidentiality), integrity (no tampering), authentication (know the sender).
- **Digital signatures:** authenticity + transferable proof of origin (not just convincing the receiver, but any third party).
- **Pseudorandom generators:** "stretch" a random seed to a longer string that "looks" random.
- **Collision-resistant hash functions:** hard to find $m \neq m'$ with $H(m) = H(m')$. In practice, signatures are applied to a hash of the message, not the full message.
- **Commitment schemes**: commit to a secret value z by publishing α that is *hiding* (reveals nothing about z) and *binding* (hard to open as a different value). Analogy: write a prediction on paper, seal it in an envelope. Neither party can alter it, but nobody can read it until the envelope is opened.
- **Private Information Retrieval (PIR):** librarian can't determine which book you read / which data you downloaded.
- **Homomorphic encryption:** "compute" on encrypted values.
- **Non-malleable encryption:** given $E(x)$ and function f, can't find $E(f(x))$.
- **Zero-knowledge proofs:** prove a statement without revealing why it is true.

13.3 Two Innovations of Modern Cryptography

13.3.1 Innovation 1: Base security on complexity theory

Instead of requiring information-theoretic impossibility, require only that breaking the scheme is computationally infeasible. The central primitive is a *one-way function*.

Definition 13.1 (One-way function, informal)**.** *f is one-way if: given x, easy to compute $f(x)$; given $f(x)$, computationally hard to find any x' with $f(x') = f(x)$.*

Hardness is **computational, not information-theoretic.**

A *trapdoor function* is a one-way function that becomes easy to invert given secret auxiliary information (the "trapdoor"). This is essential for public-key encryption: everyone can encrypt; only the receiver can decrypt.

13.3.2 Innovation 2: Security by reduction

Reduction paradigm

$$\text{Break primitive} \implies \text{Solve hard problem.}$$

If the underlying problem is believed hard, the primitive is secure. *"Science wins either way":* either we get a secure scheme, or a supposedly hard problem turns out to be easy.

13.4 Complexity-Theoretic Background

13.4.1 Asymptotic notation

An algorithm has *running time* $O(f(n))$ if there exist constants C and n_0 such that for all $n \geq n_0$, the running time is at most $C \cdot f(n)$. This captures the growth rate but hides constants and small-n behavior. Example: sorting n numbers requires $\Theta(n \log n)$ comparisons (both upper and lower bound).

13.4.2 Polynomial time and negligible functions

A problem is *efficiently solvable* if there exists a polynomial-time algorithm for it. In cryptography we want to argue the *opposite*: that no efficient algorithm can break a scheme. This requires care, since asserting "no polynomial-time algorithm exists" is an unproven conjecture for most problems of interest (and proving it would resolve P vs. NP).

We capture hardness probabilistically: a function $\mu(k)$ is *negligible* if it decreases faster than the inverse of any polynomial—for every polynomial p, there exists k_0 such that $\mu(k) < 1/p(k)$ for all $k > k_0$.

13.4.3 What does it mean for an algorithm to fail?

Saying an algorithm A *does not factor* integers requires precision. Options:

- Fails on *all* inputs of each size—too strong; easy inputs (even numbers) are trivially factorable.
- Fails on *all but a negligible fraction* of inputs of each size—the right notion. For every polynomial p and all sufficiently large k, A fails to factor a randomly chosen k-bit number with probability at least $1 - 1/p(k)$.

The second formulation correctly captures that factoring a random integer is hard even if a constant fraction of inputs are easy.

13.4.4 Uniform vs. non-uniform computation

A *Turing machine* (or equivalently, a fixed program in C) is a *uniform* model: one program of fixed length works for all input sizes. A *circuit family* is a *non-uniform* model: one circuit per input size, and each circuit can be different (and can have hard-coded solutions for specific instances). Non-uniform circuits are strictly more powerful; in cryptography we usually work with uniform (Turing machine) adversaries, though the distinction matters in some proofs.

13.5 One-Way Functions

Definition 13.1: One-Way Function (informal)

A function $f : \{0,1\}^* \to \{0,1\}^*$ is *one-way* if:

1. **Easy to compute**: $f(x)$ can be computed in polynomial time.
2. **Hard to invert**: for any polynomial-time algorithm A, the probability

$$\Pr_{x \leftarrow \{0,1\}^k}\left[f(A(f(x), 1^k)) = f(x)\right]$$

is negligible in k.

Security parameter. The string 1^k (a sequence of k ones) is fed to A to ensure it can run in time polynomial in k. Without this, A might not have enough time to read the implicit security level.
Valid inversions. Note that A wins if it produces *any* pre-image of $f(x)$, not necessarily x itself (since f need not be injective).

Candidate: integer factoring. Let $f(a, b) = a \cdot b$ for k-bit primes a, b. Computing f is easy (multiplication); inverting (factoring the product into its prime factors) is believed to be hard. RSA is built on this conjecture. *Note*: the naive formulation "given a random k-bit integer, factor it" is too weak because roughly half of all integers are even and trivially factorable. The correct hardness conjecture restricts to products of two random k-bit primes.

Candidate: lattice problems. Problems such as finding the shortest vector in a lattice, or learning with errors (LWE), have been studied for decades without yielding polynomial-time solutions. They form the basis of *post-quantum* cryptography—schemes believed to resist quantum computers. (Ajtai 1996 initiated this direction; Google's current post-quantum transition is likely lattice-based.)

13.6 Trapdoor Functions

Definition 13.2: Trapdoor One-Way Function

A *trapdoor function* is a one-way function f equipped with a secret *trapdoor* τ such that, given τ, inverting f is easy. Without τ, inversion is hard.

Trapdoor functions are essential for public-key encryption: the public key encodes f, the private key encodes τ. Anyone can encrypt (apply f); only the key holder can decrypt (invert using τ).

13.7 Finding Large Primes Efficiently

Building RSA requires generating large primes. This is surprisingly easy:

1. By the *prime number theorem*, a random k-bit integer is prime with probability approximately $1/(k \ln 2) = \Theta(1/k)$.

2. Primality testing can be done in polynomial time (AKS, or probabilistic tests like Miller–Rabin).

3. **Algorithm**: repeatedly sample a random k-bit integer and test for primality. Expected number of trials: $O(k)$. Each test runs in polynomial time in k, so the whole procedure is efficient.

In practice, RSA uses primes of 1024 bits each; their product is a 2048-bit modulus. The randomization in key generation is essential: if the same prime were always chosen, an attacker could precompute.

Chapter 14

One-Way Functions and Pseudorandom Generators

This chapter develops the formal theory underlying the primitives introduced in Chapter 13. The material follows Goldwasser–Bellare (GB) Sections 2.4.1, 3.1–3.4, and Barak Section 2.4.

14.1 Formal Definition of One-Way Functions

Definition 14.1: One-Way Function (GB Def. 2.1)

A function $f : \{0,1\}^* \to \{0,1\}^*$ is a *one-way function* if:

1. **Efficiently computable.** There is a polynomial-time algorithm computing $f(x)$.
2. **Hard to invert.** For every PPT adversary $\mathcal{A}$ and every polynomial p, for all sufficiently large k:

$$\Pr_{x \leftarrow \{0,1\}^k}\left[\mathcal{A}(1^k, f(x)) \in f^{-1}(f(x))\right] \;<\; \frac{1}{p(k)}.$$

Unpacking the definition

- The adversary receives 1^k (the security parameter in unary, so the algorithm runs in time polynomial in k) *and* $f(x)$.
- Success means finding *any* preimage of $f(x)$, not necessarily x itself.
- The probability is over the uniform choice of x and the adversary's internal randomness.
- "Hard" means success probability is negligible — smaller than $1/p(k)$ for every polynomial p.

Important warning. OWF hardness is *directional* (easy forward, hard backward), not the same as NP-hardness. OWF existence is an unproven assumption; it implies P $\neq$ NP but is believed to be strictly stronger.

Definition 14.2: One-Way Permutation

A *one-way permutation* is a one-way function $f : \{0,1\}^n \to \{0,1\}^n$ that is also a bijection (one-to-one and onto). Equivalently, f is an efficiently computable, length-preserving permutation of $\{0,1\}^n$ that is hard to invert.

One-way permutations are strictly easier to work with than general one-way functions when constructing PRGs: because f is a bijection, feeding a uniform n-bit seed through f still produces a uniform n-bit output, which lets the hybrid argument go through cleanly.

14.2 Hard-Core Predicates

A one-way function f hides its input x when given $f(x)$, but *how much* does it hide? A *hard-core predicate* extracts a single bit of x that is provably hidden.

Definition 14.3: Hard-Core Predicate (GB Def. 2.48)

A poly-time computable function $B : \{0,1\}^* \to \{0,1\}$ is a *hard-core predicate for* f if for every PPT $\mathcal{A}$ and every polynomial p, for all sufficiently large k:

$$\Pr_{x \leftarrow \{0,1\}^k}\big[\mathcal{A}(f(x), 1^k) = B(x)\big] \;\le\; \frac{1}{2} + \frac{1}{p(k)}.$$

In words: given $f(x)$, the adversary cannot guess $B(x)$ with probability noticeably better than $1/2$ (random guessing).

An alternative but equivalent characterization (GB Def. 2.50):

Definition 14.4: Hard-Core Predicate, Indistinguishability Form (GB Def. 2.50)

B is a hard-core predicate for f if and only if

$$\big(f(x),\, B(x)\big) \;\approx_p\; \big(f(x),\, U_1\big),$$

where U_1 is a uniformly random bit independent of x. That is, the bit $B(x)$ looks like a fresh random bit to any poly-time observer who sees $f(x)$.

14.2.1 The Goldreich–Levin Theorem

The most important result about hard-core predicates is that *every* one-way function has one — we need not search for f-specific tricks.

Theorem 14.1: Goldreich–Levin (GB Thm. 2.49)

Let f be any one-way function. Define

$$f'(x \circ r) \;=\; f(x) \circ r, \qquad x, r \in \{0,1\}^k.$$

(That is, f' takes a $2k$-bit input, outputs $f(x)$ concatenated with r.) Then

$$B(x \circ r) \;=\; \langle x,\, r\rangle \;=\; \sum_{i=1}^{k} x_i r_i \pmod 2$$

is a hard-core predicate for f'.

Why must r be included in the output of f'? To compute $B(x \circ r) = \langle x, r \rangle$, the receiver needs r. So f' reveals r (which is public anyway) while hiding x (since f is one-way). If r were hidden, even the legitimate user could not evaluate B.

Proof sketch (by reduction). Suppose $\mathcal{A}$ predicts $B(x \circ r)$ with advantage $\varepsilon > 1/p(k)$. One constructs a reduction $\mathcal{B}$ that, given $f(x)$, uses $\mathcal{A}$ as a subroutine to recover x with non-negligible probability — contradicting that f is a OWF. The key idea: $\langle x, r \rangle \bmod 2$ is a random parity of x; random r makes all such parities "look independent," so any non-trivial prediction of the parity implies partial knowledge of x.

14.3 Pseudorandom Generators: Definitions

A *pseudorandom generator* (PRG) stretches a short truly random seed into a longer output that is computationally indistinguishable from uniform randomness.

Definition 14.5: Poly-Time Indistinguishability (GB Def. 3.1)

Distributions X and Y over $\{0,1\}^\ell$ are *poly-time indistinguishable*, written $X \approx_p Y$, if for every poly-time distinguisher D:

$$|\Pr[D(X) = 1] - \Pr[D(Y) = 1]| \;=\; \mathrm{negl}(k).$$

Definition 14.6: Pseudorandom Distribution (GB Def. 3.2)

A distribution X over $\{0,1\}^{\ell(k)}$ is *pseudorandom* if $X \approx_p U_{\ell(k)}$, where U_n denotes the uniform distribution on $\{0,1\}^n$.

Definition 14.7: Pseudorandom Generator (GB Def. 3.3)

$G : \{0,1\}^k \to \{0,1\}^{\ell(k)}$ is a *pseudorandom generator (PRG)* if:

1. $\ell(k) > k$ (*stretching*: output is longer than input).
2. G is poly-time computable.
3. $G(U_k) \approx_p U_{\ell(k)}$ (output is pseudorandom).

The input is called the *seed* (length k); $\ell(k)$ is the *expansion factor*.

Why PRGs matter

A PRG lets us replace a long one-time-pad key with a short seed: encrypt as $c = m \oplus G(s)$ for $|m| = \ell(k)$ and seed $|s| = k \ll \ell(k)$. The result is computationally (not perfectly) secret, but secure against all PPT adversaries.

Relationship to Barak Definition 2.10. The PRG security condition $G(U_k) \approx_p U_{\ell(k)}$ is exactly the asymptotic indistinguishability (Definition 2.10 from Barak, see §1.3) applied to the two ensembles $\{G(U_k)\}$ and $\{U_{\ell(k)}\}$.

14.4 Constructing a PRG from a One-Way Function

Theorem 14.2: PRG from OWF (GB Thm. 3.4)

If a one-way function exists, then a pseudorandom generator exists.

The proof is constructive and proceeds in two steps.

14.4.1 Step 1: The G^1 Construction (1-bit stretching)

Definition 14.8: G^1 — one-bit PRG extender

Let f be any OWF and let B be the Goldreich–Levin hard-core predicate for the derived function $f'(x \circ r) = f(x) \circ r$. Define

$$G^1(s) \;=\; f(s) \;\circ\; B(s).$$

Input: seed $s \in \{0,1\}^k$. **Output:** $(k+1)$ bits. G^1 stretches the seed by exactly 1 bit.

Why G^1 is a PRG. We must show $(f(s), B(s)) \approx_p U_{k+1}$. By Definition 2.50 (GB), the hard-core property gives $(f(s), B(s)) \approx_p (f(s), U_1)$. Since f is one-way, $f(s)$ already "looks random" in the relevant sense, so $(f(s), U_1) \approx_p (U_k, U_1) = U_{k+1}$. Chaining these two indistinguishabilities (using transitivity of $\approx_p$), the output of G^1 is pseudorandom.

14.4.2 Step 2: The G^Q Construction (polynomial stretching)

A 1-bit extension is not useful on its own. We chain G^1 to produce polynomially many bits.

Definition 14.9: G^Q — Q-bit PRG

Let $Q = Q(k)$ be any polynomial. Iterate:

1. Set $s_0 = s$ (the seed).
2. For $i = 1, \ldots, Q$: let $s_i = f(s_{i-1})$ and $b_i = B(s_{i-1})$.
3. Output $b_1 \circ b_2 \circ \cdots \circ b_Q$.

Equivalently,

$$G^Q(s) \;=\; B(s_0) \;\circ\; B(s_1) \;\circ\; \cdots \;\circ\; B(s_{Q-1}), \quad s_i = f(s_{i-1}).$$

Output length: $Q(k)$ bits from a k-bit seed.

14.5 The Hybrid Argument: Proving G^Q is a PRG

The hybrid argument — core technique

To show $G^Q(U_k) \approx_p U_Q$, define a sequence of *hybrid distributions* $H_0, H_1, \ldots, H_Q$ interpolating between the two endpoints:

- $H_0 = G^Q(U_k)$ (all Q bits generated by the PRG).
- $H_Q = U_Q$ (all Q bits truly random).
- H_i $(0 < i < Q)$: the first $Q - i$ bits from the PRG, the last i bits truly random.

Key step. Show $H_{i-1} \approx_p H_i$ for each i. Each transition replaces one hard-core bit $B(s_{Q-i})$ with a truly random bit. By the hard-core predicate property (Definition 2.48), no PPT adversary can distinguish these with non-negligible advantage.
Conclusion. $H_0 \approx_p H_1 \approx_p \cdots \approx_p H_Q$, so $G^Q(U_k) \approx_p U_{Q(k)}$ by transitivity.

Error accumulation. Each step introduces indistinguishability error $\mathrm{negl}(k)$. Over $Q(k) = \mathrm{poly}(k)$ steps, total error is $Q(k) \cdot \mathrm{negl}(k) = \mathrm{negl}(k)$ (a polynomial times a negligible function is negligible).

Why adjacent hybrids are indistinguishable. H_{i-1} and H_i differ only in position $Q - i$: H_{i-1} has $B(s_{Q-i})$ (hard-core bit), H_i has a truly random U_1. Any distinguisher D separating H_{i-1} from H_i yields an adversary breaking the hard-core predicate property of B — contradicting OWF hardness. This is a reduction: $D \Rightarrow$ adversary against hard-core predicate $\Rightarrow$ OWF inversion.

Why the chain of implications works

OWF $\xRightarrow{\text{G-L Thm}}$ **Hard-Core Predicate** $\xRightarrow{G^1 \text{ construction}}$ **1-bit PRG** $\xRightarrow{G^Q\text{+hybrid}}$ **poly(k)-bit PRG** $\xRightarrow{\text{XOR with msg}}$ **Computationally secure stream cipher**

14.6 Next-Bit Tests

There is a striking alternative characterization of PRGs: a string looks random if and only if no poly-time algorithm can predict its next bit from its previous bits.

Definition 14.10: Next-Bit Test (GB Def. 3.6)

A poly-time algorithm T is a *next-bit test at position i* if, given the first i bits of a string, T predicts bit $i+1$. T *succeeds* at position i if

$$\Pr\big[T(s_1, \ldots, s_i) = s_{i+1}\big] \;>\; \frac{1}{2} + \frac{1}{p(k)}$$

for some polynomial p and infinitely many k.

Definition 14.11: Next-Bit Security (GB Def. 3.8)

$G : \{0,1\}^k \to \{0,1\}^{\ell(k)}$ *passes all next-bit tests* if for every poly-time T, every polynomial p, all large k, and all $1 \le i < \ell(k)$:

$$\Pr\big[T(G(s)_1, \ldots, G(s)_i) = G(s)_{i+1}\big] \;\le\; \frac{1}{2} + \frac{1}{p(k)},$$

where s is a uniform random seed.

Definition 14.12: Statistical Test (GB Def. 3.9)

A *statistical test* is any poly-time $T : \{0,1\}^\ell \to \{0,1\}$. T *distinguishes* G if $|\Pr[T(G(U_k)) = 1] - \Pr\big[T(U_{\ell(k)}) = 1\big]| \ge 1/p(k)$ for infinitely many k.

Theorem 14.3: Next-Bit Tests ≡ All Statistical Tests (GB Thm. 3.10)

G is a PRG (passes all statistical tests) if and only if G passes all next-bit tests.

Proof sketch (⇒). If G fails a statistical test T — i.e., T distinguishes $G(U_k)$ from $U_{\ell(k)}$ — then there exists a position i such that T distinguishes the distribution of the first $i+1$ bits of G from the first $i+1$ bits of $U_{\ell(k)}$, even given that the first i bits are the same. This directly yields a next-bit predictor.

Significance. To prove G is a PRG, it suffices to show next-bit unpredictability — a simpler, local condition — rather than fooling all possible statistical tests. This is why the hybrid argument works: each step only needs to argue about one bit at a time.

14.7 Two Intuitions for Randomness

Before the formal definition, it is worth grounding the notion of a PRG in two complementary intuitions about what it means for a bit string to "look" random.

Intuition 1: Passes all statistical tests. A random sequence should be unbiased (equal numbers of 0s and 1s in the long run), should have every fixed-length pattern appear with the expected frequency, and should pass any other efficiently computable statistical test. A distinguisher in the formal definition is exactly such a test: it takes a string and says "random" or "not random."

Intuition 2: Unpredictability. Given any prefix of the sequence, a computationally bounded observer should not be able to guess the next bit with probability noticeably better than $\frac{1}{2}$. This is the *next-bit test* view.

Key fact

These two intuitions are equivalent for pseudorandom generators (GB Thm. 3.10, above): G fools all poly-time statistical tests if and only if G passes all next-bit tests.

Why restrict to polynomial-time distinguishers? An exponential-time adversary given a string of length $\ell(n)$ could enumerate all 2^n seeds, compute $G(\text{seed})$ for each, and check whether the string is in the range of G. A random string of length $\ell(n)$ lands in the range with probability at most $2^n/2^{\ell(n)}$, which is negligible when $\ell(n) > n$. So exponential time *can* distinguish; we restrict to polynomial time because that is the realistic adversary model.

(Remark: for other applications — such as *derandomization* and complexity theory — people study pseudorandomness against weaker classes, e.g. log-space bounded tests. The connection to *outcome indistinguishability* from the fairness portion of this course is not a coincidence: OI was framed knowing exactly this type of poly-time indistinguishability formula.)

14.8 Why PRGs Are Practically Useful

True randomness is expensive. Generating truly random bits requires a physical entropy source (thermal noise, radioactive decay, quantum measurement, atmospheric noise, lava lamps, ...). These sources are slow and require special hardware. In contrast, a PRG lets you spend a small amount on a short truly random seed and then *stretch* it into as many pseudorandom bits as needed.

Derandomization. An important open question: does randomness help computation? We do not know whether BPP = P (roughly, whether every efficient randomized algorithm has an efficient deterministic simulation). PRGs are a tool for minimizing the number of truly random bits required by an algorithm, with the hope of eventually eliminating them entirely. If strong enough PRGs exist, BPP = P follows.

PRGs vs. OWFs. A PRG G must certainly be hard to invert (otherwise, given an output, one could recover the seed and distinguish pseudo from truly random). In fact, every PRG is a OWF. The converse is also true: *OWFs exist if and only if PRGs exist* (a deep result; in class we prove the easier direction via the GL theorem and the G^Q construction).

14.9 Hybrid Argument: Proof Structure for Next-Bit ⇔ PRG

The proof that next-bit unpredictability implies full pseudorandomness uses a *hybrid argument* — the second of the two fundamental techniques in cryptography (the first being reductions).

Two fundamental crypto techniques

1. **Reduction:** "If you could do X, you could do Y" — used to derive contradictions.
2. **Hybrid argument:** Interpolate between two distributions through a chain of intermediate hybrids; if the endpoints are distinguishable, some adjacent pair must be distinguishable, reducing to a simpler sub-problem.

Both appeared earlier in this course: reductions throughout crypto; the hybrid argument appeared in the k-fold DP composition proof (walking one database toward another one person at a time).

Construction for this proof. Assume for contradiction that some PPTM $\mathcal{A}$ distinguishes $G(U_k)$ from $U_{\ell(n)}$ with non-negligible advantage $\geq 1/R(n)$. Define $\ell(n)+1$ hybrid distributions $D_0, D_1, \ldots, D_{\ell(n)}$:

$$D_i \;=\; \underbrace{G(s)_1,\ \ldots,\ G(s)_{\ell(n)-i}}_{\text{first } \ell(n)-i \text{ bits: pseudorandom}} \;,\quad \underbrace{r_1,\ \ldots,\ r_i}_{\text{last } i \text{ bits: truly random}} \;.$$

- $D_0 = G(U_k)$ (all pseudorandom — the red distribution).
- $D_{\ell(n)} = U_{\ell(n)}$ (all truly random — the blue distribution).

Since $\mathcal{A}$ distinguishes D_0 from $D_{\ell(n)}$ with advantage $\geq 1/R(n)$, and there are $\ell(n)$ steps in the chain, by averaging there must exist some index i^* such that

$$\left|\Pr[\mathcal{A}(D_{i^*}) = 1] - \Pr[\mathcal{A}(D_{i^*+1}) = 1]\right| \;\geq\; \frac{1}{R(n)\cdot \ell(n)},$$

which is still non-negligible (since $\ell(n)$ is polynomial). But D_{i^*} and D_{i^*+1} differ only in whether position $\ell(n) - i^*$ is a pseudorandom bit $G(s)_j$ or a truly random bit r — which yields a next-bit predictor for position $\ell(n) - i^*$, contradicting our assumption.

Caveat

The conditioning required to extract a next-bit predictor from the adjacent hybrids involves some care (the prefix of the hybrid string is not entirely random). Goldwasser–Bellare carry out the remaining calculation; the structure is the same as the calculation at the start of this section.

14.10 The Blum–Blum–Shub (BBS) Generator

The BBS generator is a concrete, number-theory-based PRG with provable security under the *Quadratic Residuosity* assumption.

Setup. Let p, q be large primes with $p \equiv q \equiv 3 \pmod 4$; let $N = pq$ (*Blum integer*). Let $QR_N = \{x \in \mathbb{Z}_N^* : x = y^2 \bmod N \text{ for some } y\}$ be the quadratic residues. The squaring function $f(x) = x^2 \bmod N$ on QR_N is a one-way permutation under the Quadratic Residuosity Assumption: deciding whether an element of $\mathbb{Z}_N^*$ is a quadratic residue is hard without knowing p or q.

Hard-core predicate. The *least significant bit* $B(x) = x \bmod 2$ is a hard-core predicate for $f(x) = x^2 \bmod N$ under the Quadratic Residuosity Assumption.

Definition 14.13: Blum–Blum–Shub Generator

Seed: $x_0 \in QR_N$ chosen uniformly at random.
Iteration: $x_i = x_{i-1}^2 \bmod N$, output bit $b_i = x_i \bmod 2$.
Output: $b_1 \circ b_2 \circ \cdots \circ b_Q$.

Why BBS is notable.

- **Concrete:** security rests on a specific, well-studied number-theoretic problem (quadratic residuosity $\Leftrightarrow$ factoring).
- **Backward secure:** given all Q output bits and x_Q, one cannot recover earlier states $x_0, x_1, \ldots$ (squaring is one-way).
- **Historically important:** one of the first provably secure PRG constructions.

14.11 Chapter Summary: The Implication Chain

From OWF to stream cipher

1. OWF f exists — (assumed: factoring, discrete log, subset sum, ...)
2. $\Rightarrow$ Hard-core predicate B exists — (Goldreich–Levin, GB Thm. 2.49)
3. $\Rightarrow$ $G^1(s) = f(s) \circ B(s)$ is a 1-bit PRG — (GB §3.2 + Def. 2.50)
4. $\Rightarrow$ G^Q is a poly(k)-bit PRG — (hybrid argument)
5. $\Rightarrow$ Stream cipher $\mathrm{Enc}_s(m) = G^{|m|}(s) \oplus m$ is computationally secure

Chapter 15

Randomness, Weak/Strong OWFs, and the Goldreich–Levin Proof

15.1 Why Randomness Is Essential in Key Generation

Both in the *shared-key* setting (two parties share a secret key) and in the *public-key* setting (one party publishes a public key; any sender can encrypt; only the receiver can decrypt), keys must be generated using randomness.

Quiz question

Why must we randomize when choosing keys? What goes wrong if key generation is deterministic?

Answer. If key generation is deterministic, every party who runs the algorithm produces the *same* key. In the shared-key case, everyone would share the same secret; in the public-key case, everyone's "private" key would be identical and publicly deducible. The requirement is *unpredictability*: different parties must produce different, independently unguessable keys. This mirrors the role of randomness in differential privacy — without it one could reverse-engineer which of two adjacent databases produced a given output.

15.2 Probabilistic Polynomial-Time Machines (PPTMs)

Definition 15.1: Probabilistic Polynomial-Time Machine (PPTM)

A machine M is a *probabilistic polynomial-time machine (PPTM)* if there exists a polynomial Q such that for every input x, $M(x)$ runs in time at most $Q(|x|)$, where $|x|$ denotes the *bit-length* of x. M may flip coins during its execution.

In cryptography, "easy" means computable by a PPTM, and "hard" means no PPTM succeeds (with non-negligible probability).

15.3 Negligible Functions

Definition 15.2: Negligible Function

A function $\nu : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is *negligible* if for every constant $c > 0$ there exists k_c such that for all $k \geq k_c$:

$$\nu(k) \;<\; \frac{1}{k^c}.$$

Equivalently, ν grows more slowly than the inverse of any polynomial. We write $\nu(k) = \mathrm{negl}(k)$.

Examples. 2^{-k}, $2^{-\sqrt{k}}$, $k^{-\log k}$ are negligible. k^{-5} is *not* negligible (it is 1/poly).

Key closure property.

The sum of polynomially many negligible functions is negligible:

$$\mathrm{poly}(k) \cdot \mathrm{negl}(k) \;=\; \mathrm{negl}(k).$$

This is essential for *composition*: if each sub-protocol leaks only negligibly, running polynomially many sub-protocols in sequence still leaks negligibly in total. This is far nicer than differential privacy, where privacy losses add linearly (or sub-linearly via advanced composition, but never remain negligible for free).

15.4 Formalizing "Hard to Invert": Two Attempts

15.4.1 Straw-man definition (too weak)

A first attempt: f is hard to invert if it is *not* the case that there exists a PPTM M such that for all y, $f(M(y)) = y$. Formally:

$$\forall M,\ \exists y \text{ such that } M(y) \notin f^{-1}(y) \text{ or } M \text{ runs in time } > |y|^c.$$

This is **too weak**. It would allow a PPTM to invert f on all but a single string of each length n (for infinitely many n), which is clearly not "hard." The issue is that a finite set of exceptions can be hardcoded into the finite control of the machine, and the definition does not prevent inversion on a $1 - o(1)$ fraction of inputs.

15.4.2 Proper definition: negligible success probability

Definition 15.3: One-Way Function — Strong Form

f is *one-way* if for every PPTM adversary $\mathcal{A}$ and every polynomial Q, for all sufficiently large k:

$$\Pr_{x \leftarrow \{0,1\}^k}\Big[f\Big(\mathcal{A}(1^k, f(x))\Big) = f(x)\Big] \;<\; \frac{1}{Q(k)}.$$

Equivalently, using the negligible notation:

$$\Pr_{x \leftarrow \{0,1\}^k}\big[\mathcal{A}(1^k, f(x)) \in f^{-1}(f(x))\big] \;=\; \mathrm{negl}(k).$$

The adversary receives the security parameter 1^k (in unary, so algorithms run in time polynomial in k) to prevent trivially ruling out constant-time machines.

15.5 Weak versus Strong One-Way Functions

Definition 15.4: Weak One-Way Function

f is a *weak one-way function* if there exists a polynomial Q such that for every PPTM $\mathcal{A}$, for all sufficiently large k:

$$\Pr_{x \leftarrow \{0,1\}^k}\left[\mathcal{A}(1^k, f(x)) \in f^{-1}(f(x))\right] \;\leq\; 1 - \frac{1}{Q(k)}.$$

That is, the adversary *fails to invert with non-negligible probability* — it does not have to fail nearly all the time, just a $1/Q(k)$ fraction of the time.

Comparison.

- **Strong OWF:** success probability is negligible (adversary almost never wins).
- **Weak OWF:** failure probability is non-negligible (adversary sometimes loses).

Note the asymmetry: "success probability $\leq$ negl" is strictly stronger than "failure probability $\geq 1/\text{poly}$". (The latter allows, for example, success probability $1 - 1/n^{22}$, where the adversary is almost always right.)

Theorem 15.1: Weak implies Strong (informal)

If a weak one-way function exists, then a strong one-way function exists.

The construction amplifies hardness by evaluating f on many independent inputs; if the adversary could invert all of them simultaneously, it would need to be almost surely correct on every sub-instance.

15.6 One-Way Functions Do Not Hide All Input Bits

A common misconception: if f is one-way, then all bits of x are hidden given $f(x)$. This is *false*.

Example. Let H be a one-way function on n-bit inputs. Define $F(x \circ r) = H(x) \circ r$ where $x, r \in \{0,1\}^n$.

- F is one-way: given $H(x) \circ r$, we cannot find x (since H is one-way).
- F leaks r entirely — exactly half the input bits are public.

This same construction ($f'(x \circ r) = f(x) \circ r$) appears in the Goldreich–Levin theorem. The point is: being one-way means *some* part of the input is hidden, but the definition does not require hiding every single bit.

Practical warning

Hashing does *not* automatically protect privacy. If a database releases only hash values of sensitive fields, the hash reveals nothing about fields that happen to be hard to invert, but it may expose fields that are easy to guess (e.g., a social security number from a small space). "It's hashed" is not a privacy guarantee without further analysis.

15.7 Hardcore Predicates: Formal Definition

A *predicate* is a function $B : \{0,1\}^* \to \{0,1\}$ (output a single bit; "true" or "false" in logic).

Definition 15.5: Hardcore Predicate

A poly-time computable function $B : \{0,1\}^* \to \{0,1\}$ is a *hardcore predicate for* f if for every PPTM $\mathcal{A}$ there is a negligible function ν such that for all sufficiently large k:

$$\Pr_{x \leftarrow \{0,1\}^k}\left[\mathcal{A}(1^k,\, f(x)) = B(x)\right] \;\le\; \frac{1}{2} + \nu(k).$$

Intuition.

- $B(x)$ is a single bit, so random guessing gives success probability exactly $1/2$.
- A hardcore predicate says: seeing $f(x)$ gives the adversary *no useful hint* for guessing $B(x)$; it can do at most negligibly better than chance.
- If you *know* x, computing $B(x)$ is easy. If you only see $f(x)$, it is as hard as flipping a coin.

15.8 The Goldreich–Levin Hardcore Bit

Theorem 15.2: Goldreich–Levin (1989)

Let $f : \{0,1\}^k \to \{0,1\}^*$ be any one-way function. Define

$$G(x \circ r) \;=\; f(x) \circ r, \qquad x, r \in \{0,1\}^k,$$

and the *inner-product bit*

$$B(x \circ r) \;=\; \langle x, r\rangle \;:=\; \sum_{i=1}^{k} x_i r_i \pmod 2.$$

Then B is a hardcore predicate for G.

Why r is included in the output. To evaluate $B(x \circ r)$, one needs both x (hidden) and r (which must therefore be revealed). So G outputs r in the clear while hiding x via the OWF f.

Geometric intuition. r specifies a random subset of bit positions of x; $B(x \circ r)$ is the XOR of those bits. Although r is public, the parity of the chosen bits still hides x completely.

15.9 Polynomial-Time Indistinguishability (Revisited)

Recall (Chapter 14):

Definition 15.6: Poly-Time Indistinguishability of Ensembles

Two families of distributions $\{X_n\}$ and $\{Y_n\}$ are *poly-time indistinguishable*, written $\{X_n\} \approx_p \{Y_n\}$, if for every PPTM $\mathcal{A}$ and every polynomial R, for all sufficiently large n:

$$\left|\Pr_{t \leftarrow X_n}[\mathcal{A}(t) = 1] \;-\; \Pr_{t \leftarrow Y_n}[\mathcal{A}(t) = 1]\right| \;<\; \frac{1}{R(n)}.$$

Equivalently, the difference is negligible in n.

This is a strong definition

For *every* polynomial-time distinguisher, for *every* polynomial bound on the advantage, the two families cannot be told apart once n is large enough. The existence of pseudorandom generators — sequences indistinguishable from true randomness by any polynomial-time observer — shows this strong definition can actually be achieved under computational assumptions. *Cryptography makes the impossible possible.*

15.10 Proof that the GL Bit Is Hardcore

We prove:

$$\{(G(s),\ B(s))\} \ \approx_p\ \{(G(s),\ 1-B(s))\},$$

where $s = x \circ r$ is drawn uniformly from $\{0,1\}^{2k}$.

Unpacking: the *red* distribution is $G(s)$ concatenated with the hardcore bit $B(s)$; the *blue* distribution is $G(s)$ concatenated with $\overline{B(s)} = 1 - B(s)$. Both draw $s = x \circ r$ with $x, r \leftarrow \{0,1\}^k$ independently.

15.10.1 Proof by contradiction

Assume there exists a PPTM $\mathcal{A}$ and polynomial R such that for infinitely many n:

$$p_n - q_n \ \geq\ \frac{1}{R(n)},$$

where

$$p_n = \Pr_{s\leftarrow\{0,1\}^{2k}}[\mathcal{A}(G(s)\circ B(s)) = 1], \qquad q_n = \Pr_{s\leftarrow\{0,1\}^{2k}}[\mathcal{A}(G(s)\circ \overline{B(s)}) = 1].$$

(We handle the absolute value at the end; WLOG assume the red probability exceeds the blue probability for infinitely many n.)

Construction. Build a PPTM $\mathcal{B}$ that, given $G(s) = f(x) \circ r$, guesses $B(s)$ with probability non-negligibly greater than $1/2$:

1. Receive $G(s)$ (i.e., $f(x) \circ r$).
2. Sample a fresh uniform bit $Z \in \{0,1\}$.
3. Run $\mathcal{A}$ on $G(s) \circ Z$.
4. **If** $\mathcal{A}$ outputs 1, **output** Z (guess $B(s) = Z$).
5. **If** $\mathcal{A}$ outputs 0, **output** $1 - Z$ (guess $B(s) = 1 - Z$).

Analysis.

$$\Pr[\mathcal{B}\text{ correct}] \ =\ \Pr[Z = B(s)]\cdot\Pr[\mathcal{A}(G(s)\circ Z) = 1 \mid Z = B(s)] + \Pr[Z \neq B(s)]\cdot\Pr[\mathcal{A}(G(s)\circ Z) = 0 \mid Z \neq B(s)].$$

Since Z is an independent uniform bit, $\Pr[Z = B(s)] = 1/2$. The conditional on $Z = B(s)$ has the adversary $\mathcal{A}$ receiving a sample from the *red* distribution, giving success probability p_n. The

conditional on $Z \neq B(s)$ has $\mathcal{A}$ receiving the *blue* distribution; when $\mathcal{A}$ outputs 0, we correctly flip, giving probability $1 - q_n$. Therefore:

$$\Pr[\mathcal{B} \text{ correct}] \;=\; \frac{1}{2}\,p_n \;+\; \frac{1}{2}(1 - q_n) \;=\; \frac{1}{2} + \frac{p_n - q_n}{2} \;\geq\; \frac{1}{2} + \frac{1}{2R(n)}.$$

This is non-negligibly better than $1/2$, contradicting the hardcore predicate property of B.

Handling the absolute value. If $q_n > p_n$ for infinitely many n instead, define $\mathcal{B}$ to flip the output of $\mathcal{A}$ (run $1 - \mathcal{A}(\cdot)$ as the distinguisher); this still runs in polynomial time and achieves the same advantage. At least one of the two cases must occur infinitely often, so the argument goes through.

Connection to Barak's notes

This proof is the asymptotic version of the non-asymptotic argument in Barak §2.4. The core idea is identical: a distinguisher with polynomial advantage $p_n - q_n \geq 1/R(n)$ translates directly into a hardcore-bit guesser with advantage $\geq 1/(2R(n))$ above $1/2$.

Remark on P vs. NP. All the hardness assumptions in this chapter (OWF existence, indistinguishability) are *unproven*. If P = NP, polynomial-time algorithms exist for inverting any OWF, and modern public-key cryptography collapses. The probability of P = NP is unknown.

15.11 Candidate One-Way Functions Revisited

15.11.1 Subset Sum Modulo M

Parameters: n elements $a_1, \ldots, a_n \in \mathbb{Z}_M$, target $t \in \mathbb{Z}_M$. The function $f_a(s) = \sum_i a_i s_i \bmod M$ maps $s \in \{0,1\}^n$ to $\mathbb{Z}_M$. Inverting it (finding a subset summing to t) is a modular version of the *knapsack* problem. Hardness depends on the density $\delta = n/\log M$:

- $\delta < 1$ or $\delta > n/\log n$: efficient algorithms exist.
- $\delta = \Theta(1)$: believed hard; related to *lattice-based cryptography*.

The solution need not be unique; one-wayness only requires that *some* preimage is hard to find.

15.11.2 Discrete Logarithm

Let p be a large prime and g a *generator* of $\mathbb{Z}_p^*$, i.e., the powers $g^1, g^2, \ldots, g^{p-1}$ enumerate all elements of $\{1, \ldots, p-1\}$.

Definition 15.7: Discrete Logarithm Problem

Given p, g, and $y \in \mathbb{Z}_p^*$, find x such that $g^x \equiv y \pmod{p}$.

The forward direction (exponentiation: given x, compute $y = g^x \bmod p$) is easy via fast exponentiation. The inverse (given y, find x) is the *discrete log* problem, which is conjectured hard. The famous *ElGamal cryptosystem* is based on this hardness.

15.11.3 Post-Quantum Cryptography

Both integer factoring and discrete log can be solved in *polynomial time on a quantum computer* via Shor's algorithm (1994). If large-scale quantum computers are built, these cryptosystems are broken.

Post-quantum cryptography refers to schemes whose security does not rely on the hardness of factoring or discrete log. The leading candidate is *lattice-based cryptography*, where hardness is based on problems like Learning With Errors (LWE). Interestingly, lattice algorithms (due to Lenstra–Lenstra–Lovász) were originally used to *break* knapsack cryptosystems; Regev and others later showed lattice problems could also be used to *build* secure systems.

15.12 GL Proof: Refined Case Analysis

The following is a more explicit version of the argument from §15.10, using conditional probabilities to make each case transparent.

Setup. As before, $\mathcal{B}$ receives $G(s) = f(x) \circ r$, samples a fresh uniform bit $Z \in \{0,1\}$, and runs $\mathcal{A}$ on $G(s) \circ Z$.

Case 1: $\mathcal{A}$ outputs 1. $\mathcal{B}$ guesses $B(s) = Z$. This is correct iff $Z = B(s)$, which happens with probability $\frac{1}{2}$ (since Z is chosen uniformly, independent of s). Conditioned on $Z = B(s)$, we are feeding $\mathcal{A}$ from the *red* distribution, so

$$\Pr[\mathcal{A}(G(s) \circ Z) = 1 \mid Z = B(s)] \;=\; p_n.$$

Contribution to $\Pr[\mathcal{B}$ correct$]$:

$$\Pr[Z = B(s)] \cdot \Pr[\mathcal{A}(G(s) \circ Z) = 1 \mid Z = B(s)] \;=\; \tfrac{1}{2}\, p_n.$$

Case 2: $\mathcal{A}$ outputs 0. $\mathcal{B}$ guesses $B(s) = 1 - Z$. This is correct iff $Z = 1 - B(s)$ (i.e. $Z \neq B(s)$), which also happens with probability $\frac{1}{2}$. Conditioned on $Z = 1 - B(s)$, we are feeding $\mathcal{A}$ from the *blue* distribution, so

$$\Pr[\mathcal{A}(G(s) \circ Z) = 0 \mid Z = 1 - B(s)] \;=\; 1 - q_n.$$

Contribution to $\Pr[\mathcal{B}$ correct$]$:

$$\Pr[Z = 1 - B(s)] \cdot \Pr[\mathcal{A}(G(s) \circ Z) = 0 \mid Z = 1 - B(s)] \;=\; \tfrac{1}{2}(1 - q_n).$$

Total. By the law of total probability:

$$\Pr[\mathcal{B} \text{ correct}] \;=\; \tfrac{1}{2}p_n + \tfrac{1}{2}(1 - q_n) \;=\; \tfrac{1}{2} + \tfrac{p_n - q_n}{2} \;\geq\; \tfrac{1}{2} + \frac{1}{2R(n)},$$

which is non-negligibly better than $\frac{1}{2}$, contradicting the hardcore property.

15.13 Corollary: Hardcore Bit Is Indistinguishable from a Random Bit

The previous theorem showed:

$$\big\{G(s) \circ B(s)\big\} \;\approx_p\; \big\{G(s) \circ (1 - B(s))\big\}.$$

A useful corollary strengthens this: the hardcore bit is indistinguishable from a *uniformly random* bit.

Theorem 15.3: Hardcore Bit vs. Random Bit

If B is a hardcore predicate for G, then

$$\{G(s) \circ B(s)\}_{s \leftarrow \{0,1\}^{2k}} \approx_p \{G(s) \circ U\}_{s \leftarrow \{0,1\}^{2k},\, U \leftarrow \{0,1\}},$$

where U is a fresh uniform bit independent of s.

Proof. Let $\mathcal{A}$ be any PPTM that distinguishes $X_n = G(s) \circ B(s)$ from $Y_n = G(s) \circ U$. Define

$$p_n = \Pr[\mathcal{A}(X_n) = 1], \quad q_n = \Pr[\mathcal{A}(Y_n) = 1].$$

Decompose Y_n by conditioning on whether $U = B(s)$ or $U = 1 - B(s)$, each with probability $\frac{1}{2}$:

$$q_n \;=\; \tfrac{1}{2}\Pr[\mathcal{A}(G(s) \circ B(s)) = 1] + \tfrac{1}{2}\Pr[\mathcal{A}(G(s) \circ (1 - B(s))) = 1] \;=\; \tfrac{1}{2}\,p_n + \tfrac{1}{2}\,r_n,$$

where $r_n = \Pr[\mathcal{A}(G(s) \circ (1 - B(s))) = 1]$. Therefore:

$$p_n - q_n \;=\; p_n - \tfrac{1}{2}\,p_n - \tfrac{1}{2}\,r_n \;=\; \tfrac{1}{2}(p_n - r_n).$$

By the previously proved theorem, $p_n - r_n = \mathrm{negl}(n)$ (since $G(s) \circ B(s)$ and $G(s) \circ (1 - B(s))$ are already indistinguishable). Hence $p_n - q_n = \mathrm{negl}(n)$, so $\mathcal{A}$ cannot distinguish X_n from Y_n with non-negligible advantage. □

Crypto application: why encryption must be randomized

This corollary is the key insight behind semantic security. Suppose a public-key cryptosystem encrypts messages bit by bit. If encryptions of 0 and encryptions of 1 are computationally indistinguishable, then the exact same argument shows that encryptions of a fixed bit b are indistinguishable from encryptions of a *random* bit U.

Why must encryption be randomized? If encryption were deterministic, there would be exactly one ciphertext for 0 and one for 1; an adversary who sees the public key can compute both and compare — trivially distinguishing. Randomization ensures the set of possible ciphertexts for each plaintext overlaps in a computationally undetectable way.

Chapter 16

Encryption and Semantic Security

16.1 Private-Key Encryption: Syntax

Definition 16.1: Private-Key Encryption Scheme

A *private-key encryption scheme* is a triple $(\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$:

- $\mathrm{Gen}(1^k) \to k$: probabilistic key generation.
- $\mathrm{Enc}_k(m) \to c$: encryption of message m under key k.
- $\mathrm{Dec}_k(c) \to m$: decryption.

Correctness: $\mathrm{Dec}_k(\mathrm{Enc}_k(m)) = m$ for all k, m.

16.2 Perfect Secrecy (Shannon)

Definition 16.1 (Perfect Secrecy)**.** *A scheme has* perfect secrecy *if* $\Pr[M = m \mid C = c] = \Pr[M = m]$ *for all* m, c*, equivalently* $\Pr[\mathrm{Enc}_k(m_0) = c] = \Pr[\mathrm{Enc}_k(m_1) = c]$ *for all* m_0, m_1, c*.*

The **One-Time Pad (OTP)** achieves perfect secrecy: $c = m \oplus k$ for a uniformly random key k of the same length as m.

Shannon's impossibility. Any perfectly secret scheme requires $|K| \geq |M|$ — the key must be at least as long as the message. This makes perfect secrecy impractical, motivating computational security.

16.3 Semantic Security

Definition 16.2: Semantic Security (Goldwasser–Micali)

An encryption scheme $(\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$ is *semantically secure* if for every PPT adversary $\mathcal{A}$, every function f, and every polynomial p, for all sufficiently large k:

$$\Pr\Big[\mathcal{A}(1^k, \mathrm{Enc}_k(m)) = f(m)\Big] \;\leq\; \Pr\Big[\mathcal{A}(1^k) = f(m)\Big] + \frac{1}{p(k)}.$$

An adversary who sees the ciphertext learns nothing about the message beyond what she already knew.

16.4 IND-CPA Security

Definition 16.3: IND-CPA (Indistinguishability under Chosen Plaintext Attack)

A scheme is *IND-CPA secure* if for every PPT adversary $\mathcal{A}$:

$$\left|\Pr[b' = b] - \frac{1}{2}\right| < \frac{1}{\text{poly}(k)},$$

where the experiment is:

1. Generate $k \leftarrow \text{Gen}(1^k)$.
2. $\mathcal{A}$ queries an encryption oracle $\text{Enc}_k(\cdot)$ adaptively.
3. $\mathcal{A}$ submits two messages m_0, m_1.
4. Challenger picks $b \leftarrow \{0,1\}$, sends $c^* = \text{Enc}_k(m_b)$.
5. $\mathcal{A}$ continues to query $\text{Enc}_k(\cdot)$, then outputs b'.

Theorem 16.1: Semantic Security $\iff$ IND-CPA

A private-key encryption scheme is semantically secure if and only if it is IND-CPA secure.

Conceptual importance. Semantic security is the "right" intuitive definition (nothing is leaked), while IND-CPA is the "right" technical definition (clean game, easy to use in reductions). Their equivalence means we can use whichever is convenient.

16.5 Stream Ciphers from PRGs

Definition 16.4: PRG-based Stream Cipher

Let $G : \{0,1\}^k \to \{0,1\}^{\ell(k)}$ be a PRG. Define:

$$\text{Enc}_s(m) = G(s) \oplus m, \qquad \text{Dec}_s(c) = G(s) \oplus c, \qquad m \in \{0,1\}^{\ell(k)}.$$

Theorem 16.2: PRG Stream Cipher is IND-CPA

If G is a PRG, the stream cipher above is IND-CPA secure (for single-message security).

Proof sketch. If $\mathcal{A}$ distinguishes $\text{Enc}_s(m_0)$ from $\text{Enc}_s(m_1)$, it distinguishes $G(s) \oplus m_0$ from $G(s) \oplus m_1$. Since XOR with a fixed message is a bijection, this implies distinguishing $G(s)$ from $U_{\ell(k)}$ — breaking the PRG.

Caution: single-use. If the same seed s is reused to encrypt two messages m, m', the adversary sees $G(s) \oplus m$ and $G(s) \oplus m'$ and can XOR them to recover $m \oplus m'$, completely breaking secrecy.

16.6 Public-Key Encryption

Private-key encryption requires the sender and receiver to share a secret key in advance. This is the *key distribution problem*: how do two parties who have never met establish a shared secret over a public channel?

Public-key encryption (PKE) resolves this by having each party hold a *key pair*: a public key they broadcast freely and a private key they keep secret. Anyone can encrypt to you using your public key; only you can decrypt using your private key.

Definition 16.5: Public-Key Encryption Scheme

A *public-key encryption scheme* is a triple $(\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$:

- $\mathrm{Gen}(1^k) \to (pk, sk)$: generate a public key and a secret key.
- $\mathrm{Enc}_{pk}(m) \to c$: encrypt message m using the public key.
- $\mathrm{Dec}_{sk}(c) \to m$: decrypt using the secret key.

Correctness: $\mathrm{Dec}_{sk}(\mathrm{Enc}_{pk}(m)) = m$ for all valid key pairs and messages.

IND-CPA for PKE. The definition is the same as for private-key IND-CPA, except the adversary is given the *public key* pk at the start. This is actually a stronger requirement: since the adversary has pk, they can encrypt arbitrary messages themselves, so the scheme must remain secure even against adversaries with full encryption access.

Why encryption must be randomized. In a public-key cryptosystem, if encryption were deterministic, any observer could encrypt both messages themselves (the public key is public) and compare against the ciphertext, immediately learning which message was encrypted. A secure scheme must have multiple possible encryptions of each message—i.e., encryption must sample from a distribution. A corollary: if encryptions of 0 are computationally indistinguishable from encryptions of 1, then by the same hybrid argument used in the GL hardcore bit proof, encryptions of any fixed bit b are also indistinguishable from encryptions of a uniformly random bit.

16.7 Quadratic Residues and the QR Hardness Assumption

The Goldwasser–Micali scheme (below) builds on the computational hardness of distinguishing squares from non-squares modulo a composite.

Definition 16.6: Quadratic Residue

Let $N = pq$ be a product of two distinct odd primes. An element $x \in \mathbb{Z}_N^*$ is a *quadratic residue* mod N (written $x \in \mathrm{QR}_N$) if there exists $s \in \mathbb{Z}_N^*$ with $s^2 \equiv x \pmod N$. Otherwise x is a *quadratic non-residue* ($x \in \mathrm{QNR}_N$).

The Jacobi symbol. For $N = pq$, the *Jacobi symbol* $J_N(x) = \left(\frac{x}{p}\right)\left(\frac{x}{q}\right)$ (product of Legendre symbols) can be computed without knowing p and q.

- If $J_N(x) = -1$: x is definitely a QNR.
- If $J_N(x) = +1$: x is *either* a QR or a QNR with "fake" Jacobi symbol (called a *pseudo-square*).

For $N = pq$ with $p \equiv q \equiv 3 \pmod 4$, exactly half of the elements with $J_N(x) = 1$ are genuine QRs and half are pseudo-squares.

QR Hardness Assumption

Given $N = pq$ (where p, q are random k-bit primes with $p \equiv q \equiv 3 \pmod 4$) and $x \in \mathbb{Z}_N^*$ with $J_N(x) = 1$, no PPT algorithm can determine whether $x \in \mathrm{QR}_N$ with non-negligible advantage over $\frac{1}{2}$.

Key multiplicative structure. QR $\times$ QR = QR; QR $\times$ QNR = QNR; QNR $\times$ QNR = QR.
Multiplication in $\mathbb{Z}_N^*$ corresponds to XOR of the "squareness bit."

16.8 Goldwasser–Micali Probabilistic Encryption

Definition 16.7: Goldwasser–Micali (GM) Encryption

Key generation. Choose random k-bit primes $p \equiv q \equiv 3 \pmod 4$, set $N = pq$. Choose $y \in \mathbb{Z}_N^*$ with $J_N(y) = 1$ but $y \in \mathrm{QNR}_N$ (a pseudo-square). Publish $pk = (N, y)$; keep $sk = (p, q)$.

Encryption of a single bit $b \in \{0, 1\}$. Choose $r \leftarrow \mathbb{Z}_N^*$ uniformly at random and output:

$$c = r^2 \cdot y^b \bmod N.$$

- $b = 0$: $c = r^2$ is a random QR.
- $b = 1$: $c = r^2 y$ is a random pseudo-square (QNR with $J_N(c) = 1$).

Decryption. Using (p, q), check whether $c \in \mathrm{QR}_N$. Output 0 if yes, 1 if no.
Multi-bit messages. Encrypt each bit independently with fresh randomness.

Theorem 16.3: GM Security

Under the QR hardness assumption, GM encryption is semantically secure (IND-CPA).

Why it is probabilistic. Different random choices of r produce different ciphertexts for the same message. This is necessary: a deterministic PKE scheme is never IND-CPA secure (the adversary simply encrypts both challenge messages and compares).

Homomorphic property. If $c_1 = \mathrm{Enc}(b_1)$ and $c_2 = \mathrm{Enc}(b_2)$ (with independent randomness), then

$$c_1 \cdot c_2 \bmod N \;=\; \mathrm{Enc}(b_1 \oplus b_2),$$

since $r_1^2 y^{b_1} \cdot r_2^2 y^{b_2} = (r_1 r_2)^2 y^{b_1 + b_2}$ and the QR/QNR parity of $y^{b_1+b_2}$ corresponds to $b_1 \oplus b_2$. GM is thus a *homomorphic* encryption scheme with respect to XOR.

16.9 Chosen-Ciphertext Attacks

The IND-CPA model gives the adversary access to an encryption oracle. A stronger model gives access to a *decryption* oracle as well.

Definition 16.8: CCA Security (Two Variants)

CCA-pre (non-adaptive): The adversary may query the decryption oracle $\mathrm{Dec}_{sk}(\cdot)$ on any ciphertexts of their choice *before* receiving the challenge ciphertext $c^\star$. The oracle is then removed.
CCA-post (adaptive, also called CCA2): The adversary retains access to $\mathrm{Dec}_{sk}(\cdot)$ even *after* receiving $c^\star$, with the single restriction that they may not query $\mathrm{Dec}_{sk}(c^\star)$ directly.

Why CCA-post? Giving the adversary a decryption oracle even after seeing the challenge may seem like overkill. The motivation is that in practice we rarely know exactly what capabilities an attacker has. By proving security against the strongest plausible adversary—one who can decrypt anything except the exact ciphertext they are trying to break—we get a guarantee that holds regardless of how the attacker acquires auxiliary decryption power.

Implication chain. CCA-post $\Rightarrow$ CCA-pre $\Rightarrow$ IND-CPA. Each implication is strict: there exist schemes secure under the weaker model but broken by the stronger.

16.10 Malleability and Non-Malleability

Definition 16.9: Malleable Encryption Scheme

An encryption scheme is *malleable* if an adversary, given a ciphertext $\alpha = \mathrm{Enc}(m)$, can produce a different ciphertext $\beta \neq \alpha$ such that $\mathrm{Dec}(\beta)$ has a *known relationship* to m (e.g., β decrypts to $2m$, or to $1 - m$, or to $m \oplus t$ for a known t).

Malleability is a problem because in a CCA setting the adversary can: (1) receive challenge ciphertext α encrypting unknown m, (2) produce a related β via the malleability attack, (3) submit $\beta \neq \alpha$ to the decryption oracle, (4) learn $\mathrm{Dec}(\beta)$, which reveals information about m.

Definition 16.10: Non-Malleability

A scheme is *non-malleable* if for every PPT adversary $\mathcal{A}$ and every efficiently computable relation R on messages, given a challenge ciphertext $\alpha = \mathrm{Enc}(m)$, $\mathcal{A}$ cannot produce $\beta \neq \alpha$ such that $R(m, \mathrm{Dec}(\beta)) = 1$ with non-negligible probability.

Non-malleability is a *stronger* guarantee than semantic security: any function $f(m)$ that the adversary could learn under a semantic-security break can be recast as a relation $R(m, m') = \mathbf{1}[f(m) = f(m')]$, so non-malleability implies semantic security. The converse fails: a semantically secure scheme can be malleable (as GM demonstrates below).

16.11 The Six Notions of Security

Combining adversary power with break type yields six standard notions:

	CPA	**CCA-pre**	**CCA-post**
Semantic security	IND-CPA	IND-CCA-pre	IND-CCA2
Non-malleability	NM-CPA	NM-CCA-pre	NM-CCA2

These are ordered by strength: NM-CCA2 is the strongest and IND-CPA the weakest. For most practical protocols (e.g., TLS), IND-CCA2 security is the target.

16.12 The GM Scheme is Malleable

Goldwasser–Micali is IND-CPA secure (Theorem 16.3), but it is *malleable* under CCA, because the multiplicative structure of $\mathbb{Z}_N^*$ is public.

Attack: produce another encryption of the same bit. Given challenge $\alpha = r^2 \cdot y^b \bmod N$ (an encryption of bit b): choose a fresh random $s \in \mathbb{Z}_N^*$ and set $\beta = s^2 \cdot \alpha \bmod N$. Since s^2 is a QR, multiplying by it preserves quadratic residuosity:

$$\beta = (sr)^2 \cdot y^b \bmod N,$$

which is a fresh, independently distributed encryption of b. The adversary can submit $\beta \neq \alpha$ to the decryption oracle and learns b.

Attack: produce an encryption of the flipped bit. Set $\beta = (-1) \cdot \alpha \bmod N$. Since -1 is a pseudo-square (QNR with Jacobi symbol $+1$) by the choice of $p \equiv q \equiv 3 \pmod 4$, multiplying flips QR $\leftrightarrow$ QNR:

$$\beta \text{ is an encryption of } 1 - b.$$

Decrypting β via the CCA oracle reveals $1 - b$, hence b.

Takeaway

The GM scheme's homomorphic property (XOR on ciphertexts) is what makes it useful for certain applications—but it is also exactly what makes it malleable. Achieving NM-CCA2 security requires abandoning homomorphism or adding non-interactive zero-knowledge proofs of well-formedness (as in the Cramer–Shoup scheme).

Chapter 17

Zero Knowledge

17.1 From Classical Proofs to Interactive Proofs

Classical proofs. A classical mathematical proof is a checkable sequence of statements: fix axioms and rules of inference, and every line is either an axiom or the result of applying a deduction rule to earlier lines. This is the setting of Euclidean geometry, Peano arithmetic, and so on. (By Gödel's incompleteness theorem, no such system can be simultaneously complete and consistent — but this will not concern us here.)

NP and polynomial witnesses. The complexity class **NP** formalizes classical proofs: a language L is in NP if membership has a *polynomial-length, efficiently checkable* proof. Formally, $x \in L$ iff $\exists\, w$ with $|w| = \text{poly}(|x|)$ and $R(x, w) = 1$ (where R is polynomial-time).

- **3-SAT**: w is a satisfying variable assignment; checking is linear time.
- **3-coloring**: w is a coloring; checking (no edge monochromatic) is linear.
- **Graph isomorphism**: w is the map $\varphi : V(G) \to V(H)$; checking is easy.
- **Hamiltonicity**: w is the Hamiltonian cycle; checking is linear.

The witness w may not be unique: many satisfying assignments may exist for the same formula.

The limitation: co-NP. Proving that a statement is *false* — a formula is *un*satisfiable, two graphs are *non*-isomorphic, a graph has *no* Hamiltonian cycle — amounts to a "for all" quantifier. There is no known short proof of such statements (that would imply P = NP or NP = co-NP). Intuitively, it is easier to *demonstrate* a satisfying assignment than to argue that *none exists*.

Interactive proofs. In an *interactive proof system*, the verifier is not passive. Instead of receiving a proof and checking it line by line, the verifier asks questions. This is particularly powerful for statements outside NP.

Definition 17.1 (Interactive Proof System)**.** *An* interactive proof system *for language L is a protocol between a prover P and verifier V satisfying:*

- ***Completeness:*** *if $x \in L$, V accepts with probability $\geq 2/3$.*
- ***Soundness:*** *if $x \notin L$, no prover P^* makes V accept with probability $> 1/3$.*

By repeating the protocol k times and accepting only if all rounds accept, completeness can be boosted to $1 - 2^{-\Omega(k)}$ and soundness error reduced to $2^{-\Omega(k)}$.

17.2 Graph Non-Isomorphism: An Interactive Proof

Graph isomorphism (GI) is in NP: prove $G \cong H$ by exhibiting the isomorphism. But graph *non*-isomorphism (GNI) is not known to be in NP — there is no known short proof that two graphs are *not* isomorphic. Nevertheless, a powerful prover can convince a verifier interactively.

Protocol (one round).

1. V picks $X \in \{G, H\}$ uniformly at random, computes a uniformly random isomorphic copy $Y = \varphi(X)$, and sends Y to P.
2. P identifies which of G or H the graph Y was derived from, and sends its guess $\hat{b}$ to V.
3. V accepts iff $\hat{b}$ equals its original choice.

Repeat k times independently.

Analysis.

- **Completeness.** If $G \ncong H$, a computationally powerful P can always determine which graph Y came from (since $\varphi(G)$ and $\varphi(H)$ are non-isomorphic). Accepts with probability 1.
- **Soundness.** If $G \cong H$, then $\varphi(G)$ and $\varphi(H)$ have the same distribution. Any prover guesses correctly with probability at most $\frac{1}{2}$ per round; after k rounds, a cheating prover wins with probability at most 2^{-k}.

GNI has an interactive proof even though it is not known to be in NP. Interactive proofs are provably more powerful than classical NP proofs (under standard assumptions).

17.3 Zero-Knowledge Proofs: Definition and Motivation

In practice, the prover often holds a *secret witness* — a satisfying assignment, a graph coloring, a Hamiltonian cycle, a private key — and wants to convince the verifier that the statement is true *without revealing the witness.*

Why keep the witness secret?

- A satisfying assignment for a hard formula can serve as a password. The account holder wants to prove they know it without transmitting it.
- A graph coloring or Hamiltonian cycle may contain proprietary information.
- In lattice-based and other cryptographic settings, the witness *is* the private key; revealing it would break the system entirely.

Definition 17.1: Zero-Knowledge Proof (Goldwasser–Micali–Rackoff)

An interactive proof (P, V) for L is *(computational) zero knowledge* if for every PPT cheating verifier V^* and every auxiliary input z, there exists a PPT *simulator* S such that for all $x \in L$:

$$\mathrm{View}_{V^*}(P(x, w),\, V^*(x, z)) \;\overset{c}{\approx}\; S(x, z),$$

where View_{V^*} is the full transcript of the interaction seen by V^*.

Interpretation. The simulator has no access to the prover or the witness w, yet it produces transcripts computationally indistinguishable from real interactions. Since the verifier can generate equally good transcripts on its own, real transcripts cannot be leaking anything about w. This is the exact analogue of semantic security: seeing the ciphertext (or the transcript) gives the adversary no advantage.

17.4 The Lady Tasting Tea

The statistician Muriel Bristol claimed to distinguish tea poured milk-first from tea-first. In a blind test with 8 cups, she identified all 8 correctly — an event with probability 2^{-8} if guessing at random. The test proves she has the ability, but reveals nothing about *how*: no information about her taste physiology, sensitivity, or method is ever disclosed.

This is an informal zero-knowledge proof of capability: the verifier is convinced of the fact, but learns nothing about the witness behind it.

17.5 Commitment Schemes

Physical intuition: the sealed envelope

You write your prediction on paper, seal it in an envelope, and hand it to the verifier. Before opening, the verifier has no idea what is inside (*hiding*). Once handed over, you cannot change the contents (*binding*). When opened, the verifier can check your claim.

Definition 17.2: Commitment Scheme

A *commitment scheme* has two phases:

1. **Commit phase.** The sender transmits $\mathrm{com}(v; r)$ (where r is fresh randomness) to the receiver. The sender is now bound to v, but the receiver learns nothing about it.
2. **Open phase.** The sender reveals (v, r); the receiver verifies.

Hiding: A PPT receiver cannot distinguish $\mathrm{com}(0; r)$ from $\mathrm{com}(1; r)$ during the commit phase.
Binding: The sender cannot produce (v', r') with $v' \neq v$ opening the same commitment.

Information-theoretic vs. computational. Each of hiding and binding can hold either information-theoretically (no algorithm, regardless of power, can break it) or computationally (no PPT algorithm can break it under a hardness assumption).

Fundamental impossibility

No commitment scheme can be *simultaneously* information-theoretically hiding *and* information-theoretically binding. All other three combinations are achievable.

Example (QR-based). Commit to bit b by sending a random QR (if $b = 0$) or QNR with $J_N = 1$ (if $b = 1$) modulo N. Binding is information-theoretic (a number is or is not a square; this cannot be changed). Hiding is computational: under the QR assumption, QRs and pseudo-squares are indistinguishable in poly time.

17.6 Interactive Proof for Quadratic Non-Residuosity

Both P and V know $X \in \mathbb{Z}_N^*$ with $J_N(X) = 1$. The prover claims $X \in \mathrm{QNR}_N$ and knows the factorization of N.

Protocol (one round).

1. V picks $S \leftarrow \mathbb{Z}_N^*$ and $b \leftarrow \{0,1\}$ uniformly. Sends:

$$Y = \begin{cases} S^2 \bmod N & b = 0 \\ S^2 X \bmod N & b = 1 \end{cases}$$

2. P identifies b (using the factorization to test whether $Y \in \mathrm{QR}_N$) and sends $\hat{b}$ to V.

3. V accepts iff $\hat{b} = b$.

Analysis.

- **Completeness.** If X is a QNR: S^2 is a QR; S^2X is a QNR. A powerful P always identifies b correctly.

- **Soundness.** If X is a QR: S^2 and S^2X are both QRs, identically distributed. No prover can distinguish them; cheating succeeds with probability $\frac{1}{2}$ per round. After k rounds: 2^{-k}.

17.7 Zero-Knowledge Proof for Quadratic Residuosity

Now P claims $X \in \mathrm{QR}_N$ and knows s with $s^2 \equiv X \pmod{N}$. The goal is to prove this without revealing s.

Protocol (one round).

1. P picks $T \leftarrow \mathbb{Z}_N^*$ uniformly. Sends $\alpha = T^2 X \bmod N$.

2. V picks $b \leftarrow \{0,1\}$ and sends b to P.

3. P sends:

$$Y = \begin{cases} TS \bmod N & b = 0 \\ T \bmod N & b = 1 \end{cases}$$

4. V checks:

$$\begin{cases} Y^2 \equiv \alpha \pmod{N} & b = 0 \\ Y^2 X \equiv \alpha \pmod{N} & b = 1 \end{cases}$$

Correctness.

- $b = 0$: $Y = TS$, so $Y^2 = T^2 S^2 = T^2 X = \alpha$. ✓

- $b = 1$: $Y = T$, so $Y^2 X = T^2 X = \alpha$. ✓

Zero knowledge via the rewind simulator. The real transcript distribution is:

- $b = 0$: $(Y^2,\ 0,\ Y)$ for uniform $Y = TS$ (which is uniform since T is uniform).
- $b = 1$: $(T^2X,\ 1,\ T)$ for uniform T.

The secret s is always multiplied by a fresh uniform T; the verifier never sees s directly.

The simulator proceeds:

1. Flip a coin to guess $\hat{b} \in \{0, 1\}$.
2. Pick uniform $Y' \in \mathbb{Z}_N^*$.
3. If $\hat{b} = 0$: set $\alpha' = (Y')^2$; prepare to send Y'.
   If $\hat{b} = 1$: set $\alpha' = (Y')^2 X$; prepare to send Y'.
4. Send α' to V^* and receive b_{actual}.
5. If $b_{\text{actual}} = \hat{b}$: output the transcript $(\alpha', b_{\text{actual}}, Y')$.
6. Otherwise: **rewind** V^* to its state before step 4 and retry.

In expectation, only 2 attempts suffice per round. In either case, the produced transcript is identically distributed to a real interaction: $b = 0$ gives $(Y^2, 0, Y)$ for uniform Y; $b = 1$ gives $(Y^2X, 1, Y)$ for uniform Y.

17.8 Zero-Knowledge Proof of Hamiltonicity

A graph $G = (V, E)$ is *Hamiltonian* if it contains a cycle visiting every vertex exactly once. Hamiltonicity is NP-complete, so the Hamiltonian cycle C is the witness. The following protocol (from Barak's notes, with the commitment scheme abstracted) convinces the verifier that G is Hamiltonian without revealing C.

Protocol (one round)

1. **Commit.** P picks a uniformly random permutation π of $V(G)$ and forms $H = \pi(G)$ (same edges, vertices relabeled by π). P sends a *commitment* to each entry of the $|V| \times |V|$ adjacency matrix of H, one bit per entry.
2. **Challenge.** V picks $b \leftarrow \{0, 1\}$ and sends b to P.
3. **Response.**
   - $b = 0$: P opens *all* commitments and reveals π. V checks that H is a valid permutation of G.
   - $b = 1$: P opens only the entries of the adjacency matrix of H corresponding to the permuted cycle $\pi(C)$ (i.e., the n edges $\pi(c_1)\pi(c_2),\ \pi(c_2)\pi(c_3),\ \ldots$). V checks these entries form a Hamiltonian cycle in H.

Repeat k times with fresh randomness from P each round.

Correctness and Soundness

- **Completeness.** An honest P with cycle C passes both checks.
- **Soundness.** If G has no Hamiltonian cycle, no P^* can simultaneously prepare commitments that pass both checks. Passing $b = 0$ requires a valid permutation of G (which contains no Hamiltonian cycle); passing $b = 1$ requires opening a Hamiltonian cycle — which does not exist. A cheating prover passes at most one check per round: soundness error $\leq \frac{1}{2}$ per round, 2^{-k} after k rounds.

Zero Knowledge: The Rewind Simulator

Simulator for the Hamiltonicity protocol

The simulator S for each round:

1. **Guess the challenge.** Flip a coin: guess $\hat{b} \in \{0, 1\}$.
2. **Prepare commitments.**
   - $\hat{b} = 0$: commit to the adjacency matrix of a genuine random permutation $\pi(G)$ (can open the whole matrix to pass $b = 0$).
   - $\hat{b} = 1$: commit to a matrix with 1s on an arbitrary Hamiltonian cycle and arbitrary values elsewhere (can open the cycle to pass $b = 1$).
3. **Run the (possibly cheating) verifier** V^* and receive b_{actual}.
4. **If** $b_{\text{actual}} = \hat{b}$**:** open the prepared commitments and output the transcript.
   If $b_{\text{actual}} \neq \hat{b}$**:** rewind V^* to the start of this round and repeat from step 1.

In expectation, 2 attempts suffice.

The verifier sees either the whole permuted graph (but no cycle information) or just the cycle positions (but not the rest of the graph) — never both at once. The commitment scheme's hiding property ensures that unopened entries reveal nothing. The Hamiltonian cycle C is never disclosed.

17.9 Zero-Knowledge Proof for 3-Colorability

The canonical theoretical example: every NP language has a zero-knowledge proof.

Protocol. Given graph $G = (V, E)$ and a 3-coloring χ known to the prover:

1. P picks a uniformly random permutation π of $\{1, 2, 3\}$, applies it to get coloring $\pi \circ \chi$, and commits to each vertex's color: sends $\{\text{Commit}(\pi(\chi(v)))\}_{v \in V}$.
2. V challenges with a random edge $(u, w) \in E$.
3. P opens the commitments for u and w, revealing $\pi(\chi(u))$ and $\pi(\chi(w))$.
4. V accepts iff the two revealed colors are distinct.

Repeat $|E| \cdot k$ times.

Analysis.

- **Completeness.** A valid coloring always passes (adjacent vertices have distinct colors; π preserves distinctness).
- **Soundness.** If G is not 3-colorable, some edge must be monochromatic; the verifier catches it with probability $\geq 1/|E|$ per round. After $|E| \cdot k$ rounds, soundness error is $\leq (1-1/|E|)^{|E|k} \leq e^{-k}$.
- **Zero knowledge.** The simulator (without knowing χ) generates a random pair of distinct colors and fake commitments for the challenged edge, producing an identically distributed transcript. The random permutation π ensures the revealed colors look fresh and uniform.

Theorem 17.1: NP $\subseteq$ ZK (under computational assumptions)

Under the assumption that one-way functions exist, every language in NP has a computational zero-knowledge interactive proof system.

This follows from 3-colorability having a ZK proof, combined with the NP-completeness of 3-coloring (via Cook–Levin and reduction): any NP language reduces to 3-coloring in polynomial time.

Connection to commitment schemes. The ZK protocols above rely on commitments: binding ensures soundness (the prover cannot change the committed coloring or permutation after the challenge); hiding ensures zero knowledge (unopened commitments reveal nothing). Commitment schemes can be constructed from one-way functions.

17.10 Post-Quantum Cryptography: Lattice-Based Schemes

The cryptographic constructions discussed in this course — factoring-based (Goldwasser–Micali) and discrete-log-based systems — are *not* post-quantum secure. Shor's algorithm solves both integer factorization and discrete logarithm in polynomial time on a quantum computer.

Lattice problems offer a post-quantum alternative. Ajtai (1996) introduced lattice problems with a remarkable *worst-case to average-case reduction*: solving random instances efficiently would imply solving the *hardest* instances efficiently. This gives a provable guarantee that factoring lacks: random instances are provably as hard as the worst case. Moreover, no efficient quantum algorithm for lattice problems is known.

Why lattice-based cryptography matters

- **Post-quantum:** no known quantum speedup.
- **Worst-case hardness:** security provably tied to hard worst-case instances, not just empirical hardness of random ones.
- **Versatility:** supports fully homomorphic encryption and other advanced primitives not achievable from factoring alone.

17.11 ZK-SNARKs: Succinct Non-Interactive Zero Knowledge

17.11.1 Motivation: Privacy in Distributed Systems

Bitcoin introduced a pseudonymous transaction model: every user operates under a public key rather than a name. But pseudonymity is not anonymity. Because the blockchain is a public

ledger, an observer can link transactions to the same pseudonym, and then link pseudonyms to identities via auxiliary information — precisely the linkage-attack strategy that defeated Netflix's "anonymous" ratings (see Chapter 11).

ZK-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) were developed, in part, to provide genuinely private transactions: a user proves they are authorized to spend funds without revealing which funds, in which wallet, at what time.

17.11.2 The SNARK Acronym Unpacked

Zero-Knowledge (ZK): the verifier learns nothing beyond the truth of the statement (the simulator paradigm of Goldwasser–Micali–Rackoff).

Succinct (S): the proof has size $\text{poly}\log N$, where N is the length of the witness, rather than $\text{poly}(N)$.

Non-Interactive (NI): no back-and-forth between prover and verifier; a single message suffices (achieved via the Fiat–Shamir heuristic).

Argument (A): soundness holds only against computationally bounded provers (under cryptographic assumptions), not information-theoretically.

of Knowledge (K): the prover does not merely show a witness *exists*; the protocol *extracts* the witness from any successful prover, proving that the prover genuinely "knows" it.

17.11.3 Construction: PCP + Merkle Tree + Fiat–Shamir

The construction proceeds in three steps:

Step 1: Probabilistically Checkable Proof (PCP). By the PCP theorem, any NP witness w of length N can be encoded into a *probabilistically checkable proof* π of length $\text{poly}(N)$ such that a verifier who reads only $\text{poly}\log N$ *randomly chosen* positions of π can check validity with high probability. This is remarkable: the verifier does not read the whole proof.

Step 2: Merkle Tree for Commitment and Selective Opening. Naively, the prover would send all of π to the verifier, which is $\text{poly}(N)$ bits. Instead:

1. The prover *commits* to π by building a Merkle tree over the leaves $\pi_1, \ldots, \pi_{\text{poly}(N)}$ and sending only the root hash $h = \text{MerkleRoot}(\pi)$.

2. The verifier requests $\text{poly}\log N$ leaf positions.

3. The prover opens each requested leaf with an authentication path (a sequence of sibling hashes of length $O(\log N)$).

Each opening is itself a proof of membership in the Merkle tree, which can be verified against the root h in $O(\log N)$ hash evaluations. The prover then gives a *zero-knowledge interactive proof* for each opening, ensuring no information about the unopened leaves is revealed.

Step 3: Fiat–Shamir Heuristic. The protocol so far is interactive (the verifier sends random challenges). The **Fiat–Shamir heuristic** replaces every verifier message with the output of a cryptographic hash function applied to the transcript so far:

$$c_i \;=\; H(\text{transcript up to round } i).$$

This makes the protocol *non-interactive*: the prover computes the "challenges" herself using the hash, then sends the entire transcript in one message. Security holds in the *random oracle model*, where H is modeled as a truly random function.

Theorem 17.2: ZK-SNARK Witness Size

Using PCP + Merkle commitment + Fiat–Shamir, the resulting proof has

$$|\text{proof}| = \text{poly}\log N,$$

where N is the length of the original NP witness. The verifier runs in $\text{poly}\log N$ time.

Why this is remarkable. For NP statements with witnesses of length, say, $N = 10^9$ bits, a classical interactive ZK proof would require the verifier to process $\text{poly}(10^9)$ data. A ZK-SNARK reduces this to $\text{poly}\log(10^9) \approx 900$ bits. The verifier's work is essentially *independent of the witness size*.

17.12 Proofs of Work: Origins and Design

17.12.1 The Original Problem: Spam Deterrence

Proofs of work were invented in 1992 by **Dwork and Naor** for a purpose entirely unrelated to cryptocurrency: *combating email spam*.

The core observation: sending an email is essentially free for the sender but costly (in attention) for the recipient. If sending a single email required even a small amount of *computation* — say, 10 seconds of CPU time — then legitimate users would barely notice, but a spammer sending millions of emails would face an insurmountable computational burden.

17.12.2 Moderately Hard Functions

The technical object is a *moderately hard function*: a function f such that

- Computing $f(x)$ requires a specified amount of work (not too easy, not too hard).
- Verifying $f(x) = y$ is fast (much cheaper than computing f).
- The work is *per-message, per-recipient*: the solution to one puzzle cannot be reused for a different message or a different recipient. This non-reusability is essential — without it, a spammer could solve one puzzle and send to a million recipients.

17.12.3 The Puzzle Structure

In Dwork and Naor's scheme, the puzzle for a message m sent to recipient r is:

$$\text{find } x \text{ such that } H(\underbrace{m, r,}_{\text{binding}}\ x) < T,$$

where H is a hash function and T is a difficulty threshold. The message m and recipient r are bound into the puzzle, preventing reuse.

Trapdoor for legitimate bulk mailers. Dwork and Naor's original scheme included a *trapdoor*: a trusted authority could issue "puzzle masters" who can solve puzzles in $O(1)$ time using a private key. This allowed legitimate bulk mailers (e.g., newsletters) to send efficiently while still imposing cost on unauthorized spammers.

17.12.4 Bitcoin Diverges from the Original Design

Bitcoin's proof-of-work mechanism shares the high-level structure (find a nonce x such that $H(\text{block header}, x) < T$) but differs in two important ways:

1. **No trapdoor.** Bitcoin's hash-based PoW has no shortcut: every miner, including the protocol's creators, must perform full brute-force search. This is a deliberate design choice for decentralization.

2. **Environmental cost.** Without a trapdoor, *all* computation is wasted work. Bitcoin's global mining network consumes electricity comparable to small countries. The Dwork–Naor design explicitly avoids this: legitimate senders with the trapdoor pay negligible cost; only spammers pay.

17.12.5 Implementation: Microsoft Outlook

Microsoft implemented a proof-of-work filter in Outlook: outgoing emails would include a small computational certificate, computed silently in the background. Mail servers could weight incoming mail higher or lower based on whether a valid certificate was present. This is a direct descendant of the Dwork–Naor design.

Key distinction

Dwork–Naor (1992): moderately hard, *per-message-per-recipient*, *trapdoor exists* for authorized bulk senders. Bitcoin (2008): moderately hard, *global race*, *no trapdoor*, no trusted authority — at the cost of enormous energy use.

17.13 Paper Readings

Goldwasser, Micali, Rackoff (1989) — *The Knowledge Complexity of Interactive Proof Systems.* Introduces interactive proof systems and defines zero knowledge via the simulator paradigm: a proof is zero-knowledge if a polynomial-time simulator, without access to the prover or the witness, can produce transcripts computationally indistinguishable from real interactions. The paper proves that graph isomorphism has a zero-knowledge proof and introduces the notion of "knowledge complexity" to quantify how much information a proof leaks. The simulator paradigm established here is now the standard framework for defining security in virtually every area of modern cryptography, from encryption to multi-party computation.

Kilian (1992) — *A Note on Efficient Zero-Knowledge Proofs and Arguments.* Combines probabilistically checkable proofs, Merkle tree commitments, and zero-knowledge proofs of knowledge into the first succinct argument system: the prover Merkle-commits to a PCP encoding of the witness, the verifier queries a polylogarithmic number of positions, and the prover responds with a ZK proof that the PCP verifier would accept the committed values — without ever revealing the PCP itself. Communication is polylog($|T|$) rather than polynomial, a dramatic improvement over raw ZK proofs. This construction is the conceptual ancestor of SNARKs, STARKs, and the shielded transaction system of Zcash discussed in Chapter 18.

Chapter 18

Bridging Fairness and Privacy

18.1 Bitcoin and Blockchain: A Case Study in Distributed Trust

18.1.1 The Double-Spend Problem

Digital money faces a fundamental problem that physical cash does not: a digital file representing a coin can be copied. Alice could send Bob a digital coin and simultaneously send Carol the same coin. *Digital signatures* solve part of the problem — they prove Alice authorized a transaction — but they do not prevent Alice from signing two conflicting transactions.

Prior attempts. Pre-Bitcoin digital currency schemes (e-cash, DigiCash) relied on a trusted central server to maintain a "spent coins" list. This is exactly what a bank does. Nakamoto's insight was to eliminate the trusted central party entirely.

18.1.2 Blockchain as a Distributed Timestamp Server

A **blockchain** is an append-only linked list of *blocks*, where each block

1. Contains a batch of transactions,
2. Contains the cryptographic hash of the previous block (creating a *chain*: modifying any past block changes all subsequent hashes), and
3. Contains a proof-of-work certificate.

Because blocks are linked by hashes, an adversary who wants to alter a historical transaction must recompute *all* subsequent blocks' proofs of work — an infeasible task if the adversary controls less than half the network's compute power.

18.1.3 Proof of Work as Byzantine Fault Tolerance

The core mechanism is the proof of work (see Section 17.12 for the 1992 origins). Miners compete to extend the chain: the first to find a valid nonce broadcasts the new block and receives the block reward. By the **longest-chain rule** (Nakamoto consensus), nodes accept the chain with the most cumulative work as the canonical history.

Why this prevents double-spend. Suppose Alice broadcasts conflicting transactions T_1 (to Bob) and T_2 (to Carol). Both transactions propagate through the network, but only one can appear in the canonical chain. Nakamoto consensus resolves the conflict: whichever transaction gets mined first into the longest chain wins; the other is discarded. Alice would need to out-mine the honest network (controlling $> 50\%$ of compute power) to retroactively rewrite history — a *51% attack*.

Security via gambler's ruin. If an attacker controls fraction $q < 1/2$ of compute power and an honest transaction has been confirmed with k blocks of work on top of it, the probability that the attacker can reverse the transaction converges to $(q/(1-q))^k$, which is negligible for modest k (e.g., Bitcoin conventionally waits for 6 confirmations $\approx$ 60 minutes).

18.1.4 Incentive Mechanism

Nakamoto designed a self-sustaining incentive structure:

- **Block reward:** the miner who finds a valid block creates new coins (initially 50 BTC, halving every 210,000 blocks), funding their own computational cost.
- **Transaction fees:** users can attach fees to transactions; miners prioritize high-fee transactions, creating a market for block space.

This aligns rational miners' interests with honest participation: cheating (e.g., accepting double-spends) would undermine confidence in Bitcoin and destroy the value of the very coins they just earned.

18.1.5 Merkle Trees for Efficient Verification

Each block header commits to its transaction set via a **Merkle tree**: transactions are leaves; each internal node is the hash of its two children; the root hash is stored in the block header (a single 32-byte value regardless of how many transactions are in the block).

Practical benefit. A *light client* (e.g., a mobile wallet) does not store the full blockchain. To verify that transaction T is in block B, it only needs the root hash (already stored in the block header) and the $O(\log n)$ sibling hashes along the path from T's leaf to the root — an *authentication path*. Full transaction data can be pruned after a block is sufficiently buried.

18.1.6 Privacy Limitations: Pseudonymity vs. Anonymity

Bitcoin's white paper describes a "privacy model" based on pseudonymity: instead of using real names, users operate under public keys. But this is not anonymity.

Linkage attacks. Because the blockchain is a *public ledger*, every transaction is visible. An adversary who can link one public key to a real identity (e.g., via an exchange that requires KYC, or via IP address logging when a transaction is first broadcast) can trace the graph of all that key's transactions. Further, inputs to a transaction are often assumed to belong to the same wallet, allowing clustering.

Connection to Netflix de-anonymization. This is the same structural problem as the Netflix Prize dataset (Chapter 11): releasing "anonymous" records with rich structure provides sufficient auxiliary information for re-identification. The blockchain is maximally rich in structure — a permanent, global, perfectly accurate record of every transaction ever made.

18.1.7 ZK-SNARKs as a Privacy-Preserving Upgrade

The pseudonymity failure of Bitcoin motivated the design of privacy coins such as Zcash, which use ZK-SNARKs (Section 17.11) to allow *shielded transactions*: a user proves in zero knowledge that

1. They own a valid coin (i.e., know a secret key for a coin in the Merkle tree of unspent outputs), and

2. They are not double-spending (the nullifier for this coin has not been revealed before),

without revealing *which* coin, *which* key, or *how much* is being transferred. The SNARK proof is succinct ($\text{poly}\log N$) and non-interactive, making it compatible with blockchain's trust model.

Course connection

Bitcoin illustrates the interplay of all three course themes:

- **Privacy:** pseudonymity without DP or ZK is insufficient; linkage attacks succeed as they did on Netflix.
- **Cryptography:** the system is built from digital signatures, hash functions, Merkle trees, and (in Zcash) ZK-SNARKs — every tool from this course's second half.
- **Fairness:** the incentive mechanism (Nakamoto consensus, selfish mining) raises questions about who controls the protocol and whether "one CPU, one vote" concentrates power in mining pools, mirroring algorithmic fairness concerns about concentration of decision-making power.

Selfish mining. One known attack on Nakamoto consensus is *selfish mining*: a miner with fraction $q > 1/3$ of compute power can earn disproportionate rewards by strategically withholding found blocks and releasing them to fork the honest chain. This violates the assumption that rational miners always broadcast blocks immediately, and is an active research area connecting mechanism design to cryptographic protocols.